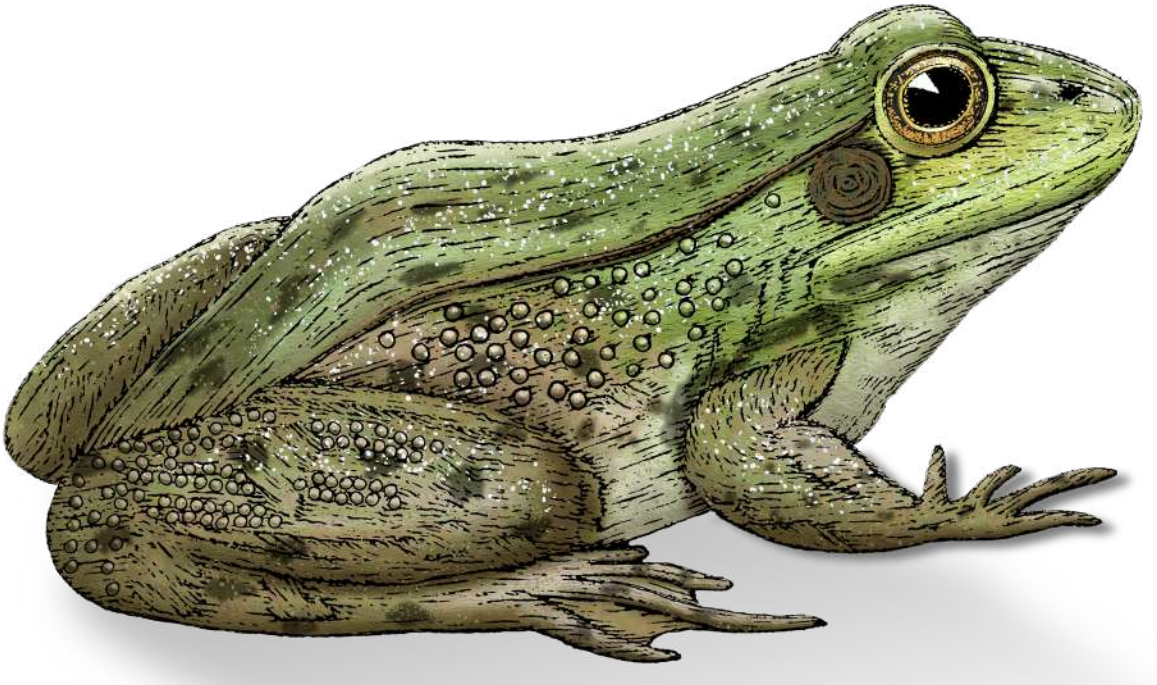
O'REILLY®

Low-Code AI

A Practical Project-Driven Introduction to Machine Learning



Gwendolyn Stripling, PhD
& Michael Abel, PhD

O'REILLY®

Low-Code AI

Take a data-first and use-case–driven approach with *Low-Code AI* to understand machine learning and deep learning concepts. This hands-on guide presents three problem-focused ways to learn no-code ML using AutoML, low-code using BigQuery ML, and custom code using scikit-learn and Keras. In each case, you'll learn key ML concepts by using real-world datasets with realistic problems.

Business and data analysts get a project-based introduction to ML/AI using a detailed, data-driven approach: loading and analyzing data; feeding data into an ML model; building, training, and testing; and deploying the model into production. Authors Michael Abel and Gwendolyn Stripling show you how to build machine learning models for retail, healthcare, financial services, energy, and telecommunications.

You'll learn how to:

- **Distinguish between structured and unstructured data and the challenges they present**
- **Visualize and analyze data**
- **Preprocess data for input into a machine learning model**
- **Differentiate between the regression and classification supervised learning models**
- **Compare different ML model types and architectures, from no code to low code to custom training**
- **Design, implement, and tune ML models**
- **Export data to a GitHub repository for data management and governance**

"This excellent and detailed guide unlocks the potential of ML, illustrated through real-world use cases and hands-on problems."
—**Michael Munn**
Research software engineer, Google

"A very special book that strikes the right balance between practical low-code solutions to get started with ML and in-depth explanations."
—**Benoit Dherin**
ML engineer, Google Cloud

Gwendolyn Stripling, PhD, is the lead AI/ML content developer at Google Cloud Learning Services. She developed two of the most popular generative AI courses on YouTube as well as the Introduction to Neural Networks series on LinkedIn Learning.

Michael Abel, PhD, is technical lead for the Specialized Training program at Google Cloud. Formerly he was a data and ML technical trainer at Google Cloud, and was a visiting assistant professor of mathematics at Duke University.

DATA

US $79.99 CAN $99.99
ISBN: 978-1-098-14682-5



Twitter: @oreillymedia
linkedin.com/company/oreilly-media
youtube.com/oreillymedia

Praise for *Low-Code AI*

Everyone from those in tech-adjacent roles to aspiring data scientists and ML engineers can benefit from the project-based approach and no-code and low-code solutions presented in this wonderfully written book.

—Eric Pilotte,
Global head of Technical and Business Training Delivery, Google Cloud

Low-Code AI is what I have been looking for to help jumpstart learning AI. This book provides an easy-to-follow guide that helps those of us who want to harness the power of AI for data-driven decision making, but that do not yet have years of ML coding experience. I am grateful for this highly accessible book and feel seen!

—Shana Rigelhaupt,
Product manager, The Carey Group, and aspiring citizen data scientist

Low-Code AI is a very special book that manages to strike the right balance between practical low-code recipes to get started with ML and in-depth explanations that are accessible to beginners. A great read to start a journey in AI from scratch and build quality intuition in this always-changing field.

—Benoit Dherin,
ML engineer, Google Cloud

Whether you are familiar with coding or are a beginner, this excellent and detailed guide unlocks the potential of ML, illustrated through real-world use cases and hands-on problems.

—Michael Munn,
Research software engineer, Google Cloud

Low-Code AI

A Practical Project-Driven Introduction to Machine Learning

Gwendolyn Stripling, PhD
& Michael Abel, PhD

Beijing • Boston • Farnham • Sebastopol • Tokyo O'REILLY®

Low-Code AI
by Gwendolyn Stripling and Michael Abel

Copyright © 2023 Gwendolyn Stripling and Michael Abel. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisition Editor: Nicole Butterfield
Development Editor: Corbin Collins
Production Editor: Clare Laylock
Copyeditor: nSight Inc.
Proofreader: Piper Editorial Consulting, LLC
Indexer: BIM Creatives, LLC
Interior Designer: David Futato
Cover Designer: Karen Montgomery
Illustrator: Kate Dullea

September 2023: First Edition

Revision History for the First Edition
2023-09-13: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098146825* for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Low-Code AI*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-14682-5

[LSI]

Table of Contents

Preface ix

1. How Data Drives Decision Making in Machine Learning 1
What Is the Goal or Use Case? 1
An Enterprise ML Workflow 4
Defining the Business Objective or Problem Statement 5
Data Collection 6
Data Preprocessing 7
Data Analysis 9
Data Transformation and Feature Selection 10
Researching the Model Selection or Using AutoML (a No-Code Solution) 11
Model Training, Evaluation, and Tuning 12
Model Testing 13
Model Deployment (Serving) 14
Maintaining Models 14
Summary 15

2. Data Is the First Step 17
Overview of Use Cases and Datasets Used in the Book 17
1. Retail: Product Pricing 18
2. Healthcare: Heart Disease Campaign 18
3. Energy: Utility Campaign 19
4. Insurance: Advertising Media Channel Sales Prediction 19
5. Financial: Fraud Detection 20
6. Energy: Power Production Prediction 21
7. Telecommunications: Customer Churn Prediction 21
8. Automotive: Improve Custom Model Performance 22
Data and File Types 23

Quantitative and Qualitative Data 23
Structured, Unstructured, and Semistructured Data 24
Data File Types 25
How Data Is Processed 26
An Overview of GitHub and Google's Colab 27
Use GitHub to Create a Data Repository for Your Projects 27
Using Google's Colaboratory for Low-Code AI Projects 30
Summary 48

3. Machine Learning Libraries and Frameworks. 49
No-Code AutoML 49
How AutoML Works 58
Machine Learning as a Service 62
Low-Code ML Frameworks 66
SQL ML Frameworks 67
Open Source ML Libraries 68
Summary 70

4. Use AutoML to Predict Advertising Media Channel Sales. 71
The Business Use Case: Media Channel Sales Prediction 71
Project Workflow 72
Project Dataset 73
Exploring the Dataset Using Pandas, Matplotlib, and Seaborn 73
Load Data into a Pandas DataFrame in a Google Colab Notebook 74
Explore the Advertising Dataset 75
Use AutoML to Train a Linear Regression Model 85
No-Code Using Vertex AI 86
Create a Managed Dataset in Vertex AI 86
Select the Model Objective 87
Build the Training Model 93
Evaluate Model Performance 98
Model Feature Importance (Attribution) 101
Get Predictions from Your Model 103
Summary 111

5. Using AutoML to Detect Fraudulent Transactions. 113
The Business Use Case: Fraud Detection for Financial Transactions 113
Project Workflow 114
Project Dataset 115
Exploring the Dataset Using Pandas, Matplotlib, and Seaborn 116
Loading Data into a Pandas DataFrame in a Google Colab Notebook 116
Exploring the Dataset 118

Exporting the Dataset 131
Classification Models and Metrics 132
Using AutoML to Train a Classification Model 134
Creating a Managed Dataset and Selecting the Model Objective 135
Exploring Dataset Statistics 136
Training the Model 137
Evaluating Model Performance 140
Model Feature Importances 141
Getting Predictions from Your Model 142
Summary 145

6. Using BigQuery ML to Train a Linear Regression Model. 147
The Business Use Case: Power Plant Production 147
Cleaning the Dataset Using SQL in BigQuery 148
Loading a Dataset into BigQuery 149
Exploring Data in BigQuery Using SQL 154
Linear Regression Models 160
Feature Selection and Correlation 163
Google Colaboratory 166
Plotting Feature Relationships to the Label 170
The CREATE MODEL Statement in BigQuery ML 175
Introducing Explainable AI 178
Explainable AI in BigQuery ML 179
Exercises 182
Neural Networks in BigQuery ML 183
Brief Overview of Neural Networks 183
Activation Functions and Nonlinearity 185
Training a Deep Neural Network in BigQuery ML 187
Exercises 190
Deep Dive: Using Cloud Shell to View Your Cloud Storage File 190
Summary 192

7. Training Custom ML Models in Python. 193
The Business Use Case: Customer Churn Prediction 193
Choosing Among No-Code, Low-Code, or Custom Code ML Solutions 194
Exploring the Dataset Using Pandas, Matplotlib, and Seaborn 196
Loading Data into a Pandas DataFrame in a Google Colab Notebook 196
Understanding and Cleaning the Customer Churn Dataset 199
Transforming Features Using Pandas and Scikit-Learn 213
Building a Logistic Regression Model Using Scikit-Learn 218
Logistic Regression 219
Training and Evaluating a Model in Scikit-Learn 221

Classification Evaluation Metrics 223
Serving Predictions with a Trained Model in Scikit-Learn 225
Pipelines in Scikit-Learn: An Introduction 228
Building a Neural Network Using Keras 231
Introduction to Keras 231
Training a Neural Network Classifier Using Keras 232
Building Custom ML Models on Vertex AI 237
Summary 248

8. Improving Custom Model Performance. 249
The Business Use Case: Used Car Auction Prices 249
Model Improvement in Scikit-Learn 250
Loading the Notebook with the Preexisting Model 251
Loading the Datasets and the Training-Validation-Test Data Split 251
Exploring the Scikit-Learn Linear Regression Model 253
Feature Engineering and Improving the Preprocessing Pipeline 256
Hyperparameter Tuning 261
Model Improvement in Keras 266
Introduction to Preprocessing Layers in Keras 266
Creating the Dataset and Preprocessing Layers for Your Model 267
Building a Neural Network Model 270
Hyperparameter Tuning in Keras 272
Hyperparameter Tuning in BigQuery ML 276
Loading and Transforming Car Auction Data 276
Training a Linear Regression Model and Using the TRANSFORM Clause 279
Configure a Hyperparameter Tuning Job in BigQuery ML 281
Options for Hyperparameter Tuning Large Models 285
Vertex AI Training and Tuning 286
Automatic Model Tuning with Amazon SageMaker 286
Azure Machine Learning 286
Summary 286

9. Next Steps in Your AI Journey. 289
Going Deeper into Data Science 289
Working with Unstructured Data 290
Generative AI 294
Explainable AI 295
ML Operations 298
Continuous Training and Evaluation 299
Summary 300

Index. 301

Preface

Artificial intelligence (AI) can be defined as the broad field of study where computers show intelligence. The phrase "show intelligence" is vague; it could be interpreted as a computer making a decision that we would expect from a living being. The concept of AI has existed since ancient times, at least in mythology. A famous example of this is the Greek myth featuring Talos, a bronze automaton made to protect Europa from invaders who wished to kidnap her. As the centuries passed, basic forms of AI passed from the realm of myth to real life.

In modern times, AI has found its home in augmenting human abilities and automating decision making and other processes that are time-consuming for people. Expert systems, first developed in the 1970s, are one such example of modern AI. An expert system leverages a knowledge base, a collection of facts and rules, and an inference system to synthesize new knowledge. The main disadvantage of an expert system is that it needs domain expert time and effort to create the facts and rules for the knowledge base.

In recent decades, another form of AI has become more ubiquitous. Machine learning (ML) is the discipline of having computers learn algorithms from the data provided rather than the programmer having to provide the algorithms. Another way to phrase this in contrast to expert systems is that ML is about using data to discover the rules versus having experts write the rules for you.

ML touches almost every industry nowadays. In retail, ML is used for demand forecasting, predicting the expected sales of products or services months in advance. The travel industry uses ML to recommend points of interest and destinations to customers based on their past travel history and other information. In healthcare, ML can be used to not only determine if an X-ray image contains a healthy or diseased lung, but it can also pinpoint the region of the X-ray image that led to the determination for medical experts to explore in more detail. The list of active uses of ML could fill up an entire book on its own. In this book we focus on a few specific

ML use cases to get you started, including advertising media channel sales, energy production, and customer churn just to name a few.

With so many applications of ML in industry, it is exciting to explore the different possibilities. One assumption that many make is that ML is solely a field of study for the experts. That is, unless you have a large amount of background knowledge across many different fields (computer science, mathematics, statistics, and so on), you have no hope of using ML in practice. That is simply not the case.

In recent years, the concept of a *citizen data scientist* has become much more common. A citizen data scientist is someone who does not necessarily have a formal education and/or role in data science or related fields but can perform some data science work alongside other domain-specific expertise they bring to the table. Many easy-to-use tools have been developed for ML that are available to this group of people, and the goal of this book is to enable and encourage more people to become citizen data scientists.

Who Should Read This Book?

The goal of this book is to teach readers how to frame ML problems for structured (tabular) data, prepare their data for ML workflows, and build and use ML models using different no-code, low-code, and some basic custom code solutions. You will go through step-by-step processes to understand these objectives within the framing of a specific business problem. The primary audience for this book are business analysts, data analysts, students, and aspiring citizen data scientists who seek to learn how to apply ML to their work very quickly using automated machine learning (AutoML), BigQuery ML (using SQL), and custom training in Python. Some basic familiarity with data analysis is assumed, but you don't need to be an expert to benefit from going through this book.

Anyone considering a career move into data science and/or ML engineering may also find this book to be a great first step toward their goal. ML practitioners will likely find this book to be too basic for their needs, but they may find discussions on some of the tools being used to be helpful if they are unfamiliar.

No prior knowledge of ML or a specific programming language is required, but readers will find the book a little easier to read with some basic knowledge of programming concepts, Python, and SQL. We include references to additional foundational material within context throughout the book. In addition to ML concepts and use case–based examples, you will also explore different tools such as Jupyter Notebooks and basic use of the Linux terminal.

What Is and Isn't in This Book

This book was created as a first step for those who wish to become ML practitioners, not as a book to turn you into an ML expert. We do not cover the theory of ML in detail, nor do we cover all of the topics from statistics and mathematics needed to be a successful data scientist. We cover the theory that is needed for the projects discussed in this book as a way to ease you into working on ML projects, but going farther than that would be beyond the scope here. We do, however, give many references to resources where you can dive deeper if you are interested in doing so.

Chapters 2 and 3 discuss many different types of data that can be used in ML problems and different tools that can be used in practice. However, no single book can cover every single circumstance with every available tool. We focus on use cases with structured data and only pursue a light discussion around ML for unstructured data in Chapters 2 and 9. Some of the most exciting applications (AI-powered chatbots and image generation, for example) use unstructured data, but in practice most applications of ML in business and industry focus on problems involving structured data.

In terms of tools, we focus on a narrow range of tools so that you can focus on the business use cases. Packages in Python such as NumPy, Seaborn, Pandas, scikit-learn, and TensorFlow are popular across all industries, and we cover those alongside many of the use cases in this book. Jupyter Notebooks are also an industry standard used to interactively run Python code in a notebook environment.

We use Google Colab, a free Jupyter Notebook service, for running our notebooks. Additionally, we will use other Google Cloud tools, such as Vertex AI AutoML for no-code ML model training and BigQuery for SQL data analysis and training ML models using SQL. Other major cloud providers, such as Microsoft Azure and Amazon Web Services (AWS), offer similar services for running Jupyter Notebooks, AutoML, analyzing data with SQL, and training ML models using SQL. We highly encourage and recommend that you explore the other tools that we mention but do not use here. Links for more information and documentation are included throughout the entire book.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`
: Shows commands or other text that should be typed literally by the user.

`Constant width italic`
: Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This icon indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *https://oreil.ly/supp-lcai*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Low-Code AI* by Gwendolyn Stripling and Michael Abel (O'Reilly). Copyright 2023 Gwendolyn Stripling and Michael Abel, 978-1-098-14682-5."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-889-8969 (in the United States or Canada)
707-829-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
https://www.oreilly.com/about/contact.html

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/LowCodeAI*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly-media*.

Follow us on Twitter: *https://twitter.com/oreillymedia*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

Acknowledgments

This book exists only because of the generous support and time of many people. We would like to thank our fellow Googlers—Enrica Filippi, Mona Mona, Benoit Dherin, and Michael Munn—for their detailed review of this book. The quality of exposition and technical details greatly benefited from their expertise. We also want to thank our coworkers—Rich Rose, Patrick Bentley, Yoanna Long, Esra Duygun, and Concepcion Diaz—for their support and feedback. Last but not least, we want to thank our managers—Kelly Thompson, Eric Pilotte, and Sree Upadhyayula—for their unwavering support for this endeavor.

A big thank you to the O'Reilly team for all their collaboration in bringing this dream to life and their patience with two writers excitedly navigating this process for the first time. We wish to thank Nicole Butterfield for believing in our project and vision from the beginning and Clare Laylock for her work in getting this book ready for print. We would like to thank Corbin Collins for his hard work and editorial expertise throughout the entire process.

Michael would like to thank his wife Jackie for her incredible support and patience throughout this entire process of writing this book, his daughter Mia for her knack of always finding the right time to interrupt work to play with the keyboard and bring a smile to his face, and his parents, Ben and Rita, for fostering his love of mathematics and computers for as long as he can remember.

Gwendolyn would like to thank her life partner, Nada Velimirović, for her unwavering support during this amazing journey, and her daughter, Saskia, and pets, Ginger and Wonton, for their incredible ability to know when to provide a hug, ask for a lap, and whine for a short walk. She would also like to thank her family: her father and mother (thank you for the DNA); her sister Belinda (thank you for seeing me); her sister, Gloria Taylor Williams, and her husband, Charles Williams; her brothers, Maurice Cobb and Carlos S. Armstrong; and her uncle, Tony Hall. She so appreciates her niece, Katrina Marie Burage, and nephew, Carlos S. Armstrong Jr., for all of their amazing support during this journey! She'd also like to thank her dear friend Christina Ramirez, who has always believed and supported her out-of-the-box thinking!

CHAPTER 1

How Data Drives Decision Making in Machine Learning

This chapter explores the role of data in the enterprise and its influence on business decision making. You also learn the components of a machine learning (ML) workflow. You may have seen many books, articles, videos, and blogs begin any discussion of the ML workflow with the gathering of data. However, before data is gathered, you need to understand what kind of data to gather. This *data understanding* can only be achieved by knowing what kind of problem you need to solve or decision you need to make.

Business case/problem definition and data understanding can then be used to formulate a no-code or low-code ML strategy. A no-code or low-code strategic approach to ML projects has several advantages/benefits. As mentioned in the introduction, a no-code AutoML approach enables anyone with domain knowledge in their area of expertise and no coding experience to develop ML models quickly, without needing to write a single line of code. This is a fast and efficient way to develop ML applications. A low-code approach enables those with some coding or deep coding experience, to develop ML applications quickly because basic code is autogenerated—and any additional custom code can be added. But, again, any ML project must begin with defining a goal, use case, or problem.

What Is the Goal or Use Case?

Businesses, educational institutions, government agencies, and practitioners face many decisions that reflect real-world examples of ML. For example:

- How can we increase patient engagement with our diabetes web app?
- How can we increase our student feedback numbers on course surveys?

- How can we increase our speed in detecting cyberattacks against our company networks?
- Can we decrease the number of spam emails entering our email servers?
- How do we decrease downtime on our manufacturing production line?
- How can we increase our customer retention rate?
- How do we reduce our customer churn (customer attrition) rate?

In each of those examples, numerous data sources must be examined to determine what ML solution is most appropriate to solve the problem or aid in decision making. Let's take the use case of reducing customer churn or loss rate—using a very simplistic example. *Churn prediction* is identifying customers that are most likely to leave your service or product. This problem falls into a supervised learning bucket as a classification problem with two classes: the "Churn-Yes" class and the "Churn-No" class.

From a data source perspective, you may need to examine customer profile information (name, address, age, job title, employment statement), purchase information (purchases and billing history), interaction information (customer experiences interacting with your products [both digitally and physically]), your customer service teams, or your digital support services. Popular data sources of customer information are customer relationship management systems, system ecommerce analytics services, and customer feedback. In essence, everything the customer "touches" as a data point should be tracked and captured as a data source.

The nature of the decision you must make is tied directly to the data you will need to gather to make that decision—which needs to be formulated into a problem statement. Let's say you are in charge of marketing for a company that makes umbrellas, and the *business goal* is to increase sales. If you reduce the selling price of your existing umbrellas, can you predict how many umbrellas you will sell? Figure 1-1 shows the data elements to consider for this option.

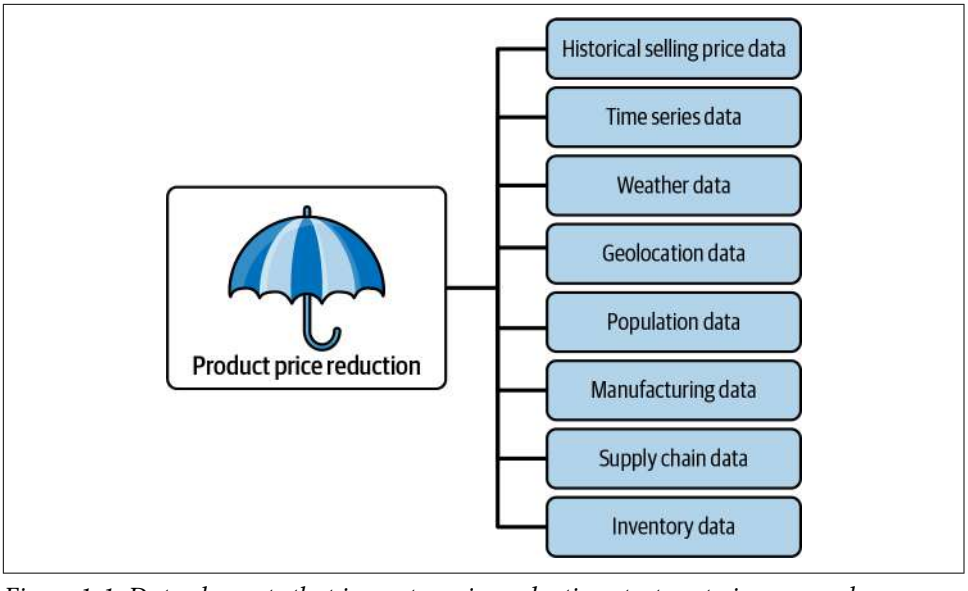


Figure 1-1. Data elements that impact a price reduction strategy to increase sales.

As you can see in this data-driven business illustration, your business goal (to increase sales) takes on a new dimension. You realize now that to understand a product price reduction, you need to include additional data dimensions aside from the selling price. You will need to know the rainy seasons in specific regions, population density, and whether your inventory is sufficient to meet the demand of a price reduction that will increase sales. You will also need to look at historical data versus data that can be captured in real time. Historical data is typically referred to as *batch*, whereas real-time data capture is typically called *streaming*. With these added dimensions, the business goal suddenly becomes a very complex problem as these additional columns may be required. For any organization, there could ostensibly exist dozens of discrete data sources—with each source requiring certain skills to understand the relationships between them. Figure 1-2 is an illustration of this challenge.

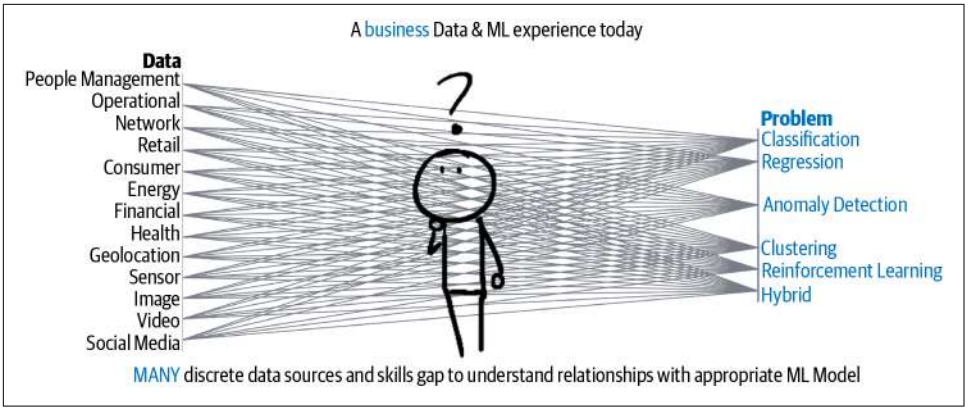


Figure 1-2. A typical business data and ML experience today.

So what is your use case here? It depends. You would need to undergo a business decision-making process, which is the process of making choices by asking questions, collecting data, and assessing alternative resolutions. Once you figure out the use case or business goal, you can use the same data to train machines to learn about your customer patterns, spot trends, and predict outcomes using AutoML or low-code AI. Figure 1-3 shows our umbrella example as a business use case that then leads to data source determination, ML framework, and then a prediction.

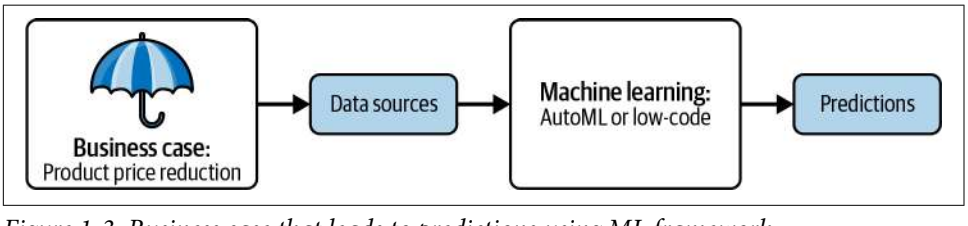


Figure 1-3. Business case that leads to predictions using ML framework.

An Enterprise ML Workflow

While decision-making processes help you identify your problem or use case, it is the ML workflow that helps you implement the solution to your problem. This section presents a typical ML workflow. In our ongoing umbrella example, you could use your data to train an ML model using an AutoML service that provides a no-code solution for running unsupervised ML clustering. From there, you could examine *clusters* of data points to see what patterns were derived. Or, you could decide to simply focus on historical data so that you could predict a specific target based on a certain number of data input features. What would your enterprise ML workflow look like? Not surprisingly, it is data-driven and requires decision making in the process.

The ML workflow can be shown as a series of steps, and the steps can be combined into phases. Figure 1-4 shows the 10 steps, and then we briefly discuss each. Later chapters provide more detailed examples of each step.

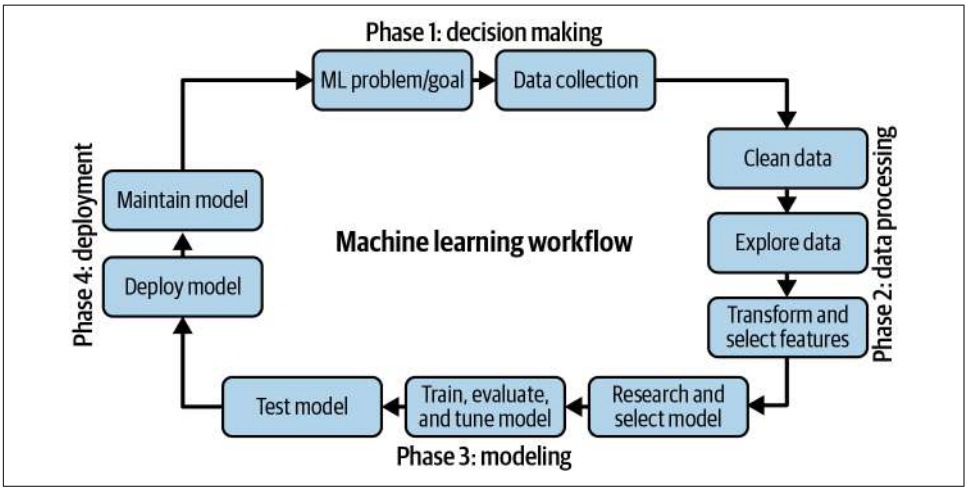


Figure 1-4. Ten-step ML workflow.

Defining the Business Objective or Problem Statement

The ML workflow starts with defining a specific question or problem with a defined boundary. In this phase you are attempting to define scope and feasibility. The right question will lead you to what data is required and potential ways data must be prepared. It is important to note that any question that may arise in analyzing data can be grouped in one of the five ML categories as shown in Table 1-1. Let's continue with our umbrella example.

Table 1-1. Categories of analyzing data

Algorithm/model	Problem or question
Regression problem	How many umbrellas do you expect to sell this month/season?
Classification problem	Did they buy straight umbrellas (A) or foldable umbrellas (B)?
Clustering problem	How many straight umbrellas were sold by month or by region?
Anomaly detection problem	Did the company sell more umbrellas in the Mojave Desert than in Portland, OR?
Reinforcement learning	Company policy is to only ship to customers with a balance owed of $500 or less. Can a manufacturing robot be trained to extract, package, load, and ship straight umbrellas to our customers based upon this policy?

Data Collection

In the early part of the 21st century, companies, universities, and researchers typically relied on local servers/hard drives or data centers to host their database applications and store their data. Relying on on-premises data centers or even renting server space in a data center was costly: server infrastructure needed to be maintained, software needed to be updated, security patches had to be installed, physical hardware was swapped out, and so on. In some cases, large amounts of data were stored across a cluster of machines.

Today, to save on costs, enterprises and educational institutions have moved to the cloud to host their database applications and store their data. Cloud storage, a service offered by cloud vendors to store files, allows you to upload different file formats or can be configured to automatically receive files from different data sources. Because most ML models are trained using data from files, storing your data in a cloud storage *bucket* makes for easy data collection. Cloud storage buckets can be used for storing both structured and unstructured data.

Another option to store data files for data collection is *GitHub*, a service designed for collaborating on coding projects. You can store data in the cloud for future use (for free), track changes, and make data publicly available for replication. This option has strict file size limits of 100 MB, but there is an option to use Git Large File Storage (LFS), an open source GitHub extension for versioning large files. Git LFS replaces large files such as datasets, audio samples, graphics, and videos with text pointers inside Git, while storing the file contents on a remote server like GitHub.com or GitHub Enterprise.

The challenge of data collection is compounded within large organizations, where many different types of operations management software such as enterprise resource planning, customer relationship management, and production systems exist and may run on different databases. Data may also need to be pulled from external sources in real time, such as Internet of Things (IoT) sensor devices from delivery trucks. Thus, organizations are challenged with collecting not only structured data, but also unstructured and semistructured data formats in batches or real time (streaming). Figure 1-5 shows various data elements that feed data collection for structured, unstructured, and semistructured data.

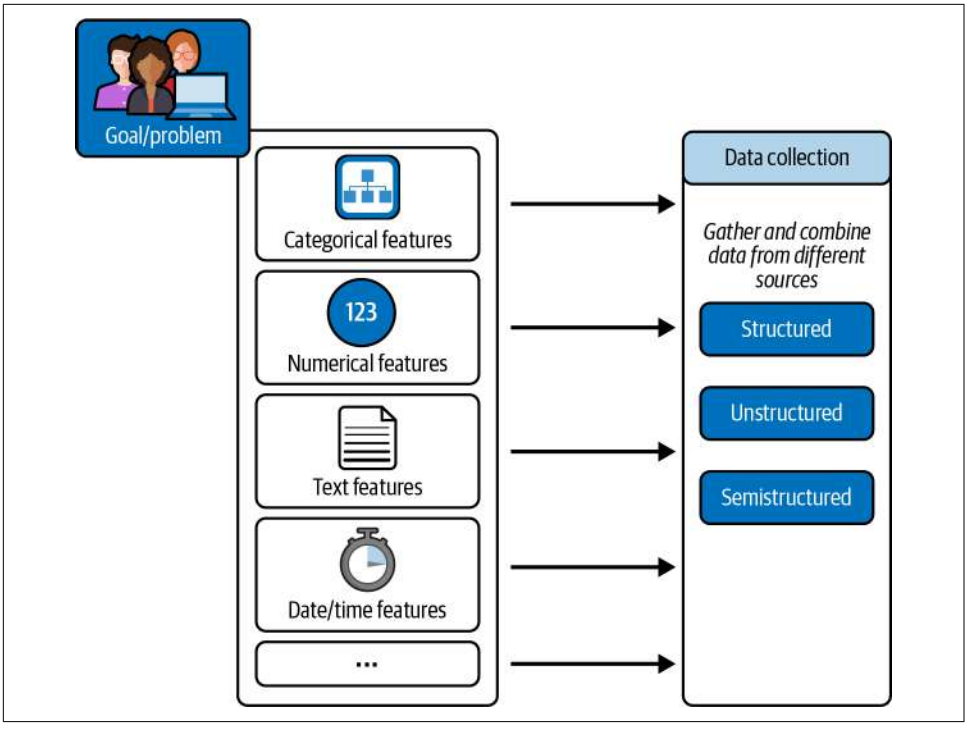


Figure 1-5. Goal/problem flow to data collection.



It is possible to have streaming structured data. Structured versus unstructured is a property of data format. Streaming versus batch is a property of latency. Chapter 2 presents more information on data format and properties.

Data Preprocessing

To perform data cleanup you'll need to deal with missing data values, duplicates, outlier data, formatting issues, or data that is inconsistent due to human error. This is because real-world data is raw and messy and filled with assumptions. One assumption could be that your data has a normal distribution, meaning that data is symmetrically distributed with no skew, and that most values cluster around a central region, with the frequency of the values decreasing further away from the center (mean or average).

Suppose your data showed, for the first time, an increase in the number of umbrellas sold in August in Palm Springs, the California desert town. Would your data be normally distributed, or would this be considered an outlier? Would it skew the results of predictions for monthly umbrella sales in August? When data does not have

a normal distribution, it needs to be *normalized*, made *normal* by grouping all the records in a range of [0,1] or [–1,1], for example. You normalize a dataset to make it easier and faster to train an ML model. Normalization is covered in Chapter 7.



This min-max normalization example can have detrimental effects if there are outliers. For example, when scaling to [0,1], it essentially maps the outlier to 1 and squashes all other values to 0. Addressing outliers and anomalies is beyond the scope of our book.

Thus, data preprocessing can mean normalizing the data (such that numeric columns in the dataset use a common scale) and *scaling* the data, which means transforming your data so that it fits within a specific range. Fortunately, normalization and standardization are easily performed in Python with just a few simple lines of code. Figure 1-6 shows actual data before and after normalization and standardization.

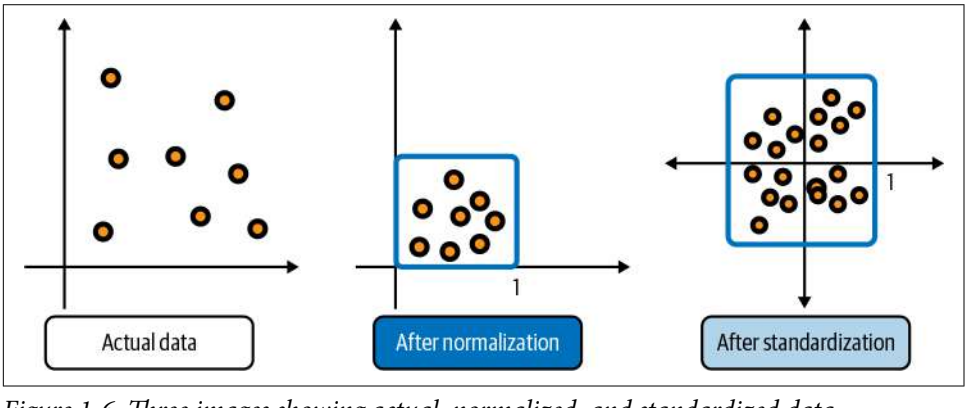


Figure 1-6. Three images showing actual, normalized, and standardized data.



Collecting data from a single source may be a relatively straightforward process. However, if you are aggregating several data sources into one file, make sure that data formats match and that any assumptions regarding time-series data (or timestamp and date ranges needed for your ML model) are validated. A common assumption is that the data is stationary—that the statistical properties (mean, variance, etc.) do not change over time.

Data Analysis

Exploratory data analysis (EDA) is a process used to explore and analyze the structure of data. In this step, you are looking to discover trends, patterns, feature relevance, and correlations, such as how one variable (feature) might correlate with another. You must select relevant feature data for your ML model based on the type of problem you are trying to solve. The outcome of this step is a feature list of input variables that can potentially be used for ML. Our hands-on exercise using EDA can be found in Chapter 6.

Figures 1-7 and 1-8 are a result of an EDA process plotted using Seaborn, a Python data visualization library (see Chapter 6 for more detail on the dataset). Figure 1-7 shows an *inverse* relationship between *x* and *y*. Figure 1-8 shows a *heat map* (or correlation matrix) and illustrates that more energy is produced when temperatures are lower.

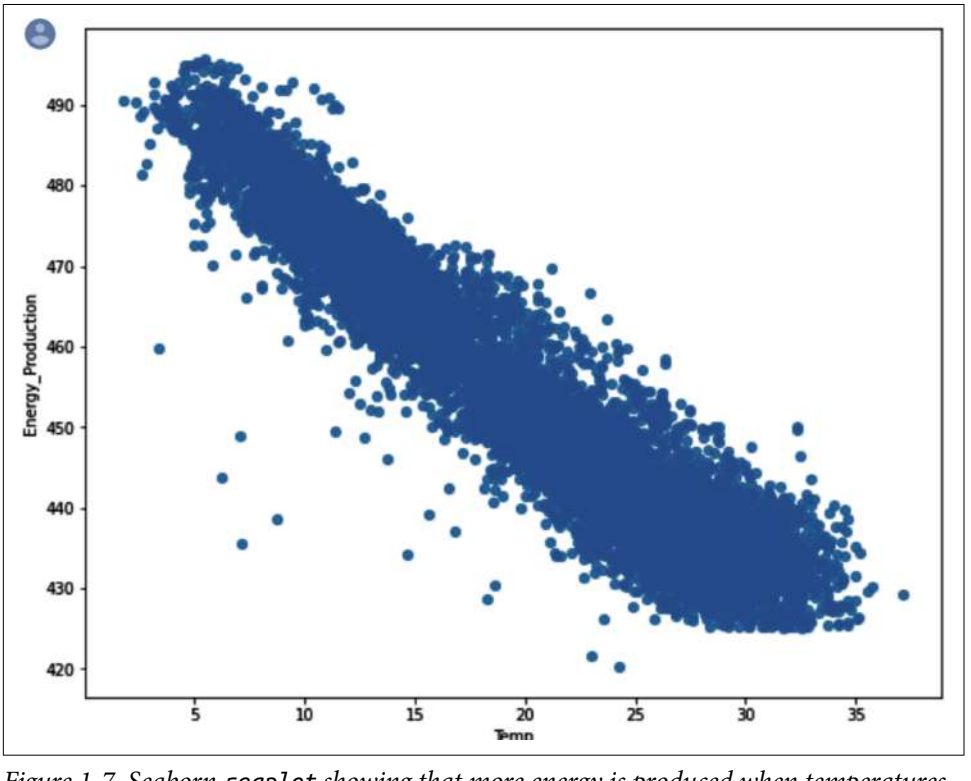


Figure 1-7. Seaborn `regplot` showing that more energy is produced when temperatures are lower.

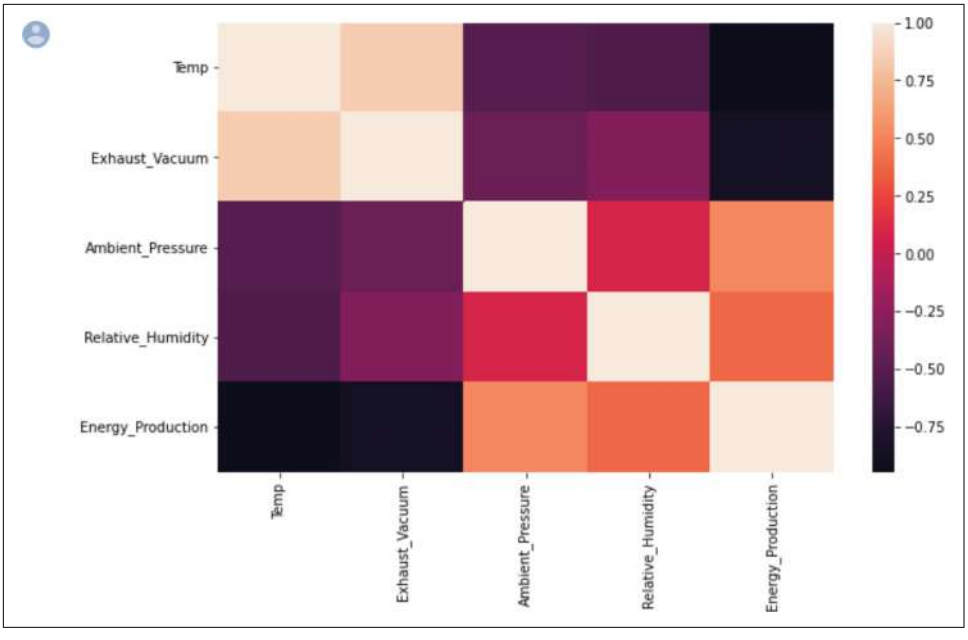


Figure 1-8. Seaborn correlation matrix (heat map) showing a strong inverse relationship between `Temp` and `Energy_Production`, -0.75.

Data Transformation and Feature Selection

After data has been cleaned and analyzed, you obtain a list of the features you think you need to help you solve your ML problem. But might other features be relevant? This is where *feature engineering* comes into play, where you *engineer* or *create new* features that were not in the original dataset. For example, if your dataset has separate fields/columns for month, day, and year, you can combine all three for a "month-day-year" time feature. Feature engineering is the final step before feature selection.

In reality, feature selection occurs at two stages: after EDA and after data transformation. For example, after EDA, you should have a potential list of features that may be candidates to create new features—for example, combining time and day of week to get an hour of day. After you perform feature engineering, you then have a final list of features from which to select. Figure 1-9 shows the position of data transformation and feature selection in the workflow.

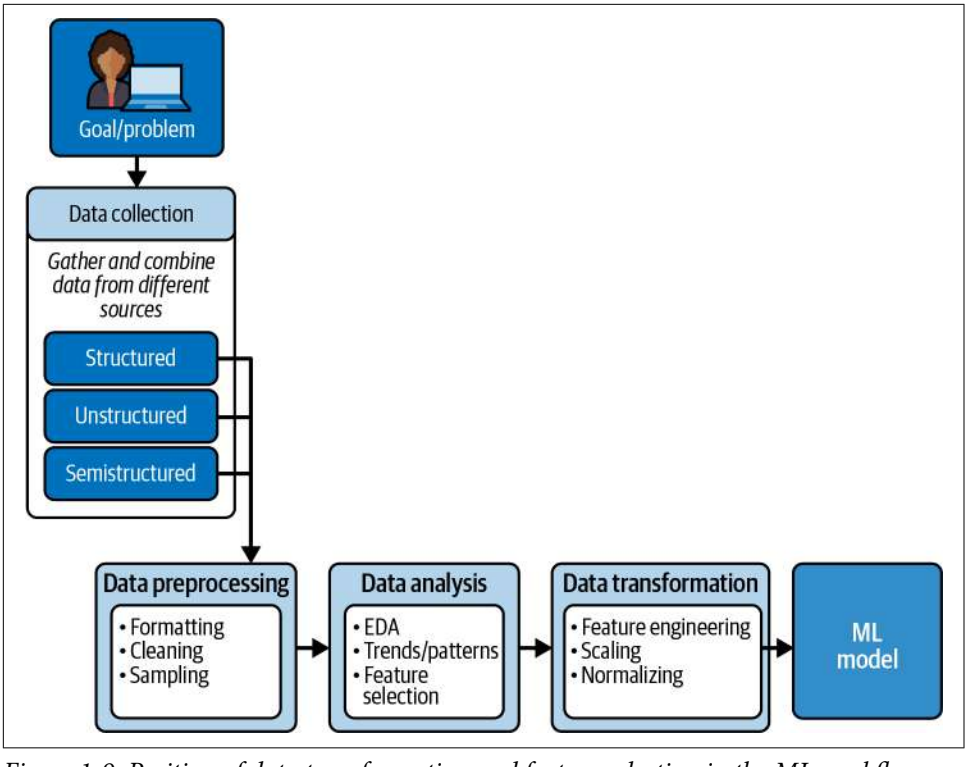


Figure 1-9. Position of data transformation and feature selection in the ML workflow.

Researching the Model Selection or Using AutoML (a No-Code Solution)

In this step, you either research the model that will be best for the type of data that fits your problem—or you could use AutoML, a no-code solution that, based on the dataset you uploaded, selects the appropriate model, trains, tests, and generates evaluation metrics. Essentially, if you use AutoML, the heavy lifting of model selection, model training, model tuning, and generating evaluation metrics is done for you. Chapter 3 introduces AutoML, and Chapter 4 starts getting hands-on with AutoML. Note that with a low-code solution, you would need to know what model to select.

Although AutoML might cover about 80% of your ML problems, you may want to build a more customized solution. In that case, having a general understanding of the types of problems ML algorithms can solve is helpful. Choosing the algorithm is solely dependent upon the problem (as discussed earlier). In Table 1-2, a "Description" column is added to further describe the ML model problem type.

Table 1-2. Describing the model type

Problem or question	Problem	Description
How much or how many umbrellas?	Regression problem	Regression algorithms are used to deal with problems with continuous and numeric output. These are usually used for problems that deal with questions like *how much* or *how many*.
Did they buy straight umbrellas (A) or foldable umbrellas (B)?	Classification problem	A problem in which the output can be only one of a fixed number of output classes, like Yes/No or True/False, is called a classification problem. Depending on the number of output classes, the problem can be a binary or multiclass classification problem.
Company policy is to only ship to customers with a balance owed of $500 or less. Can our manufacturing robot be trained to extract, package, load, and ship straight umbrellas to our customers based upon this policy?	Reinforcement learning	Reinforcement algorithms are used when a decision is to be made based on experiences of learning. The machine agent learns the behavior using trial and error in interaction with the continuously changing environment. This provides a way to program agents using the concept of rewards and penalties without specifying how the task is to be accomplished. Game-playing programs and programs for temperature control are some popular examples using reinforcement learning.

Model Training, Evaluation, and Tuning

Before an ML model can be deployed to a production environment, it has to be trained, evaluated, and tested. Training an ML model is a process in which stored data instances are fed (input) into an ML model (algorithm). Since every stored data instance has a specific characteristic (recall our umbrella examples of the different types, prices, regions sold, and so forth), patterns of these data instances can be detected using hundreds of variables, and the algorithm is thus able to learn from the training data how to make a generalized prediction based on the data.

Every ML model needs to not only be trained but also evaluated. Thus, you hold out a sample of data, called a *validation dataset*. The validation set measures how well the model generalizes to unseen or new data. The training error is used to determine how well the model fits the data because that is what the model is trained on.

Model evaluation metrics should be chosen or defined so that they align with the problem or business goals. Model tuning should improve the model performance as measured by the evaluation metrics. For example, how accurate were the sales of umbrella predictions during the month of December? Can these predictions be generalized for future forecasting efforts? Note that satisfactory performance is something that should be dictated by the business needs and should be agreed upon before starting any ML engagement.



The validation set is also used to determine if the model is overfitting. Chapter 8 discusses overfitting.

Model Testing

There is no way to know if your umbrella prediction app can be generalized for future forecasting efforts without testing the model. Once the training dataset is used to fit the model to the data, and the validation dataset is used to improve model accuracy, you test the model on data it has never seen before. Testing data is used to assess model performance.

For example, let's say you want to build an application that can recognize an umbrella's color or pattern based on images of the umbrellas. You train a model by providing it with images of all umbrellas that are each tagged with a certain color or pattern. You use that model in a mobile application to recognize any umbrella's color or pattern. The test would be how well the model performs in differentiating between umbrella colors and patterns.

Figure 1-10 shows the relationship between the training, validation, and testing datasets.

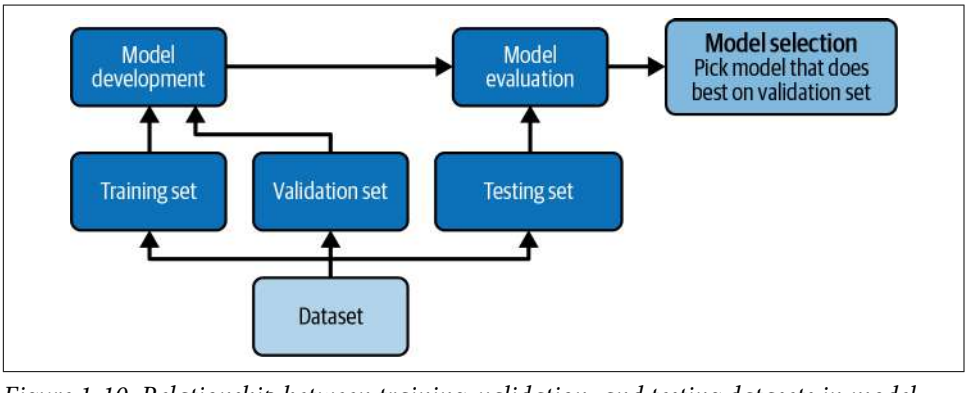


Figure 1-10. Relationship between training, validation, and testing datasets in model deployment and model evaluation.

Figure 1-11 illustrates this relationship among the training, validation, and test datasets in five process steps. For simplicity, the arrow going back to the dataset in Step 5 is not shown, since once a model is deployed as an application and it begins collecting data, new data enters the *pipeline* that may *skew* the original model's results. (At this point you enter the fascinating realm of machine learning operations, or MLOps, which is beyond the scope of the book.)

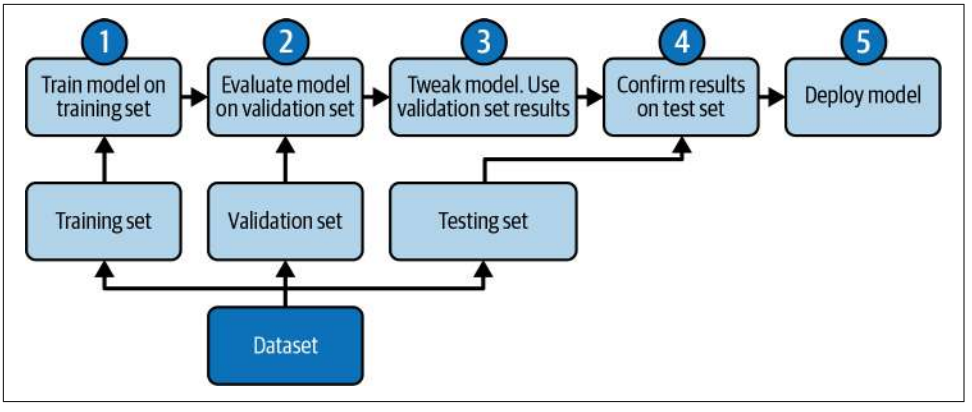


Figure 1-11. Five process steps of the ML workflow.

Model Deployment (Serving)

Once the ML model is trained, evaluated, and tested, it is deployed into a live production environment where it can be used. Note that by the time the model reaches production, it more than likely has a web app frontend (using a browser) that communicates with the production system through an *application programming interface* (API). Data can be captured in real time and streamed (ingested) into an MLOps pipeline. Or data can be captured in batch and stored for ingestion into the pipeline. Or both.

Maintaining Models

Models can become *stale* when predictions do not align with the original business goal or use case metrics. Staleness might occur when the world changes or business requirements change. These changes then impact the model. Post-deployment, you need to monitor your model to ensure it continues to perform as expected. Model and data drift is a phenomenon you should both expect and be prepared to mitigate through regular retraining using MLOps. Let's look at an example of data *drift*, which means changes in the data that you trained with and the data that is now being received from the web app.

In our umbrella example, a region that once experienced heavy rainfall is now experiencing drought conditions. Similarly, a region that once experienced drought conditions is now experiencing heavy rainfall. Any prediction tied to weather and climate and the need for umbrellas and umbrella type will be impacted. In this scenario, you would need to retrain and test a new model with new data.

Summary

Businesses, educational institutions, government agencies, and practitioners face many decisions that reflect real-world examples of ML, from increasing customer engagement to reducing customer churn. Data—its collection, analysis, and use—drives the decision making used in ML to determine the best ML strategic approach that provides real-world solutions to real-world problems.

While decision-making processes help you identify your problem or use case, it is the ML workflow that helps you implement the solution to your problem. An enterprise ML workflow is data-driven and requires decision making in the process. The ML workflow can be shown as a series of 10 steps, and the steps can be combined into four phases:

1. Decision making
2. Data processing
3. Modeling
4. Deployment

Each phase of the ML workflow can be implemented using AutoML or low-code AI. AutoML does all of the heavy lifting for you. AutoML will train the model, tune it, test it, and present you with evaluation metrics. Your role is simply to evaluate the metrics and determine if they meet your business objective or solve your problem. AutoML is recommended for quick experiments and prototypes. It is also used in production environments. A low-code approach enables those with some coding or deep coding experience to use autogenerated code that can be further customized during any phase of the ML workflow.

In this chapter, you learned about data collection and analysis as part of the ML workflow. Chapter 2 provides an overview of the datasets used in the book, where to find data sources, data file types, and the difference between batch, streaming, structured, semistructured, and unstructured data. You also get hands-on experience using basic Python code to help you perform EDA and solve dirty data problems.

CHAPTER 2

Data Is the First Step

This chapter provides an overview of the use cases and datasets used in the book while also providing information on where to find data sources for further study and practice. You'll also learn about data types, and the difference between batch and streaming data. You'll get hands-on practice with data preprocessing using Google's free browser-based open source Jupyter Notebook. The chapter concludes with a section on using GitHub to create a data repository for the selected projects used in the book.

Overview of Use Cases and Datasets Used in the Book

Hopefully, you picked up our book to learn ML not from a math-first or algorithm-first approach but from a project-based approach. The use cases we've chosen are designed to teach you ML using actual, real-world data across different sectors. There are use cases for healthcare, retail, energy, telecommunications, and finance. The use case on customer churn can be applied to any sector. Each of the use case projects can stand on its own if you have some data preprocessing experience, so feel free to skip ahead to what you need to learn to upskill yourself. Table 2-1 shows each section, its use case, sector, and whether it is no-code or low-code.

Table 2-1. List of use cases by industry sector and coding type

Section	Use case	Sector	Type
1	Product pricing	Retail	N/A
2	Heart disease	Healthcare	Low-code data preprocessing
3	Marketing campaign	Energy	No-code (AutoML)
4	Advertising media channel sales	Insurance	No-code (AutoML)
5	Fraud detection	Financial	No-code (AutoML)
6	Power plant production prediction	Energy	Low-code (BigQuery ML)

Section	Use case	Sector	Type
7	Customer churn prediction	Telecommunications	Low-code (scikit-learn and Keras)
8	Improve custom model performance	Automotive	Custom-code (scikit-learn, Keras, BigQuery ML)

1. Retail: Product Pricing

This section begins with a use case designed to illustrate the role of data in decision making. In this use case, you are in charge of marketing for a company that makes umbrellas, and the *business goal* is to increase sales. If you reduce the selling price of your existing umbrellas, can you predict how many umbrellas you will sell? Figure 2-1 shows the data elements that may impact a price reduction strategy to increase sales.

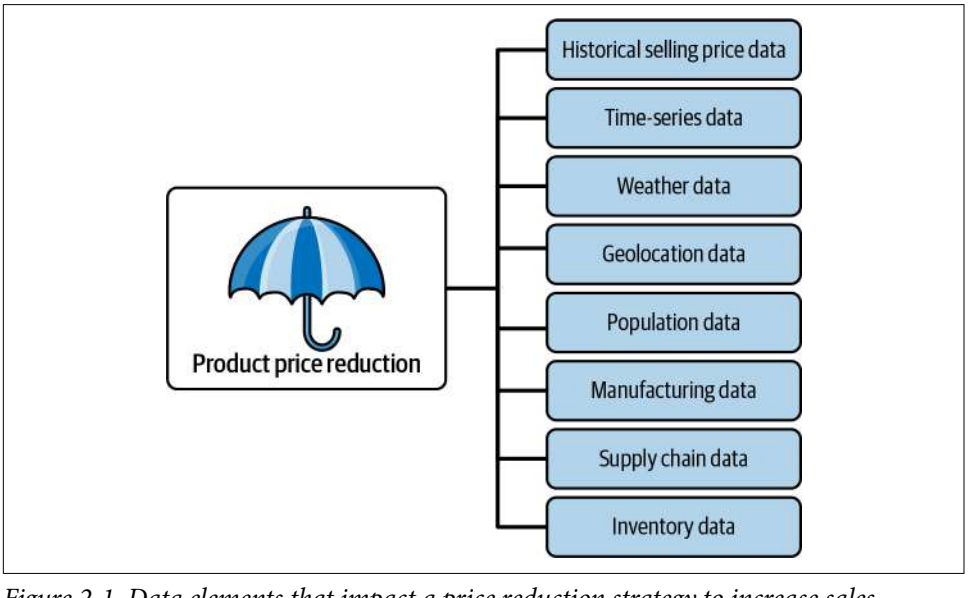


Figure 2-1. Data elements that impact a price reduction strategy to increase sales.

2. Healthcare: Heart Disease Campaign

In this one, you are a healthcare consultant and are given data on heart disease mortality for populations over the age of 35 in the United States. The goal is to analyze the heart disease mortality data and suggest a possible use case in a heart disease prevention campaign. For example, one possible use case would be to track trends in heart disease mortality over time or to develop and validate models for predicting heart disease mortality. This dataset is dirty. Some fields have missing values. One field is missing. In working through these issues, you learn to import data into a Python Jupyter Notebook, analyze it, and fix dirty elements. Figure 2-2 shows the data elements that contribute to your analysis.

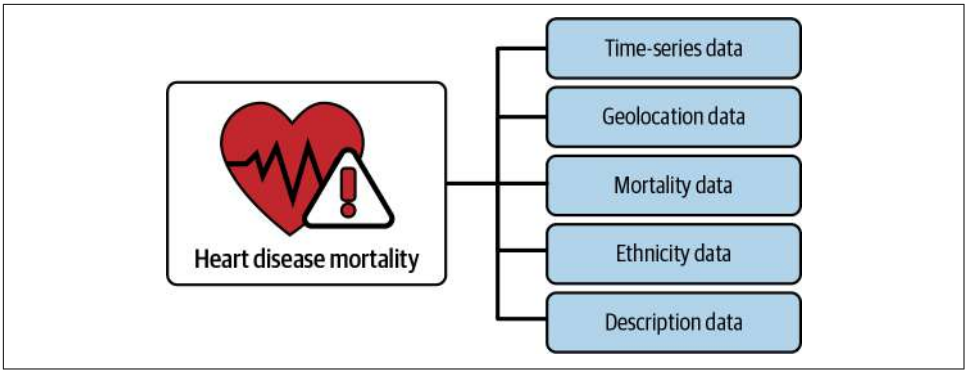


Figure 2-2. Data elements for a heart disease mortality use case.

3. Energy: Utility Campaign

Here, you are a business analyst working for a utility company. You are tasked with developing a marketing and outreach program that targets communities with high electrical energy consumption. The data has already been preprocessed. You do not have an ML background or any programming knowledge. You elect to use AutoML as your ML framework. Figure 2-3 shows the data elements that contribute to your model.

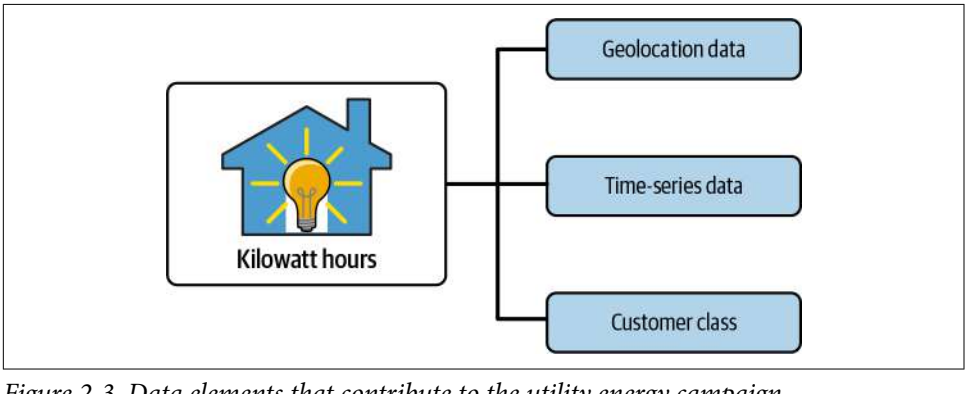


Figure 2-3. Data elements that contribute to the utility energy campaign.

4. Insurance: Advertising Media Channel Sales Prediction

In this section, you work on a team charged with developing a media strategy for an insurance company. The team wants to develop an ML model to predict sales based on advertising spend in various media channels. You are tasked with performing exploratory data analysis and with building and training the model. You do not have an ML background or any programming knowledge. You elect to use AutoML as your ML framework. Figure 2-4 shows the data elements that contribute to your model.

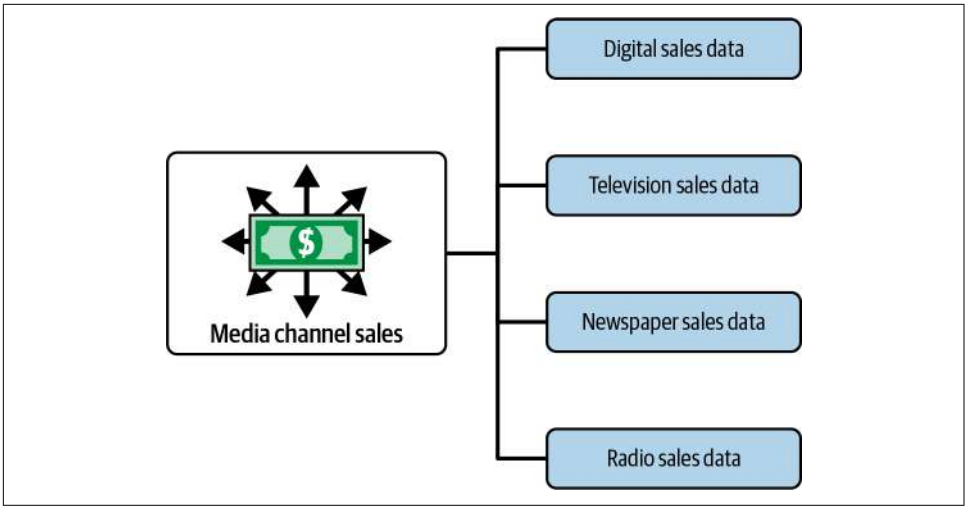


Figure 2-4. Data elements that contribute to media channel sales prediction.

5. Financial: Fraud Detection

Your goal in this project is to build a model to predict whether a financial transaction is fraudulent or legitimate. Your new company is a mobile payment service that serves hundreds of thousands of users. Fraudulent transactions are fairly rare and are usually caught by other protections. However, the unfortunate truth is that some of these are slipping through the cracks and negatively impacting your users. The dataset in this section consists of transaction data that has been simulated to replicate user behavior and fraudulent transactions. You do not have an ML background or any programming knowledge. You elect to use AutoML as your ML framework. Figure 2-5 shows the data elements that contribute to your model.

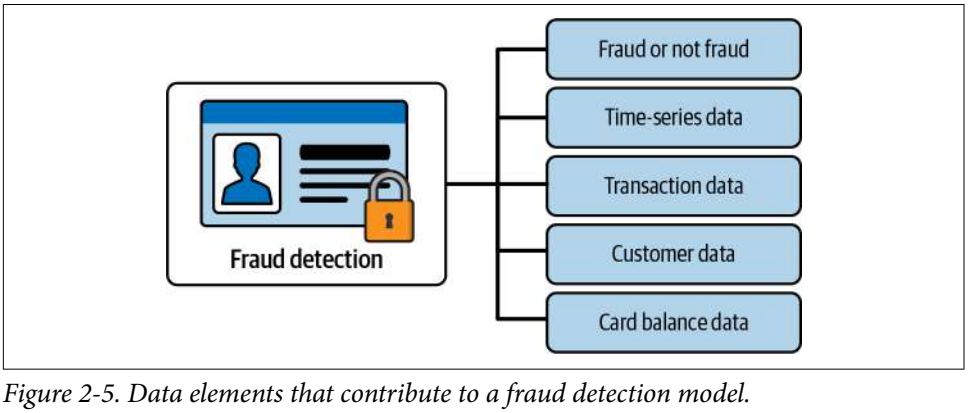


Figure 2-5. Data elements that contribute to a fraud detection model.

6. Energy: Power Production Prediction

Your goal in this project will be to predict the net hourly electrical energy output for a combined cycle power plant (CCPP) given the weather conditions near the plant at the time. The dataset in this section contains data points collected from a CCPP over a six-year period (2006–2011) when the power plant was set to work with a full load. The data is aggregated per hour, though the exact hour for the recorded weather conditions and energy production is not supplied in the dataset. From a practical viewpoint, this means that you will not be able to treat the data as sequence or time-series data, where you use information from previous records to predict future records. You have some Structured Query Language (SQL) knowledge from working with databases. You elect to use Google's BigQuery Machine Learning as your ML framework. Figure 2-6 shows the data elements that contribute to your model.

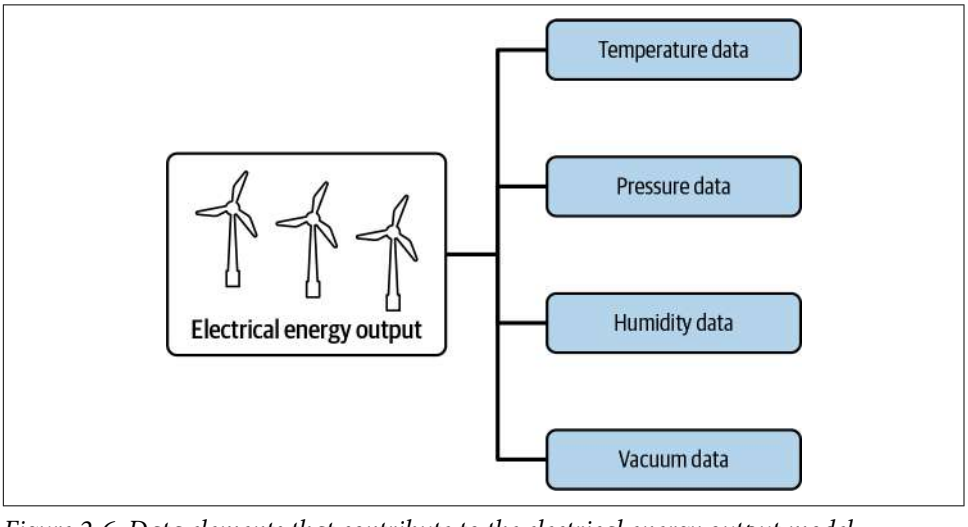


Figure 2-6. Data elements that contribute to the electrical energy output model.

7. Telecommunications: Customer Churn Prediction

Your goal in this project will be to predict customer churn for a telecommunications company. *Customer churn* is defined as the *attrition rate* for customers, or in other words, the rate of customers that choose to stop using services. Telecommunications companies often sell their products at a monthly rate or via annual contracts, so *churn* here will represent when a customer cancels their subscription or contract in the following month. The dataset contains both numeric variables and categorical variables, where the variable takes on a value from a discrete set of possibilities. You have some Python knowledge and find AutoML very powerful, yet are looking to learn low-code solutions that allow you to have a bit more control over your model.

You elect to use scikit-learn and Keras as ML frameworks. Figure 2-7 shows the data elements that contribute to your model.

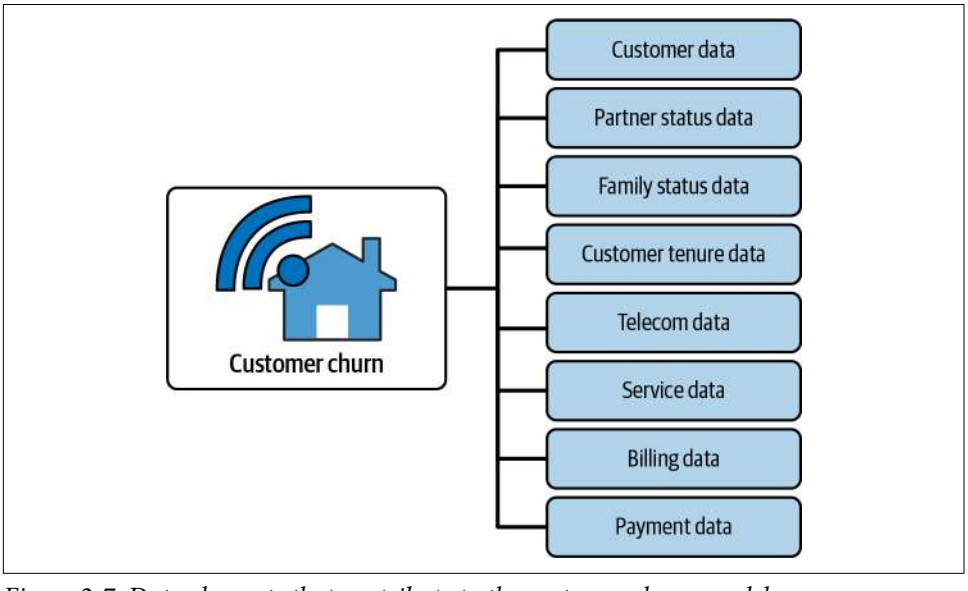


Figure 2-7. Data elements that contribute to the customer churn model.

8. Automotive: Improve Custom Model Performance

Your goal in this project (as a newer member of an ML team) will be to improve the performance of an ML model trained to predict the auction price of used cars. The initial model is a linear regression model in scikit-learn and does not quite meet your business goals. You will ultimately explore using tools in scikit-learn, Keras, and BigQuery ML to improve your model performance. The training, validation, and testing datasets used for training the linear regression model have been supplied to you as CSV files. These datasets have been cleaned (missing and incorrect values have been remedied appropriately), and the code that was used to build the scikit-learn linear regression model has also been provided. Figure 2-8 shows the data elements that contribute to your model.

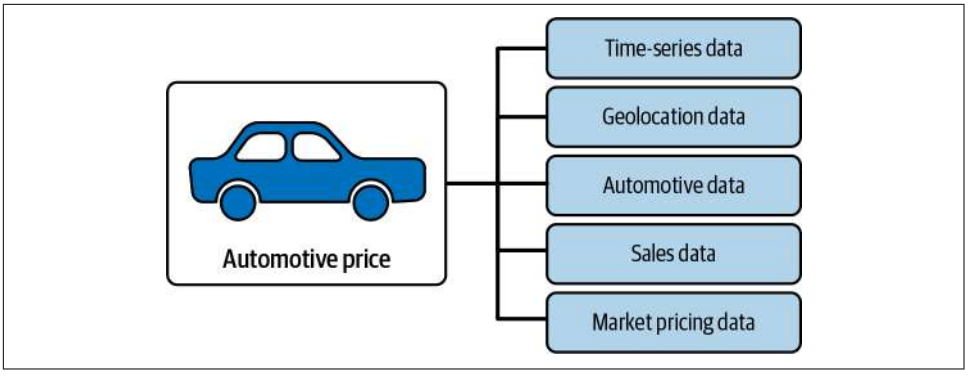


Figure 2-8. Data elements that contribute to the automotive pricing model.

Data and File Types

Data is really the first step, so let's go over some basic terminology and concepts around data. If you are already familiar with the differences between quantitative and qualitative data; between structured, semistructured, and unstructured data; and batch and streaming data, then skip to "An Overview of GitHub and Google's Colab" on page 27, where you can start creating the Jupyter Notebook in GitHub.

Quantitative and Qualitative Data

In data analysis, you work with two types of data: quantitative and qualitative. If it can be counted or measured, and given a numerical value, it's quantitative data. Quantitative data can tell you how many, how much, or how often—for example, how many people visited the website to view the product catalog? How much revenue did the company make this fiscal year? How often do the machines that manufacture your umbrella handles break?

Unlike quantitative data, qualitative data cannot be measured or counted and can include almost any non-numerical data. It's descriptive, expressed in terms of language rather than numbers. Why is this distinction important in ML? If you have qualitative data, then you need to preprocess it so that it *becomes* quantitative—that is because you cannot feed qualitative data into an ML model. You will learn how to handle some qualitative data in subsequent chapters.

Structured, Unstructured, and Semistructured Data

Data can be grouped into three buckets: structured, unstructured, and semistructured.

Structured data is information that has been formatted and transformed into a well-defined data model. A data model is a way of organizing and structuring data so that it can be easily understood and manipulated. Data models are used in a variety of applications, including databases, software applications, and data warehouses. Structured data is well organized. Table 2-2 shows the schema and data type used in Chapter 4's Advertising Media Channel Sales Prediction use case. Note that there is a column name and column type. There are four columns of numeric (quantitative) data that feed into the AutoML model.

Table 2-2. Schema and field value information for the advertising dataset from Chapter 4

Column name	Column type	Notes about field values
Digital	Numeric	Budget of advertisements in digital
Newspaper	Numeric	Budget of advertisements in newspaper
Radio	Numeric	Budget of advertisements in radio
TV	Numeric	Budget of advertisements in TV

Here are some examples of structured data:

- Customer records
- Product inventory
- Financial data
- Transaction logs
- Website analytics data
- Log files

Unstructured data is data that is not structured or tabular or formatted in a specific way. Here are some examples of unstructured data:

- Social media posts
- Chats (text)
- Videos
- Photos
- Web pages
- Audio files

Semistructured data is a type of structured data that lies between structured and unstructured data. It doesn't have a tabular data model but can include tags and semantic markers for records and fields in a dataset. Semistructured data is, essentially, a combination of structured and unstructured. Videos may contain meta tags that relate to the date or location, but the information within has no structure.

Here are some examples of semistructured data:

- CSV, XML, JSON files
- HTML
- Email (Emails are considered semistructured data because they have some structure, but not as much as structured data. Emails typically contain a header, a body, and attachments. The header contains information about the sender, recipient, and date of the message. The body of the message contains the text of the message.)

Figure 2-9 compares unstructured, semistructured, and structured data.

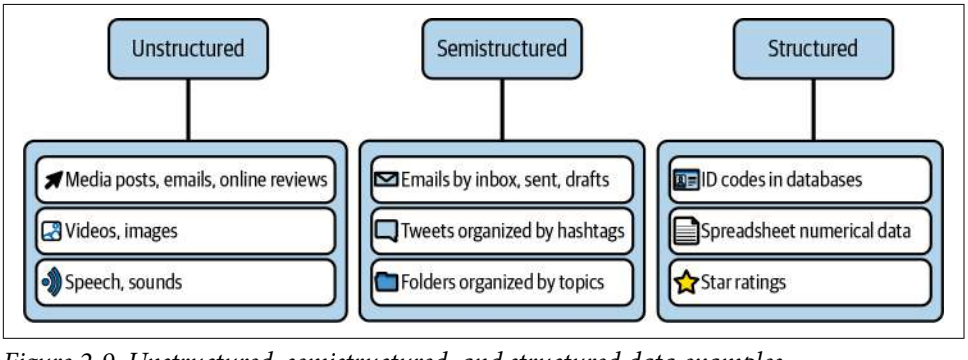


Figure 2-9. Unstructured, semistructured, and structured data examples.

Data File Types

You just learned about the different types of data, and several file types were mentioned. There are many different types of data file formats, each with its own purpose. Table 2-3 shows some of the most common data file types.

Table 2-3. Common data file types

Common data file types	Common file extensions
Text files are files that contain plain text. They are typically used to store documents, such as letters, reports, and code.	Some common text file extensions include .txt, .csv, .tsv, .log, and .json.
Spreadsheet files are files that contain data in a tabular format. They are typically used to store financial data, sales data, and other tabular data.	Some common spreadsheet file extensions include .xls, .xlsx, and .csv.
Image files are files that contain images. They are typically used to store photos, graphics, and other visual content.	Some common image file extensions include .jpg, .png, and .gif.
Audio files are files that contain audio recordings. They are typically used to store music, podcasts, and other audio content.	Some common audio file extensions include .mp3, .wav, and .ogg.
Video files are files that contain video recordings. They are typically used to store movies, TV shows, and other video content.	Some common video file extensions include .mp4, .avi, and .mov.
Webpage files are files that contain webpages. They are typically used to store HTML code, CSS code, and JavaScript code.	Some common webpage file extensions include .html, .htm, and .php.

How Data Is Processed

There are two main modes of how data is processed: batch processing and real-time processing. Batch processing is a mode of data processing where data is collected over a period of time and then processed at a later time. This is a common mode of data processing for large datasets, as it can be more efficient to process the data in batches than to process it in real time. Real-time processing is a mode of data processing where data is processed as soon as it is collected. This is a common mode of data processing for applications where the data needs to be processed quickly, such as fraud detection or stock trading.

The frequency of how data is processed can also vary. Continuous processing is a mode of data processing where data is processed continuously, as it is collected. This is a common mode of data processing for applications where the data needs to be processed in real time. Periodic processing is a mode of data processing where data is processed at regular intervals. This is a common mode of data processing for applications where the data does not need to be processed in real time, such as financial reporting.

The mode and frequency of how data is processed depends on the specific needs of the application. For example, an application that needs to process large datasets may use batch processing, while an application that needs to process data in real time may use real-time processing. Table 2-4 summarizes the different modes and frequencies of data processing.

Table 2-4. Summary of the different modes and frequencies of data processing

Mode	Frequency	Description
Batch processing	Intermittent	Data is collected over a period of time and then processed at a later time.
Real-time processing	Continuous	Data is processed as soon as it is collected.
Periodic processing	Intermittent	Data is processed at regular intervals.

Batch data and streaming data are two different types of data that are processed differently.

- *Batch* data is data that is collected over a period of time and then processed at a later time.
- *Streaming* data is data that is processed as it is received.

Batch data requires data to be collected in batches before it can be processed, stored, analyzed, and fed into an ML model.

Streaming data flows in continuously and can be processed, stored, analyzed, and acted on as soon as it is generated. Streaming data can come from a wide variety of distributed sources in many different formats. Simply stated, streaming data is data that is generated continuously and in real time. This type of data can be used to train ML models that can make predictions in real time. For example, a streaming data model could be used to detect fraud or predict customer churn.

An Overview of GitHub and Google's Colab

This section talks about how to set up a Jupyter Notebook and GitHub project repository. The GitHub repository can hold your datasets and the low-code project notebooks you create—such as the Jupyter Notebooks mentioned in this book.

Use GitHub to Create a Data Repository for Your Projects

GitHub is a code repository where you store your Jupyter notebooks and experimental raw data for free. Let's get started!

1. Sign up for a new GitHub account

GitHub offers personal accounts for individuals and organizations. When you create a personal account, it serves as your identity on *GitHub.com*. When you create a personal account, you must select a billing plan for the account.

2. Set up your project's GitHub repo

To set up your first GitHub repo, see the full steps in the "Use GitHub to Create a Data Repository for Your Projects" page in Chapter 2 of the book's GitHub repo. You can also refer to GitHub documentation on how to create a repo.

Type a short, memorable name for your repository; for example, low-code book projects. A description is optional, but in this exercise, enter **Low-code AI book projects**. Choose a repository visibility—in this case, the default is Public, which means anyone on the internet can see this repository. Figure 2-10 shows what your setup should look like.

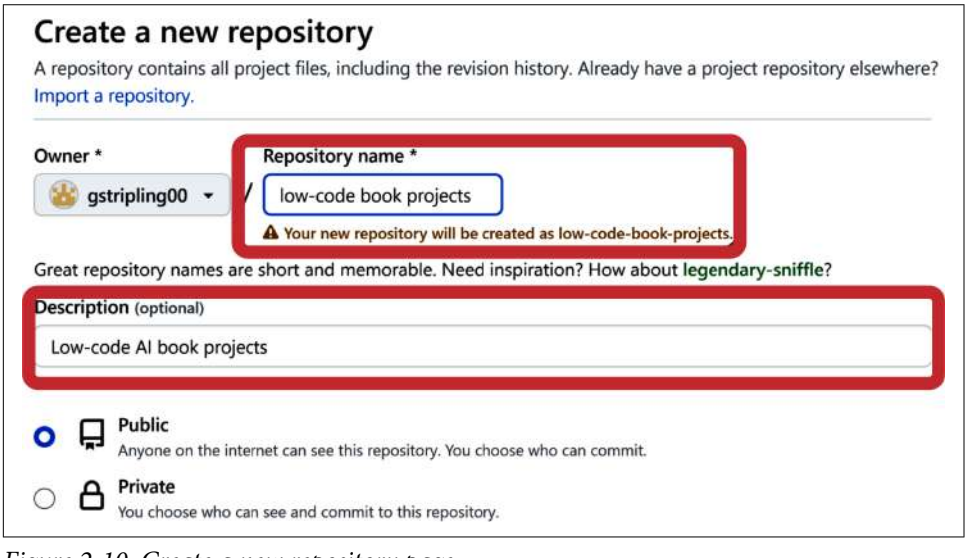


Figure 2-10. Create a new repository page.

Have GitHub create a *README.md* file. This is where you can write a long description for your project. Keep the other defaults: `.gitignore` lets you choose which files not to track, and a license tells others what they can and can't do with your code. Lastly, GitHub reminds you that you are creating a public repository in your personal account. When done, click "Create repository." Figure 2-11 shows what the page should look like.

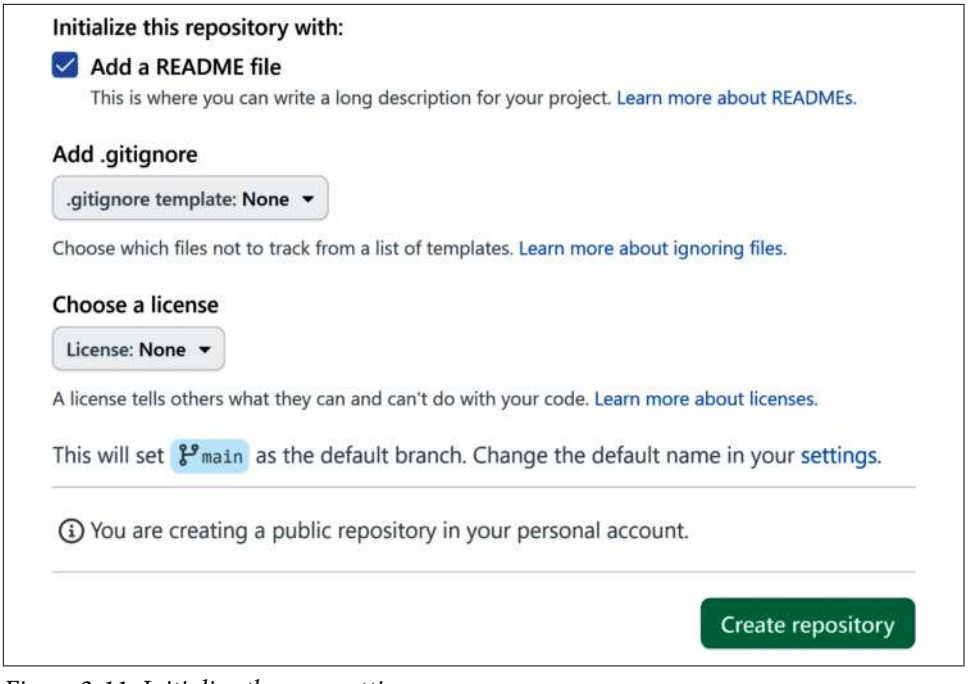


Figure 2-11. Initialize the repo settings.

After clicking "Create repository," the repo page appears, as shown in Figure 2-12.

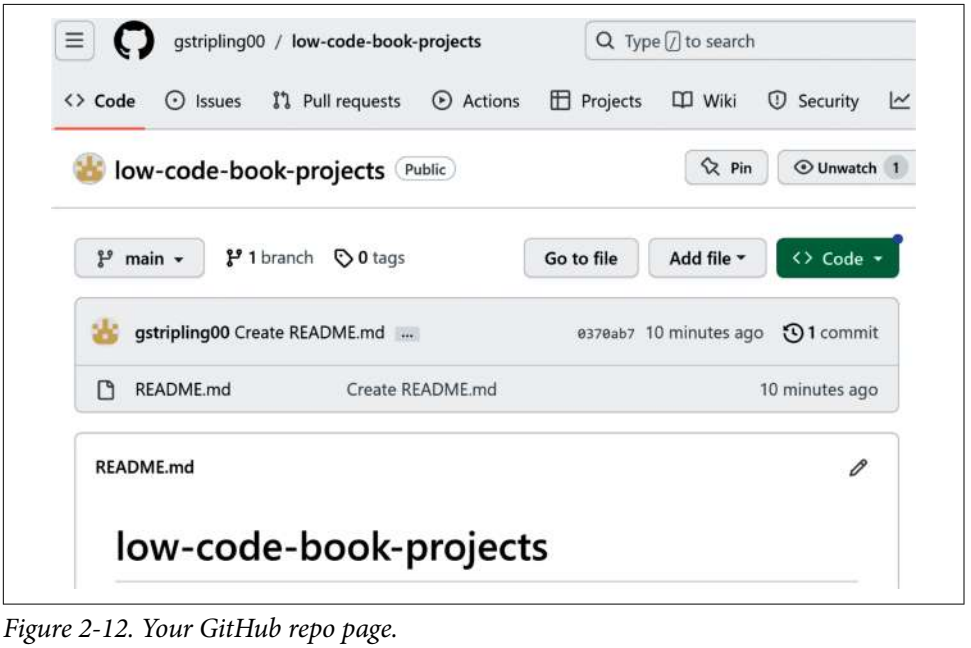


Figure 2-12. Your GitHub repo page.



In the next section, you will create a Jupyter Notebook in Google's Colaboratory. You will save the notebook file from Colab into GitHub, which will create a file under the *Main* branch. Creating a file in GitHub is a great way to improve collaboration on a project. It provides a number of features that can help teams work more effectively together:

Version control
: GitHub tracks changes to files. This means that everyone who has access to the file can see the changes that have been made, and they can revert to a previous version if necessary.

Pull requests
: Pull requests allow collaborators to propose changes to a file. This gives everyone a chance to review the changes before they are merged into the main branch.

Issues
: Issues can be used to track bugs or feature requests. This allows everyone to collaborate on solving problems and adding new features.

Comments
: Comments can be added to files to provide feedback or ask questions. This allows for a more collaborative way of working on code.

Using Google's Colaboratory for Low-Code AI Projects

Years ago, if you wanted to learn Python, you had to download the Python interpreter and install it on your computer. This could be a daunting task for beginners, as it required knowledge of how to install software and configure your computer. Today, there are many ways to learn Python without having to install anything on your computer. You can use online IDEs (integrated development environments) that allow you to write and run Python code in a web browser. You can also use cloud-based Python environments that provide you with access to a Python interpreter and all the libraries you need to get started.

These online and cloud-based resources make it easier than ever to learn Python, regardless of your level of experience or technical expertise. Here are some of the benefits of using online and cloud-based resources to learn Python:

No installation required
: You can start learning Python right away, without having to download or install any software.

Access from anywhere
: You can use online and cloud-based resources to learn Python from anywhere, as long as you have an internet connection.

Affordable
: Online and cloud-based resources are often free or very affordable.

Easy to use
: Online and cloud-based resources are designed to be easy to use, even for beginners.

You build your low-code Python Jupyter Notebook using Google's Colaboratory, or Colab. Colab is a hosted Jupyter Notebook service that requires no setup to use, while providing access to computing resources, including graphical processing units (GPUs). Colab runs in your web browser and allows you to write and execute Python code. Colab notebooks are stored in Google Drive and can be shared similarly to how you share Google Docs or Sheets.

Google Colaboratory is free to use, and there is no need to sign up for any accounts or pay for any subscriptions. You can share your notebooks with others and work on projects together.

1. Create a Colaboratory Python Jupyter Notebook

Go to Colab to create a new Python Jupyter notebook. Figure 2-13 shows the home screen.

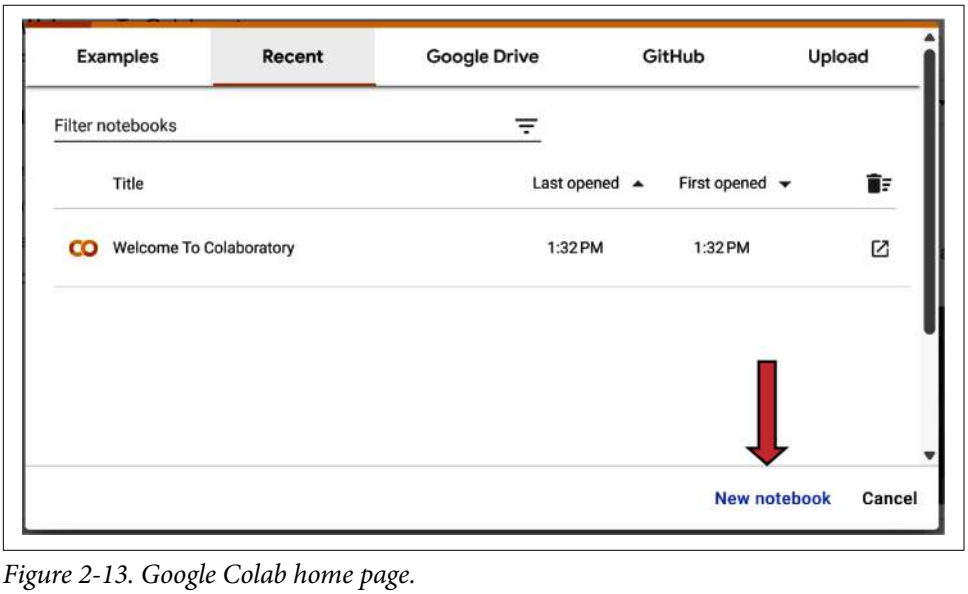


Figure 2-13. Google Colab home page.

Title the notebook in the title bar as shown in Figure 2-14 (A) and expand to show the table of contents (B). Then click the + Code button (C) to add a cell to hold your code. The + Text button allows you to add text, such as documentation.

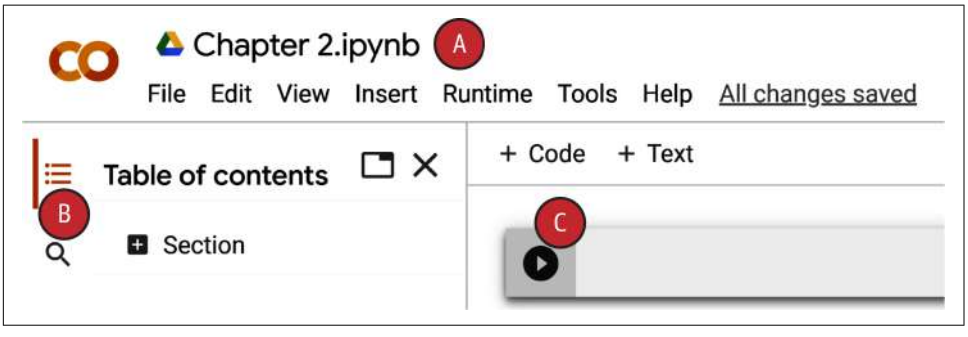


Figure 2-14. Title notebook and add a new cell code.

2. Import libraries and dataset using Pandas

Once you have added the code cell, you need to import any libraries you will need. In this simple example, you'll just import Pandas. Type **`import pandas as pd`** into the cell and run it by clicking the arrow, as shown in Figure 2-15.



Figure 2-15. Code to import Pandas.

The Pandas library is used for data analysis. Typically, when you import a library, you want to provide a way to use it without having to write out the words *Pandas* each time. Thus, the *pd* is a short-hand name (or alias) for Pandas. This alias is generally used by convention to shorten the module and submodule names.

The dataset is from the *data.gov* website. It is entitled "Heart Disease Mortality Data Among US Adults" (Figure 2-16).



Figure 2-16. Heart disease mortality data among US adults by region.

Scroll down the page until you get to the section shown in Figure 2-17. Now, there are two ways you can import the file into your Jupyter Notebook. You can download the file to your desktop and then import it, or you can use the URL. Let's use the URL method. Click on the Comma Separated Values File shown in Figure 2-17, which takes you to the URL download shown in Figure 2-18.



Figure 2-17. Downloads and resources page.



Figure 2-18. Comma separated values file URL link.

Copy the URL shown in Figure 2-18 from the website. Then, go to your Google Colab notebook and type in the code shown in Figure 2-19 into a new cell (A). Run the cell by clicking the arrow (B).

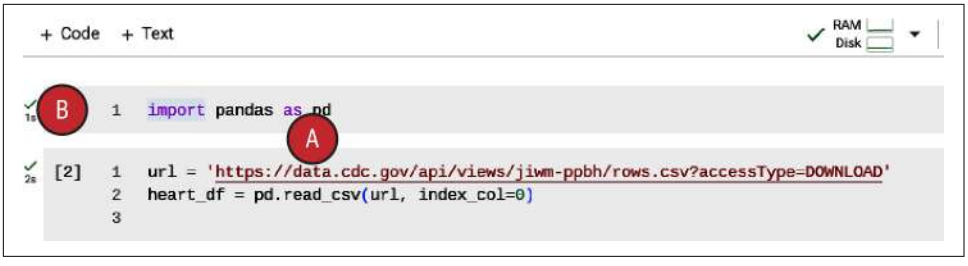


Figure 2-19. Code to read the URL into a Pandas DataFrame.

You have written code to import the dataset into a Pandas DataFrame. A Pandas DataFrame is a two-dimensional data structure that is used to store data in a table format. It is similar to a spreadsheet.

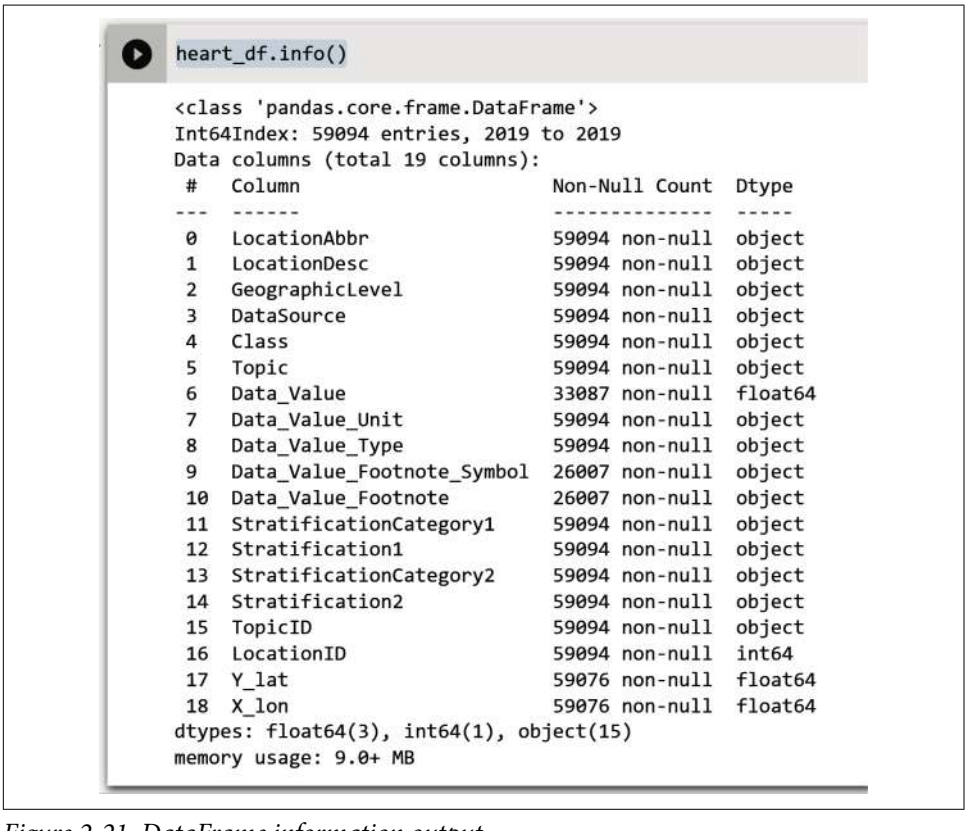
Now you add code to show the first five rows (or *head*) of the DataFrame. Add a new cell, type **`heart_df.head()`** into the cell, and run the cell. The code and output are shown in Figure 2-20.

```
heart_df.head()
```

	GeographicLevel	DataSource	Class	Topic	Data_Value	Data_Value_Unit	Data_Value_Type
	County	NVSS	Cardiovascular Diseases	Heart Disease Mortality	182.4	per 100,000 population	Age-adjusted, Spatially Smoothed, 3-year Avera...
	County	NVSS	Cardiovascular Diseases	Heart Disease Mortality	172.6	per 100,000 population	Age-adjusted, Spatially Smoothed, 3-year Avera...
	County	NVSS	Cardiovascular Diseases	Heart Disease Mortality	255.6	per 100,000 population	Age-adjusted, Spatially Smoothed, 3-year Avera...
	County	NVSS	Cardiovascular Diseases	Heart Disease Mortality	343.4	per 100,000 population	Age-adjusted, Spatially Smoothed, 3-year Avera...
	County	NVSS	Cardiovascular Diseases	Heart Disease Mortality	NaN	per 100,000 population	Age-adjusted, Spatially Smoothed, 3-year Avera...

Figure 2-20. First five rows of the DataFrame. Some columns were removed for the sake of readability.

Add a new code cell. Type **`heart_df.info()`** and run the cell to see information on the DataFrame. The `.info()` method gives you information on your dataset. The information contains the number of columns, column labels, column data types, memory usage, range index, and the number of cells in each column (non-null values). Figure 2-21 shows the output. Exact values may differ depending on when data is downloaded.



```
heart_df.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 59094 entries, 2019 to 2019
Data columns (total 19 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   LocationAbbr                 59094 non-null  object
 1   LocationDesc                 59094 non-null  object
 2   GeographicLevel              59094 non-null  object
 3   DataSource                   59094 non-null  object
 4   Class                        59094 non-null  object
 5   Topic                        59094 non-null  object
 6   Data_Value                   33087 non-null  float64
 7   Data_Value_Unit              59094 non-null  object
 8   Data_Value_Type              59094 non-null  object
 9   Data_Value_Footnote_Symbol   26007 non-null  object
 10  Data_Value_Footnote          26007 non-null  object
 11  StratificationCategory1      59094 non-null  object
 12  Stratification1              59094 non-null  object
 13  StratificationCategory2      59094 non-null  object
 14  Stratification2              59094 non-null  object
 15  TopicID                      59094 non-null  object
 16  LocationID                   59094 non-null  int64
 17  Y_lat                        59076 non-null  float64
 18  X_lon                        59076 non-null  float64
dtypes: float64(3), int64(1), object(15)
memory usage: 9.0+ MB
```

Figure 2-21. DataFrame information output.

From what the `.info()` output shows, you have 15 string object columns (which is qualitative data) and 4 numeric columns (quantitative data). Think of `int64` as a number without a decimal (for example, 25) and `float64` as a number with a decimal (25.5).

3. Data validation

As a best practice, validate any data you import from a URL—especially if you have a CSV file format to compare it with. If the dataset page had listed more metadata about the data, such as the number of columns and the column names, you could have avoided the steps to follow. But alas, it is the nature of working with data!

Now, return to the *data.gov* page and download the CSV file to your computer. You are going to validate that the file you have downloaded matches the file you imported from the URL.

You do this by uploading the downloaded file to your Colab notebook and then reading that file into a Pandas DataFrame. Expand the table of contents in your Chapter 2 notebook by selecting the folder shown in Figure 2-22 (A). Then, to upload a file, select the up arrow folder (B).

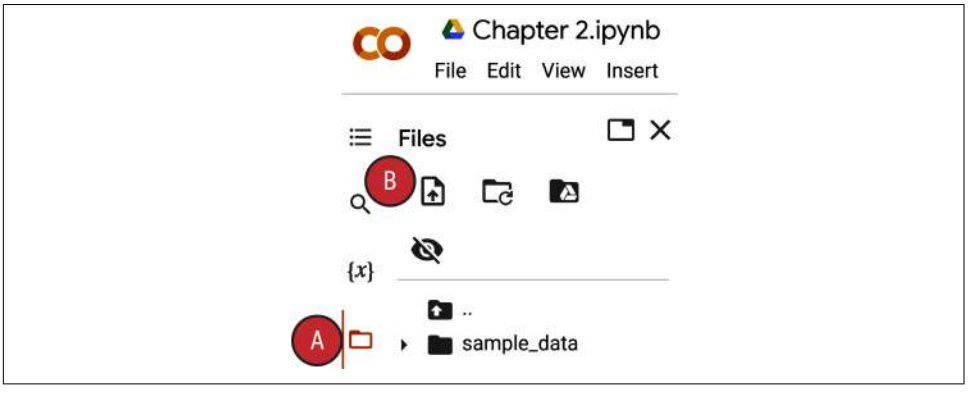


Figure 2-22. Upload file to your Colab notebook.

As you upload the file, you will see the warning message shown in Figure 2-23. This warning basically states that any file you upload will not be saved if the runtime is terminated (which can happen if you close out of Colab). Note that runtime provides the program with the environment it needs to run.

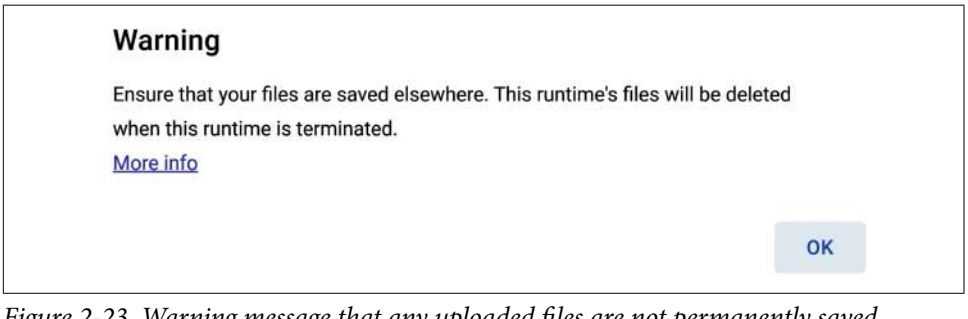


Figure 2-23. Warning message that any uploaded files are not permanently saved.

Refresh your notebook browser tab after the upload and expand it to see the table of contents. Your screen should look as shown in Figure 2-24.

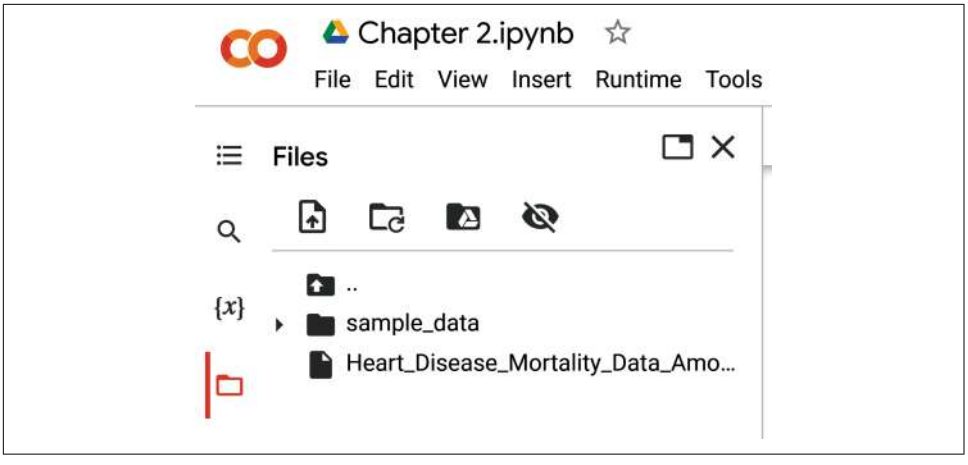


Figure 2-24. Table of contents that shows uploaded file.

Note how long the filename is—go ahead and rename it by right-clicking on the file and renaming it *heart.csv*. Your screen should look as shown in Figure 2-25.

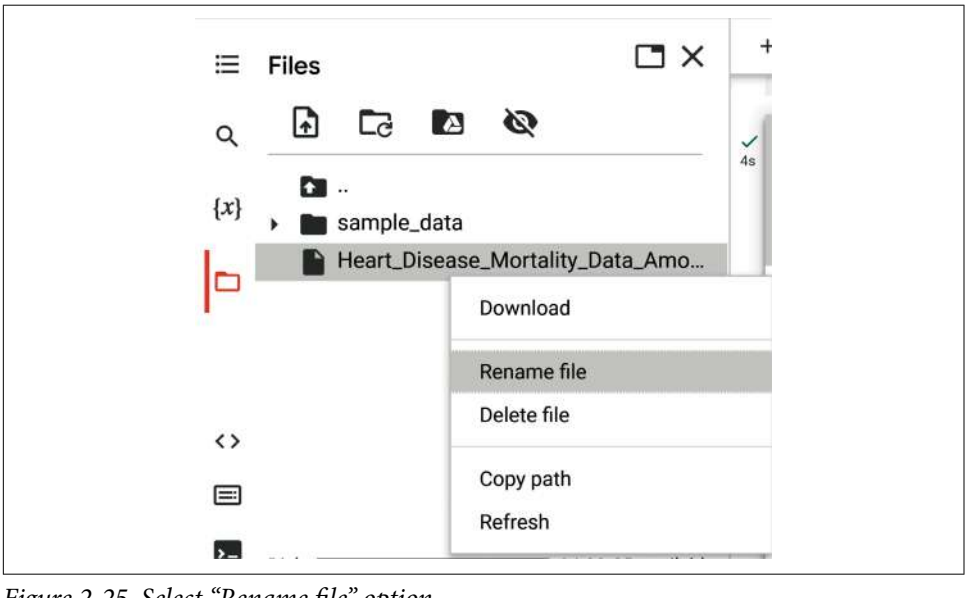


Figure 2-25. Select "Rename file" option.

Your screen should look as shown in Figure 2-26 after renaming the file.



Figure 2-26. File renamed to heart.csv.

So, you've renamed your file to *heart.csv*. Now you need to copy the path of the file, as shown in Figure 2-27. Why? You will need the exact location to input as a parameter in the Pandas `read.csv` method. Right-click on the *heart.csv* file to get the path.

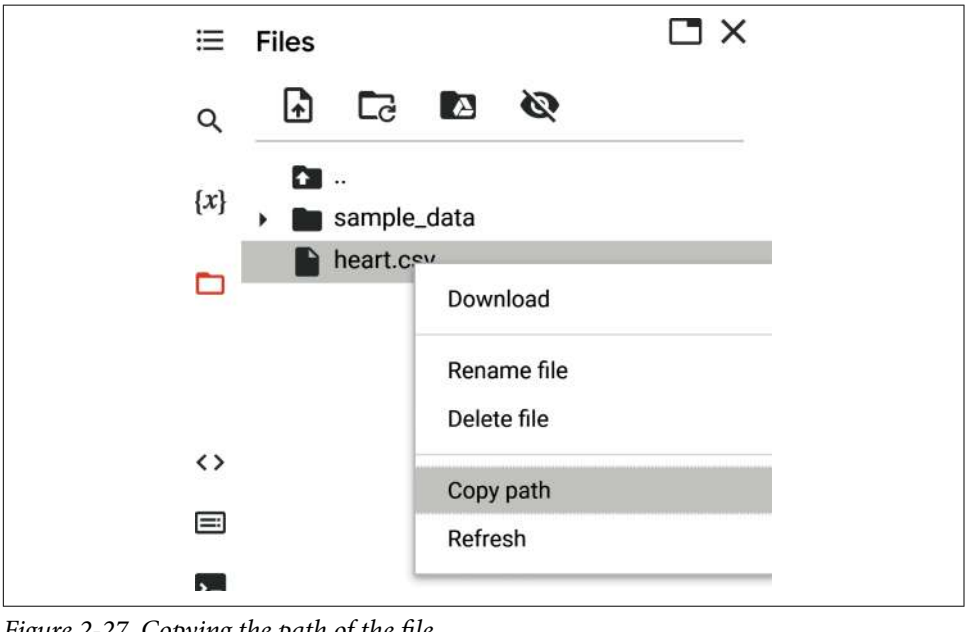


Figure 2-27. Copying the path of the file.

Add a code cell and type in the following code. Make sure to paste the path to the file between the two single quote marks around */content/heart.csv*. Run the cell and review the output:

```
heart_df = pd.read_csv('/content/heart.csv', error_bad_lines=False,
    engine="python")
heart_df.head()
```

Type **`heart_df.info()`** into a new cell. Run the cell to see DataFrame information. Compare the total columns from Figures 2-21 to 2-28. Figure 2-21 has 19 columns and Figure 2-28 has 20 columns. This means that by uploading the file, a Year column was added. The `Year` datatype is not a numeric value—it has numbers, but those numbers are meant to rank and order, not calculate on. Figure 2-28 indicates you have your first case of dirty data for our machine learning use case.

```
Data columns (total 20 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Year                        41071 non-null  int64
 1   LocationAbbr                41071 non-null  object
 2   LocationDesc                41071 non-null  object
 3   GeographicLevel             41071 non-null  object
 4   DataSource                  41071 non-null  object
 5   Class                       41071 non-null  object
 6   Topic                       41071 non-null  object
 7   Data_Value                  23316 non-null  float64
 8   Data_Value_Unit             41071 non-null  object
 9   Data_Value_Type             41071 non-null  object
 10  Data_Value_Footnote_Symbol  17755 non-null  object
 11  Data_Value_Footnote         17755 non-null  object
 12  StratificationCategory1     41071 non-null  object
 13  Stratification1             41071 non-null  object
 14  StratificationCategory2     41071 non-null  object
 15  Stratification2             41071 non-null  object
 16  TopicID                     41071 non-null  object
 17  LocationID                  41071 non-null  int64
 18  Y_lat                       41071 non-null  float64
 19  X_lon                       41071 non-null  float64
dtypes: float64(3), int64(2), object(15)
```

Figure 2-28. Year column showing as `int64`.

Type **`heart_df.isnull().sum()`** into a new cell as shown in Figure 2-29. Run the cell. Are there any null values? A *null value* is a value that indicates the absence of a value. You have your second case of dirty data! Three of your columns have null values. You will learn how to deal with missing values in a subsequent chapter. Since you really don't have a use case yet, you may be wondering whether you really need all of these features. Note that features are the input data that is used to train a model. The model then uses the features to make predictions. You'll learn about feature selection in a subsequent chapter.

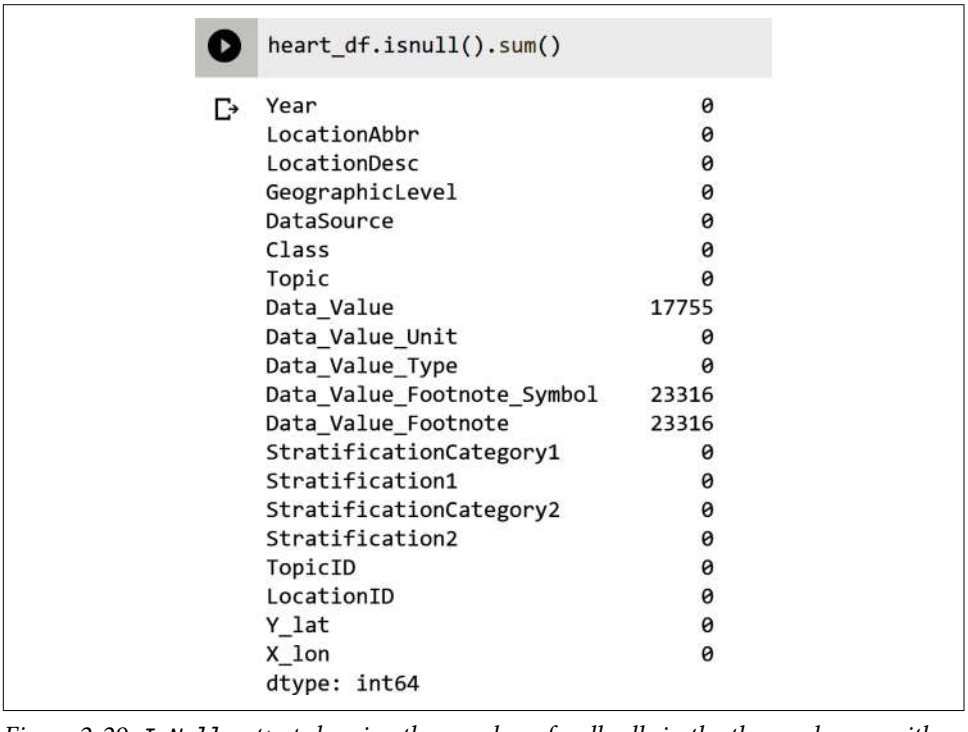


Figure 2-29. `IsNull` output showing the number of null cells in the three columns with null values.

What are your data quality issues?

Missing values
: Most algorithms do not accept missing values. Therefore, when we see missing values in our dataset, there may be a tendency to just "drop all the rows" with missing values. However, there are various ways you can deal with missing values, as explained in the following note.



There are two main approaches to handling missing values in a dataset in ML: deletion or imputation.

Deletion involves removing the rows or columns with missing values from the dataset. This can be done by dropping all rows with missing values, dropping all columns with missing values, or dropping rows or columns with a certain threshold of missing values.

Imputation involves filling in the missing values with estimates. There are many different imputation techniques, including (1) mean imputation, which replaces missing values with the mean of the observed values for that variable; (2) median imputation, which replaces missing values with the median of the observed values for that variable; (3) mode imputation, which replaces missing values with the most frequent value of the variable; and (4) regression imputation, which uses a regression model to predict the missing values based on the observed values of other variables.

Although Pandas will fill in the blank space with *NaN (not a number)*, we should handle them in some way. More on that later in the book.

Data type incorrect

`Year` is shown as an `int64` data type and should be a string object—where you would need to handle it as a qualitative, categorical feature.

Categorical columns

There are quite a number of string object features—which are not numeric. You cannot feed values like this into an ML model. These features need to be *one-hot encoded*. You'll see this in a subsequent chapter.

4. A little bit of exploratory data analysis

Before concluding this section, let's look at some simple ways to explore the data. Want to see all of the unique values in the feature `Stratification2`? Type **`heart_df.Stratification2.unique()`** into a new cell as shown in Figure 2-30. Run the cell.

```
heart_df.Stratification2.unique()

array(['Overall', 'White', 'Black', 'Hispanic',
       'Asian and Pacific Islander', 'American Indian and Alaskan Native'],
      dtype=object)
```

Figure 2-30. Code to show unique values of column `Stratification2`.

Let's use Seaborn's violin plot to visualize this feature. Seaborn is a Python library for making statistical graphics. You can write all of this in the same cell, as shown in Figure 2-31. This code uses the `Data_Value` feature as the *x* and the `Stratification2` as the *y*. Note that the `Data_Value` feature is the count of heart disease in each region and by group, as shown in Figure 2-32.

```
import seaborn as sns
sns.violinplot(x="Data_Value", y="Stratification2", data=heart_df)
```

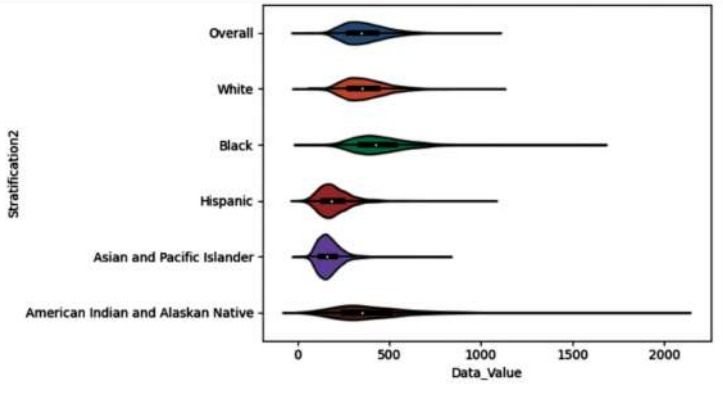


Figure 2-31. Violin plot of `Stratification2` column.

The "long tails" indicate there are more outliers in the data for that particular feature. The large shape of the violin body indicates how many heart disease cases are distributed by race.

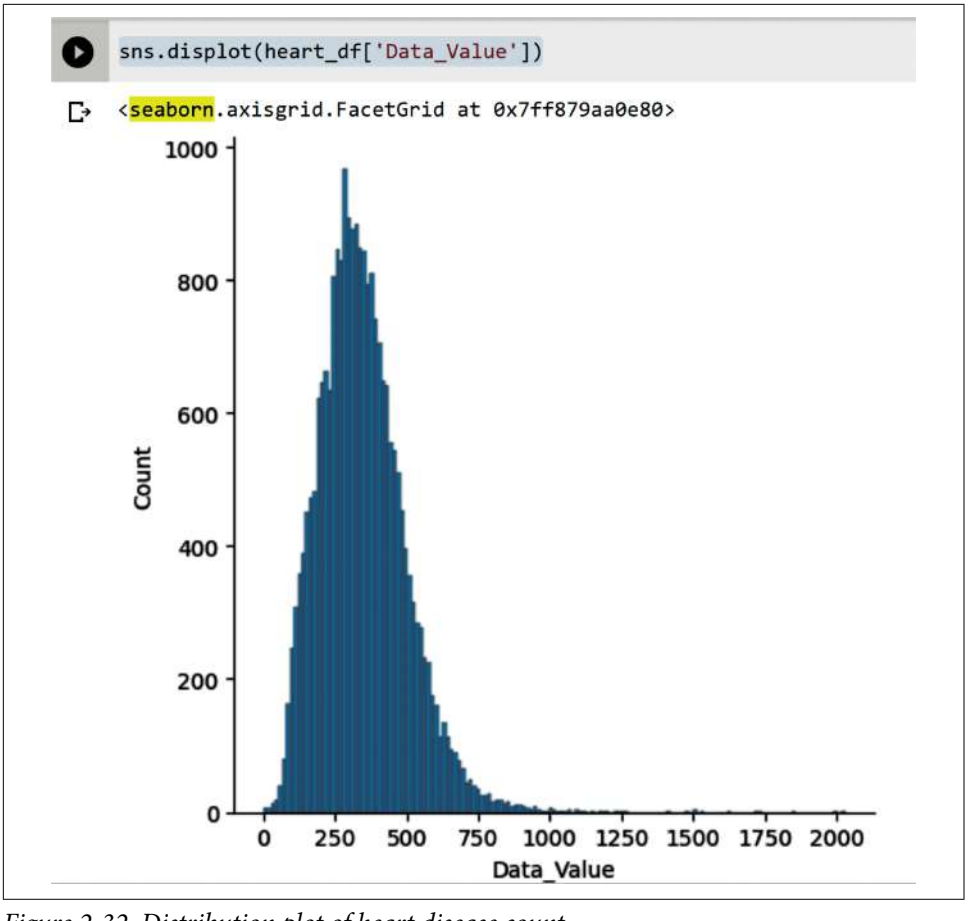


Figure 2-32. Distribution plot of heart disease count.



Seaborn's violin plot is a good way to visualize the distribution of a univariate set of data.

It is used to visualize the distribution of numerical data. Unlike a box plot that can only show summary statistics, violin plots depict summary statistics and the density of each variable.

In later chapters, you'll perform more data analysis and learn data preprocessing. For now, save your notebook to GitHub. On the Colab menu, click File > Save a copy in GitHub, as shown in Figure 2-33.

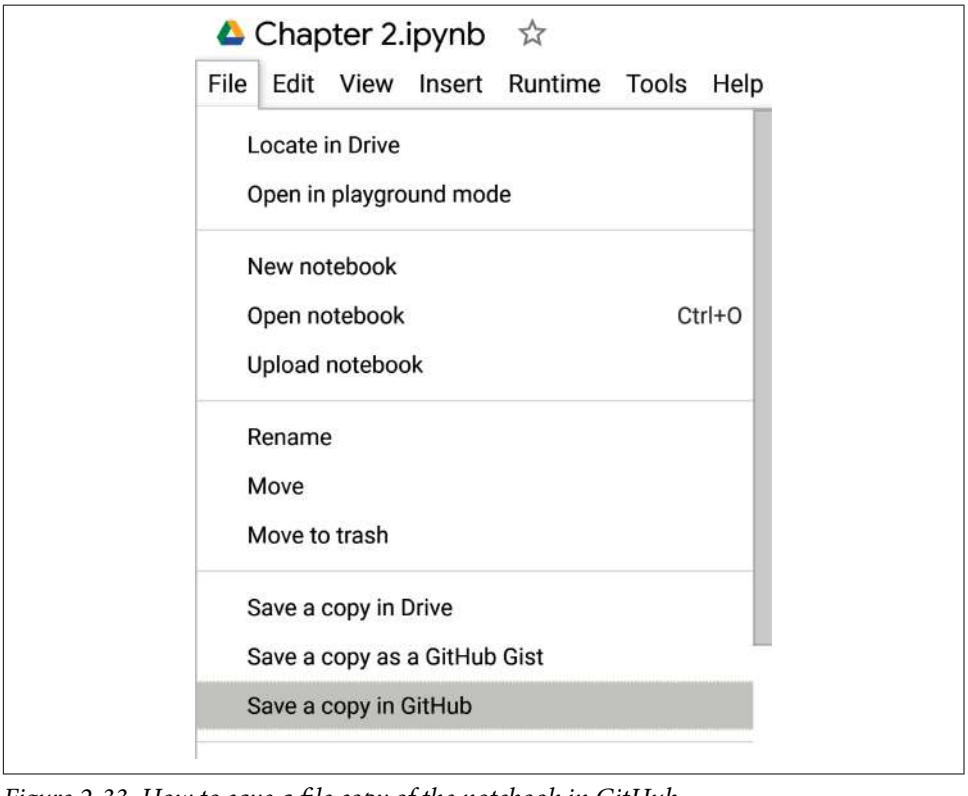


Figure 2-33. How to save a file copy of the notebook in GitHub.

Figure 2-34 shows the setup. From the drop-down under Repository, select the repo to save your notebook to. Under Branch, select Notebooks (you created this earlier). Keep the default "Include a link to Colaboratory." This will show a link in GitHub that allows you to open your notebook directly from GitHub.

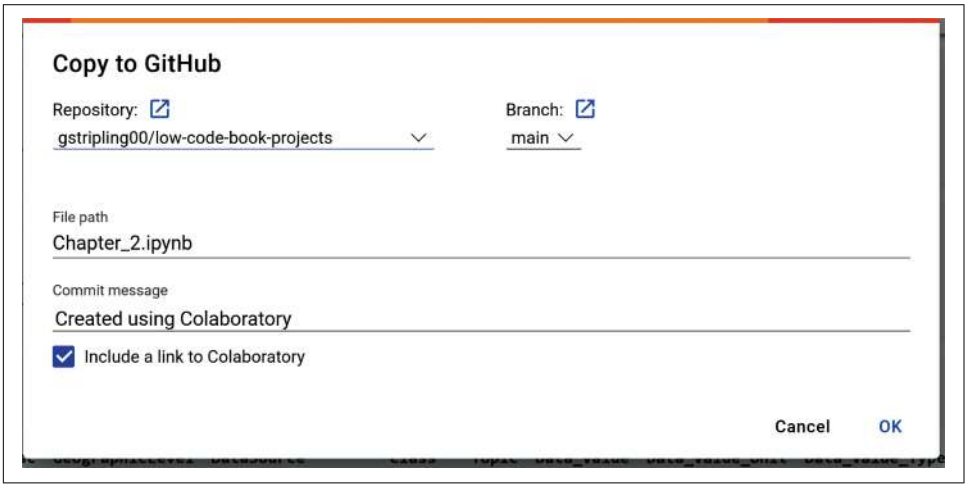


Figure 2-34. Colab's GitHub export window.

After Colab copies the notebook to GitHub, it takes you directly to GitHub, as shown in Figure 2-35.

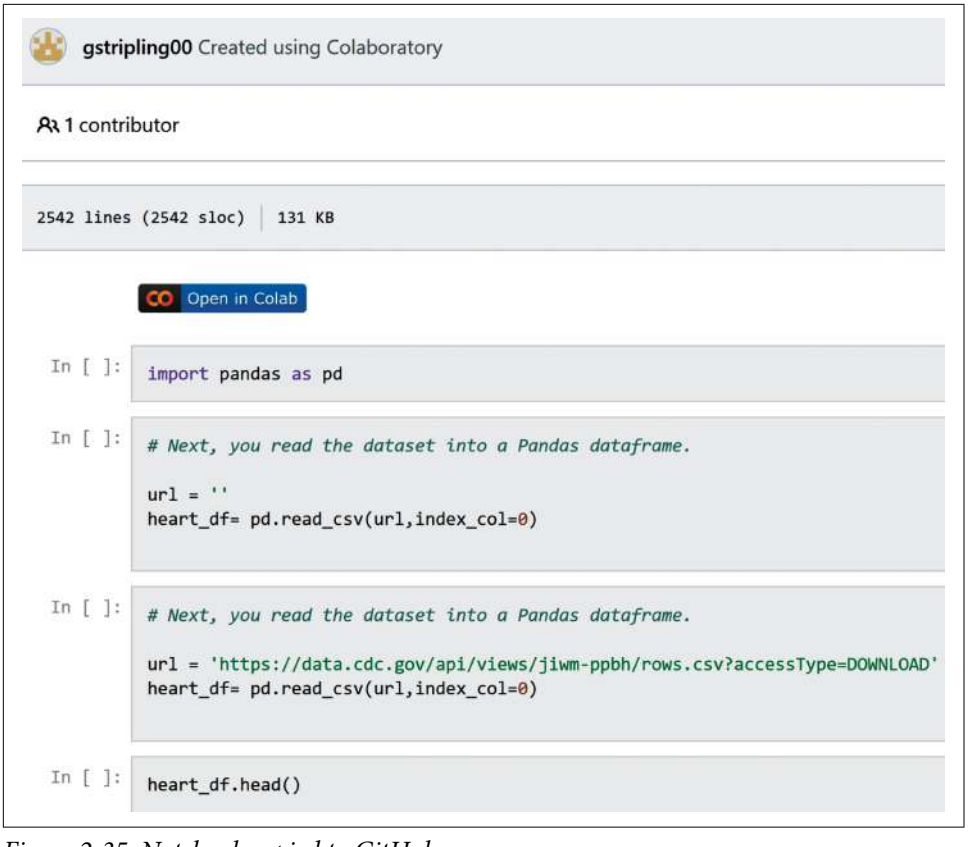


Figure 2-35. Notebook copied to GitHub.

Refresh the screen in the repo. You should see your Colab Jupyter notebook, as shown in Figure 2-36.

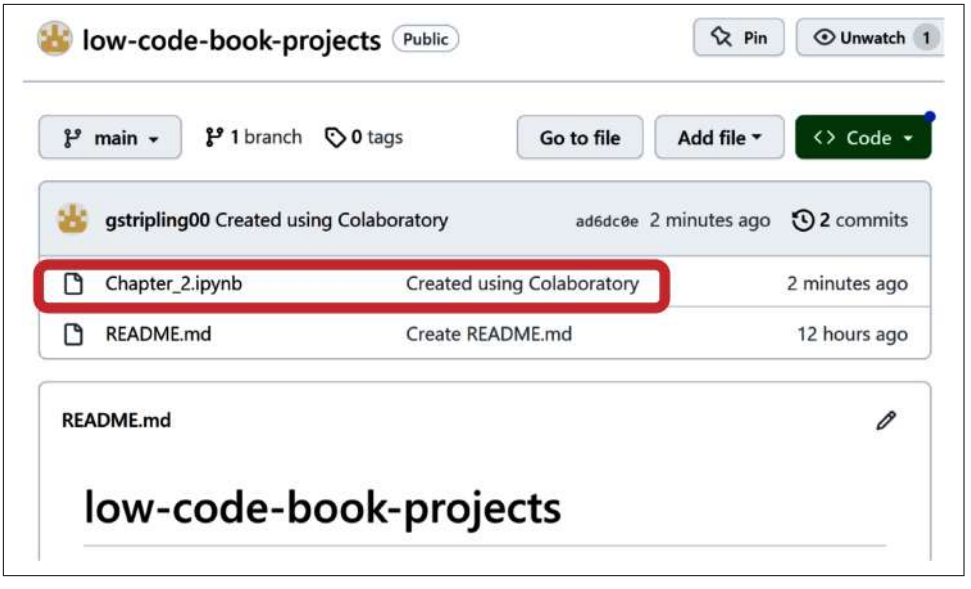


Figure 2-36. Copy of Colab Jupyter notebook in GitHub.

Summary

This chapter provided an overview of the use cases and datasets used in the book. You learned about data types and the difference between structured, semistructured, and unstructured and batch and streaming data. You got hands-on practice with a free browser-based Python Jupyter Notebook and GitHub. You discovered that dirty data is tricky and can impact data type and data ingestion into an ML model.

In the next chapter, you'll learn about ML frameworks and you'll get to work with AutoML. You are given a preprocessed dataset, and all you'll have to do is upload the dataset into the AutoML framework and you will be able to build and train a predictive model without writing a single line of code.

CHAPTER 3

Machine Learning Libraries and Frameworks

This chapter introduces machine learning (ML) frameworks that simplify the development of ML models. Typically, you need to understand the underlying working principles of mathematics, statistics, and ML to build and train ML pipelines. These frameworks help you by automating many of the time-consuming ML workflow tasks such as feature selection, algorithm selection, code writing, pipeline development, performance tuning, and model deployment.

No-Code AutoML

Imagine you are a business analyst working for a utility company. You have a project that requires you to help the company develop marketing and outreach programs that target communities with high electrical energy consumption. The data is in a comma separated value (*CSV*) file format.

You do not have an ML background or any programming knowledge—but the team lead has asked you to take on this project because you have expressed an interest in ML and how it can be applied in the organization. Although you have no coding experience, the little research you have done has yielded a few observations:

- For noncoders like yourself, there are automated *no-code* ML frameworks with a graphical user interface (GUI) that you can use to build and train an ML model without writing a single line of code.

- For light coders, there are *low-code* ML frameworks that provide the ability to build and train an ML model by writing just a little bit of code.
- For seasoned coders, there are ML libraries that allow the flexibility and control to code every phase of the ML workflow.

Based on the data from your utility marketing outreach project and use case, you determine that your target is to predict total kilowatt-hours (kWh) based on multiple variables: zip code, month, year, and customer class (Residential, Commercial, Industrial, and Agricultural).

Let's assume you need to quickly get a baseline prediction. This is an excellent use case for AutoML. A GUI-based AutoML framework is the easiest to use. Figure 3-1 shows a high-level overview of the typical AutoML no-code workflow you can use for your business use case. This example uses Google's Vertex AI, which is an ML platform that helps you build, deploy, and scale ML models. Overall, Google AutoML, Microsoft Azure AutoML, and AWS SageMaker AutoML are all powerful AutoML solutions that can help you build and deploy ML models without writing any code. The best solution for you will depend on your specific needs and requirements.

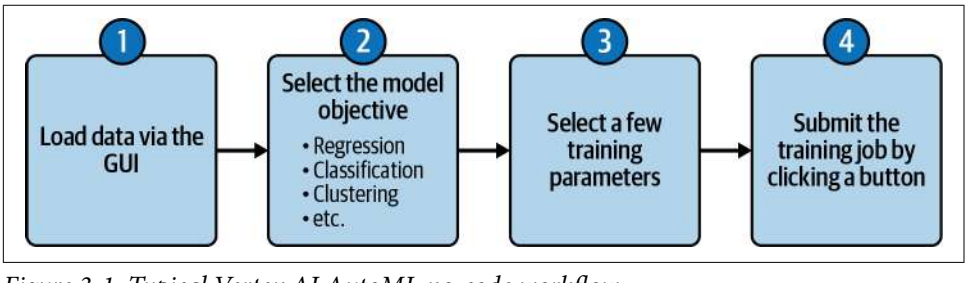


Figure 3-1. Typical Vertex AI AutoML no-code workflow.

Since the file format is CSV, you select the Tabular tab. Given that total kWh is the output and is the numeric value you want to predict, you note that this is a regression task—and since you have column names (or *labels*) for the multiple variables, it is a *supervised* ML problem. Data without labels require an unsupervised ML task, such as clustering. Figure 3-2 shows Regression/classification selected as the objective.



Vertex AI lets you create the following model types for tabular data:

Binary classification models
: These models predict a binary outcome (one of two classes). Use this type for yes or no questions.

Multi-class classification models
: These models predict one class from three or more discrete classes. Use this type for categorization.

Regression models
: These models predict a continuous value. Use this type to predict sales.

Forecasting models
: These models predict a sequence of values. Use this type to forecast daily demand.

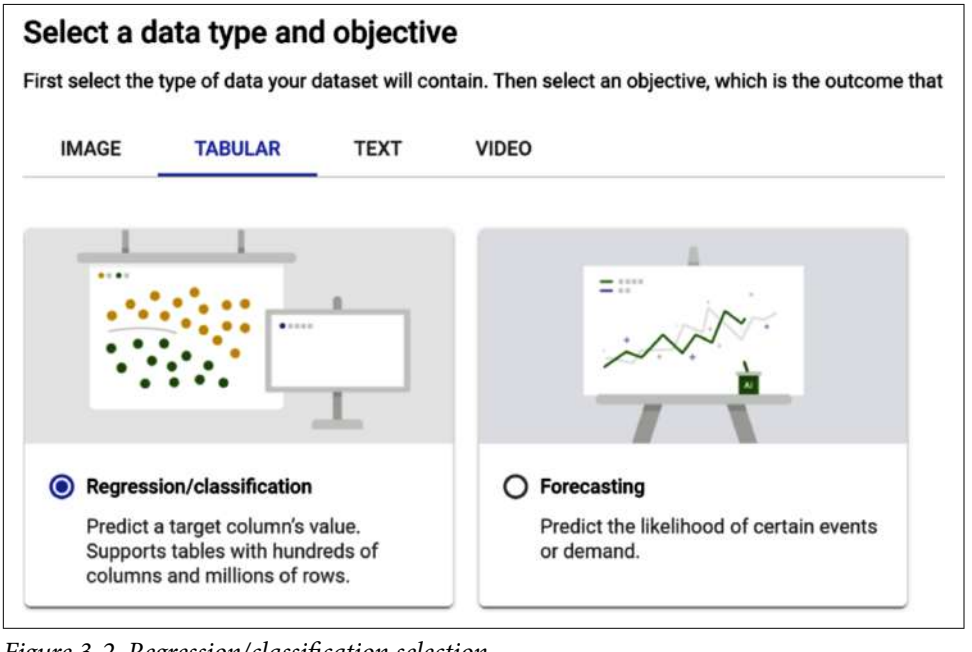


Figure 3-2. Regression/classification selection.

Some frameworks will generate statistics after the data loads. Other frameworks help minimize the need to manually clean data by automatically detecting and cleaning missing values, anomalous values, and duplicate rows and columns. Note that there are a few additional steps that you can employ, such as to review the data after it has loaded to check for missing values and view data statistics. Figure 3-3 shows the dataset upload options.

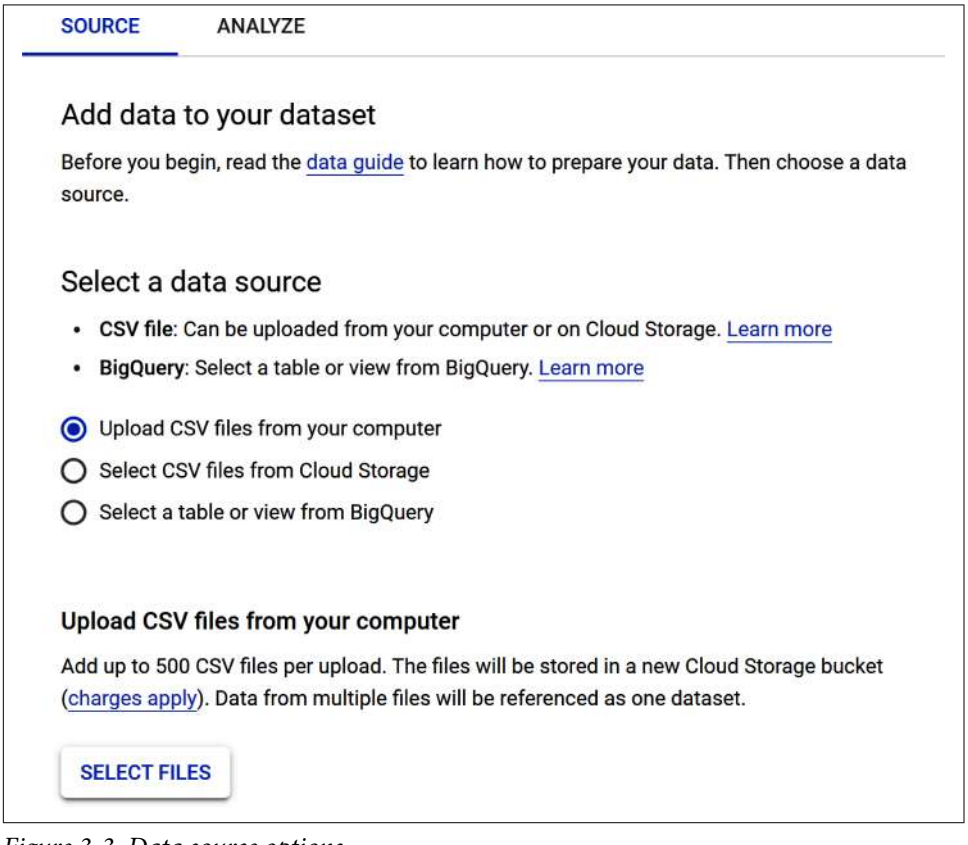


Figure 3-3. Data source options.

Figure 3-4 shows statistics of your energy utility dataset generated using Google's Vertex AI framework. There are no missing values, and the number of distinct values for each column is shown. For example, there are 145 zip codes. Because zip code is a number, the Transformation column shows it as "Numeric." However, zip code is a categorical feature, as each zip code is different from one another and can thus be put into its own "category." Changing `ZipCode` from numeric to categorical is as easy as selecting the drop-down to customize the transformation.

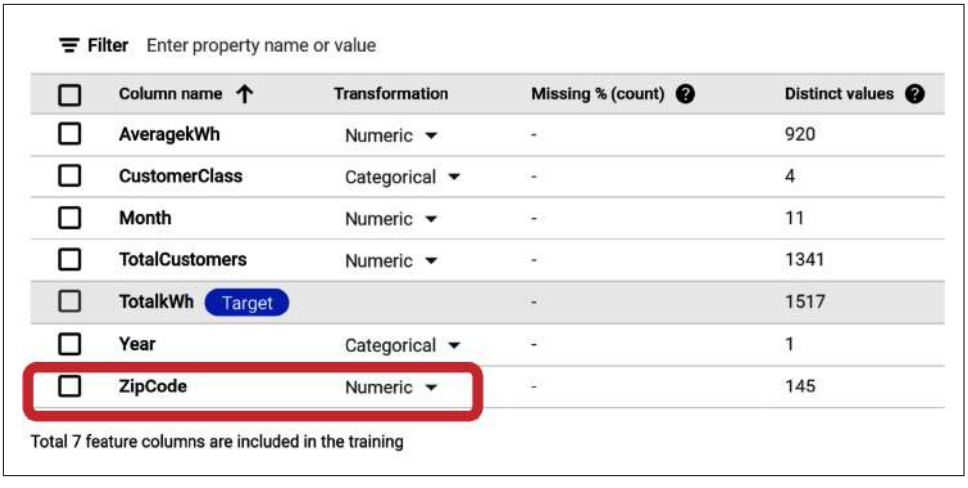


Figure 3-4. Generated statistics from Google's Vertex AI.

Figure 3-5 shows `ZipCode` now as a categorical feature. Also note the far right column, where you can select or deselect a feature to be included for training.

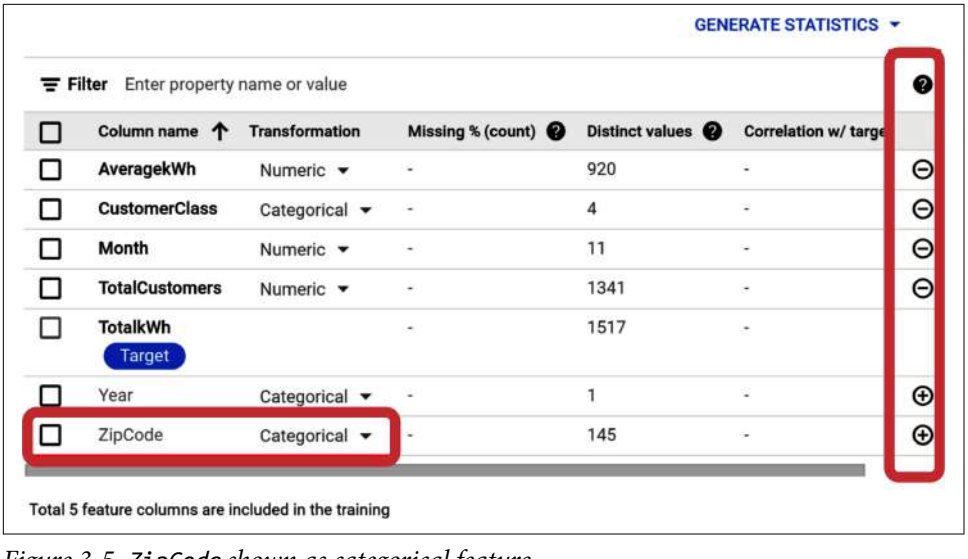


Figure 3-5. `ZipCode` shown as categorical feature.

AutoML presents a data profile of each feature. Figure 3-6 shows zip code 92694 as the most common feature, which indicates more customers live in that zip code area. You can leverage this information for your marketing campaigns.

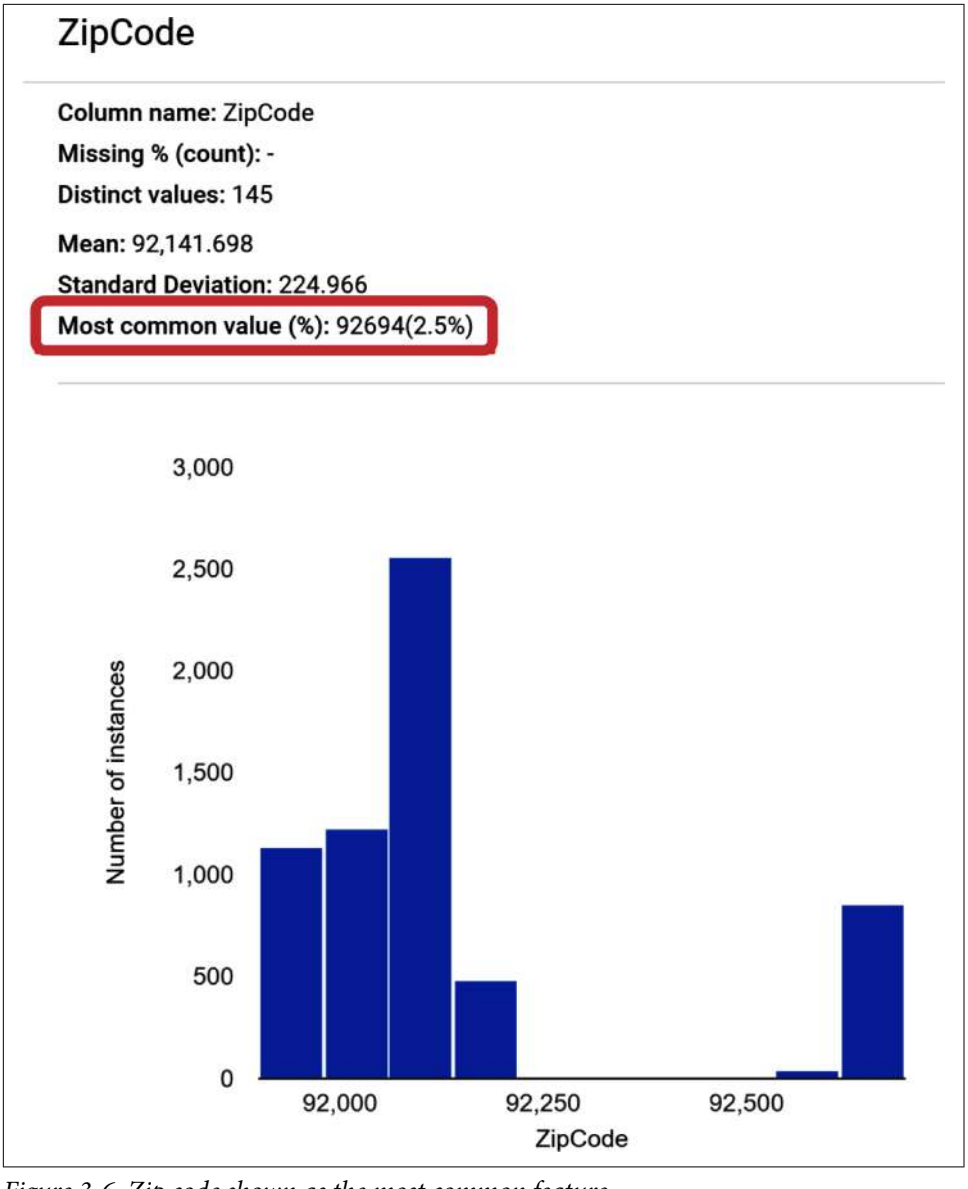


Figure 3-6. Zip code shown as the most common feature.

In Step 3, you train a new model by selecting a few training parameters. Vertex AI's "Train new model" window lets you select the training method, model details, training options, and compute and pricing. Note the dataset and the objective (Regression) are shown as inputs in the "Training method" parameter. AutoML is selected by default. Figure 3-7 shows the "Train new model" window.

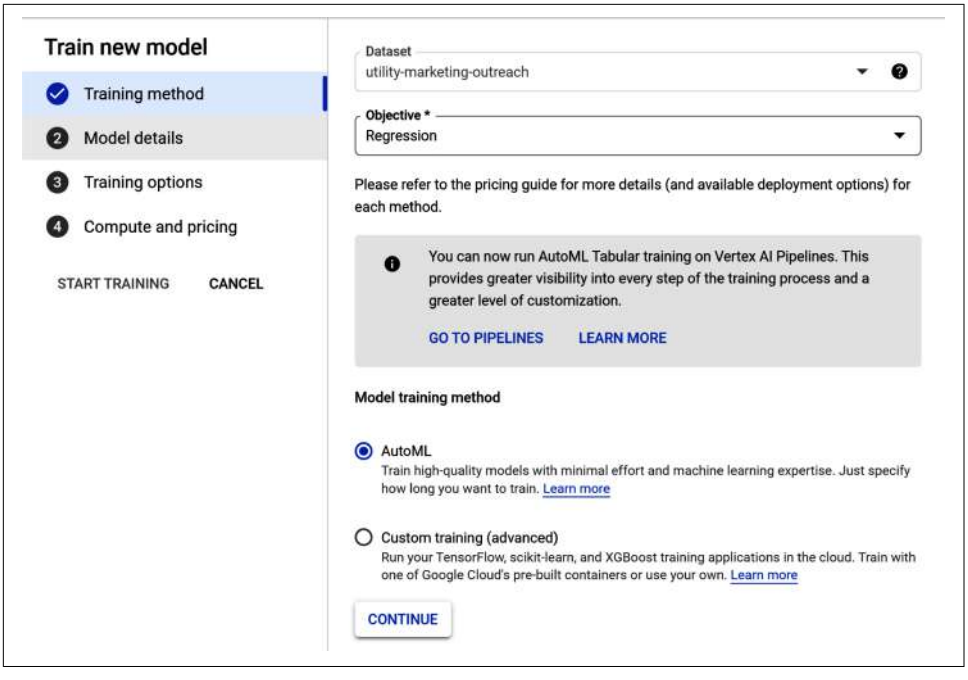


Figure 3-7. The "Train new model" window with compute and pricing selected.

Once all the parameters are entered, you start the training job. Figure 3-8 shows that it's ready to submit the training job for training.

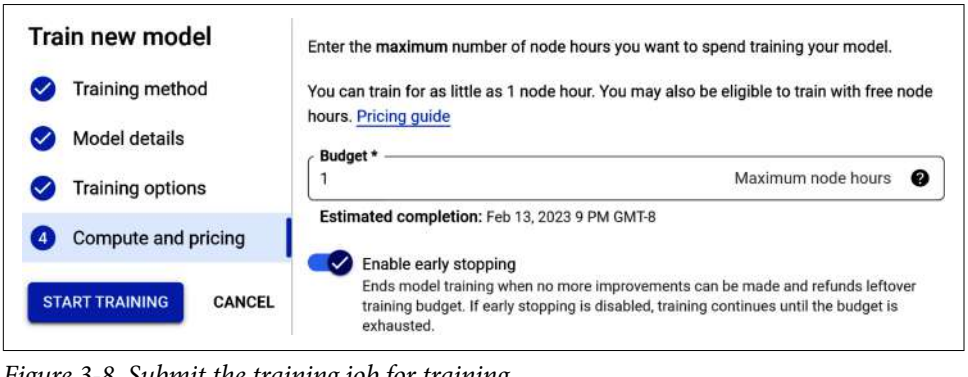


Figure 3-8. Submit the training job for training.



Training can take up to several hours, depending on the size of your data and type of model objective you choose. Image and video data types may take much longer to process than a structured data type such as a CSV file. The number of training samples also impacts training time. AutoML training is also time intensive because of the nature of AutoML and how it works.

Model training results are presented after training. You can now present your preliminary findings to the team before next steps—which could include more experimentation or deploying your model as a web page, where a user selects the customer class and zip code and the predicted total kWh is shown.

Figure 3-9 shows training results. You will see examples of a complete AutoML project in an upcoming chapter, where the metrics presented in Figure 3-9 are discussed in more detail.

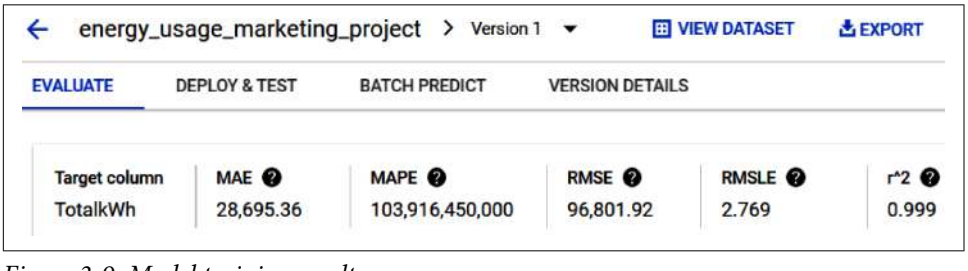


Figure 3-9. Model training results.

Model feature attribution tells you how much each feature impacted model training. Figure 3-10 shows attribution values expressed as a percentage; the higher the percentage, the stronger the correlation—that is, the more strongly that feature impacted model training. Model feature attribution is expressed using the sampled Shapley method (see the glossary on GitHub).

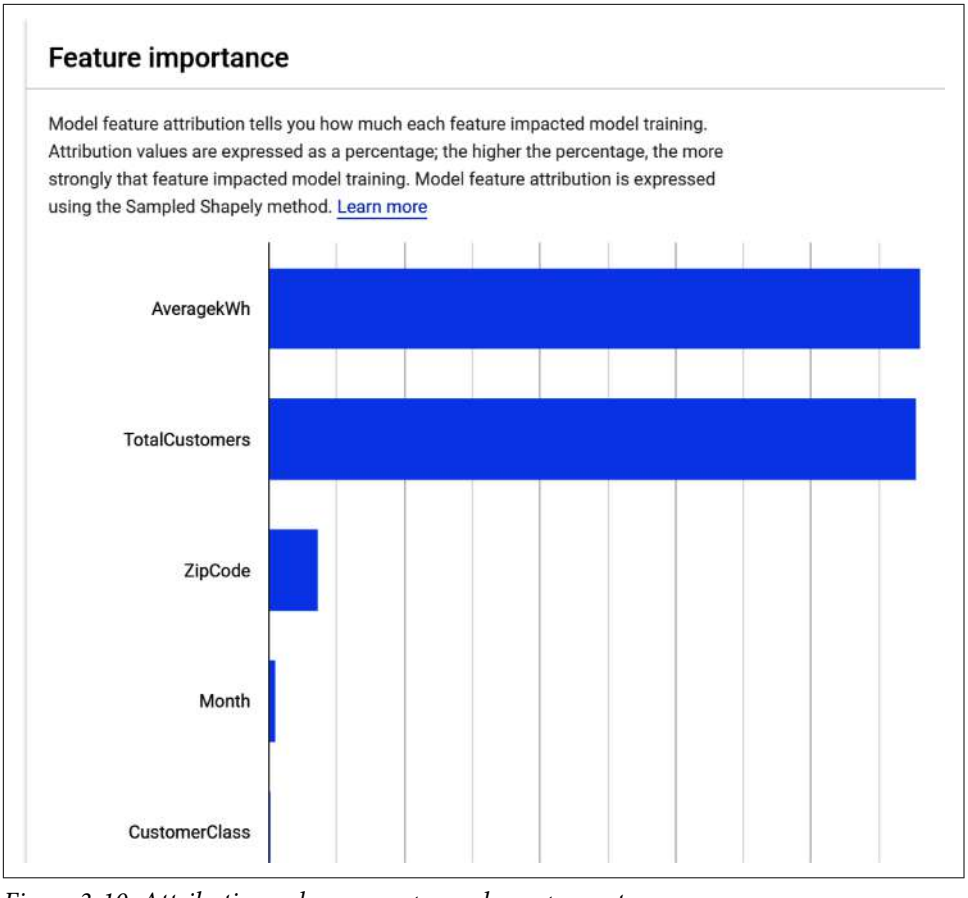
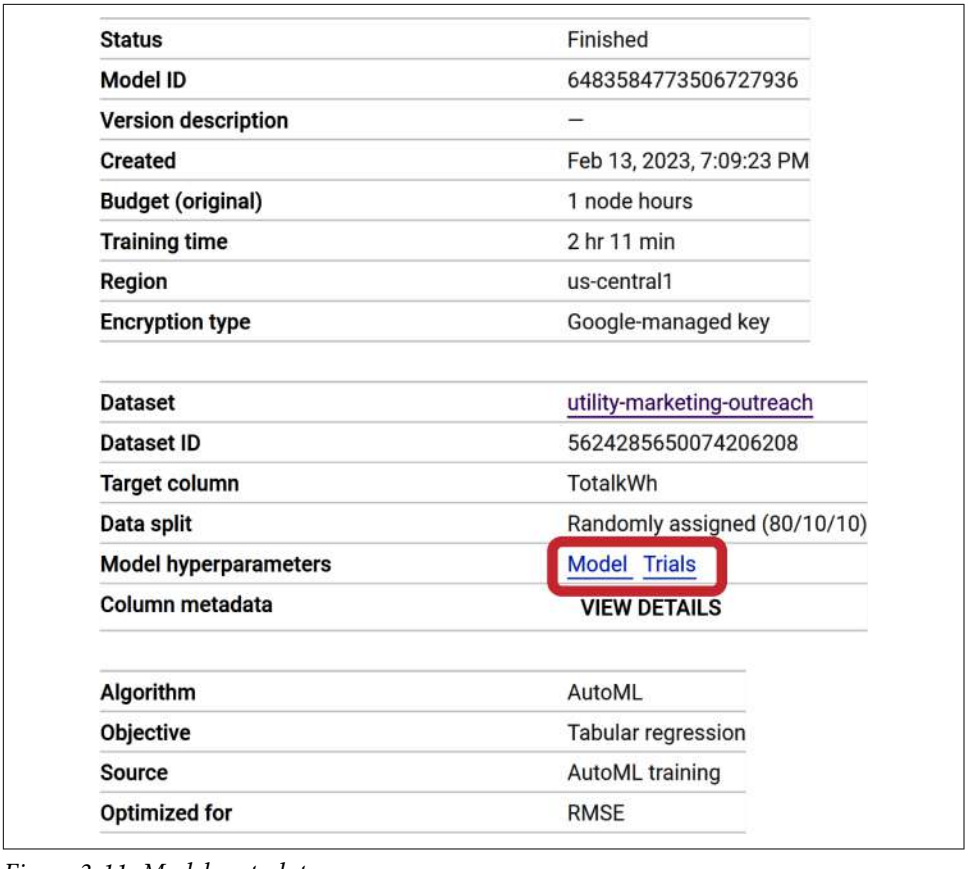


Figure 3-10. Attribution values are expressed as a percentage.

Figure 3-11 shows model metadata. You see various information about the model, from its ID, date created, and training time, to the dataset used, target column, data splitting percentage allocation, and the model evaluation metric used—in this case, root mean squared error (RMSE). Clicking on "Model" or "Trials" lets you retrieve info on which model has been used by AutoML.



Status	Finished
Model ID	6483584773506727936
Version description	—
Created	Feb 13, 2023, 7:09:23 PM
Budget (original)	1 node hours
Training time	2 hr 11 min
Region	us-central1
Encryption type	Google-managed key

Dataset	utility-marketing-outreach
Dataset ID	5624285650074206208
Target column	TotalkWh
Data split	Randomly assigned (80/10/10)
Model hyperparameters	Model Trials
Column metadata	VIEW DETAILS

Algorithm	AutoML
Objective	Tabular regression
Source	AutoML training
Optimized for	RMSE

Figure 3-11. Model metadata.

How AutoML Works

Implementing an ML workflow is time-consuming. As you saw from the preceding marketing outreach use case, AutoML simplified the process of building an ML model for you—you did not have to write a single line of code—for any tasks. Figure 3-12 shows the workflow for the utility marketing outreach project. Phases 2, 3, and 4 required no coding.

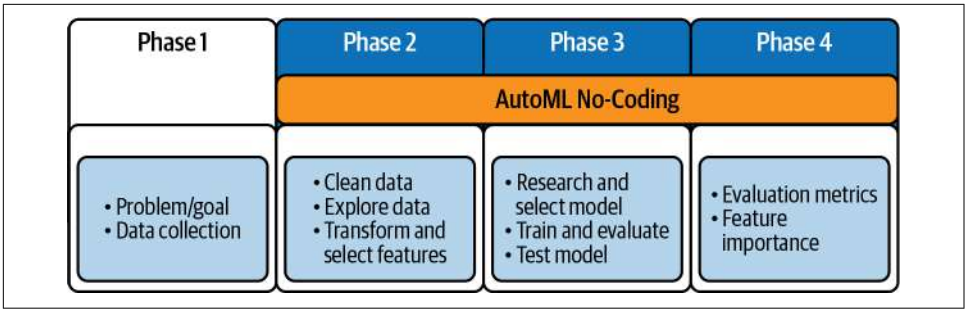


Figure 3-12. Workflow for the utility marketing outreach project.

To better understand how AutoML works, note what you did *not* have to do!

First, you did not have to set up an environment to run a Python Jupyter Notebook. There was no need to install any software libraries. In fact, there was nothing to install.

Once you uploaded the CSV data file into Vertex AI, the file was stored in the cloud. If you were using Python and Pandas, there was no need to write any code to load a dataset or even to split the dataset into training, validation, and testing files.

While the data was clean, there were two categorical features: zip code and customer class. If you had coded for those two features, you would have had to "one-hot encode" each one. *One-hot encoding* is the process of converting categorical data variables into numerical values. The following shows sample code for one-hot encoding for the feature zip code:

```
from sklearn.preprocessing import OneHotEncoder
one_hot = OneHotEncoder()
encoded = one_hot.fit_transform(df[[`zipcode`]])
df[one_hot.categories[0]] = encoded.toarray()
```

One-hot encoding is an example of feature *transformation* or engineering. You also had the ability to easily select the target (output) and/or deselect features—or *drop* them. You did not have to write code that resembled what is shown in the following, where the column "id" is being dropped from a Pandas DataFrame:

```
import pandas as pd
df = pd.read_csv(`/path/file.tsv`, header=0, delimiter=`\t`)
print df.drop(`id`, 1)
```

More features in your dataset lead to more complex relationships that may be nonlinear. Neural networks are perfect for working with nonlinear relationships. You may not have had any idea about any of this, so let's break it down a bit further.

As stated earlier, this is a prediction problem because you wanted to know whether future total kWh can be predicted based upon average KWh, customer class, month,

year, and zip code. Going a little deeper into the theory, this type of use case can be considered complex due to the number of input features—it has multiple variables and is *multivariate*. These types of complex relationships are considered *nonlinear* because you cannot simply draw a "straight" line that will be the "best fit" between what the known total kWh are and the other multiple variables.

This dataset is an excellent candidate for a neural network. *Neural networks* are difficult to construct without prior ML knowledge. Although neural networks are a topic for a later chapter, let's look quickly at an image to determine what you did *not* have to think about. Figure 3-13 shows a typical neural network with an input layer, hidden layers, and an output layer.

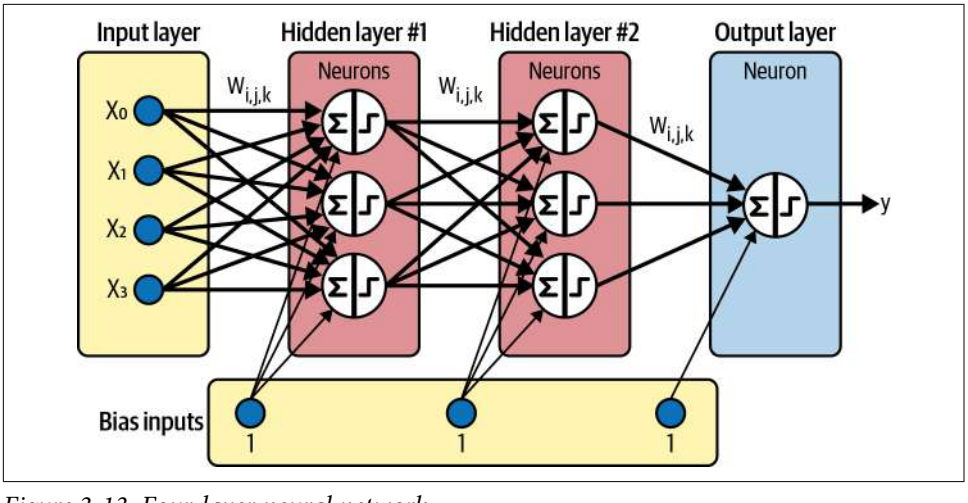


Figure 3-13. Four-layer neural network.

Coding a neural network in Python would look something like this:

```
# Create the 'Network' class and define its arguments:
# Set the number of neurons/nodes for each layer
# and initialize the weight matrices:
class Network:

    def __init__(self,
                 no_of_in_nodes,
                 no_of_out_nodes,
                 no_of_hidden_nodes,
                 learning_rate):
        self.no_of_in_nodes = no_of_in_nodes
        self.no_of_out_nodes = no_of_out_nodes
        self.no_of_hidden_nodes = no_of_hidden_nodes
        self.learning_rate = learning_rate
        self.create_weight_matrices()

    def create_weight_matrices(self):
```

```
        """ A method to initialize the weight matrices of the neural network"""
        rad = 1 / np.sqrt(self.no_of_in_nodes)
        X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
        self.weights_in_hidden = X.rvs((self.no_of_hidden_nodes,
                                        self.no_of_in_nodes))
        rad = 1 / np.sqrt(self.no_of_hidden_nodes)
        X = truncated_normal(mean=0, sd=1, low=-rad, upp=rad)
        self.weights_hidden_out = X.rvs((self.no_of_out_nodes,
                                         self.no_of_hidden_nodes))

    def train(self, input_vector, target_vector):
        pass # More work is needed to train the network

    def run(self, input_vector):
        """
        running the network with an input vector 'input_vector'.
        'input_vector' can be tuple, list or ndarray
        """
        # Turn the input vector into a column vector:
        input_vector = np.array(input_vector, ndmin=2).T

        # activation_function() implements the expit function,
        # which is an implementation of the sigmoid function:
        input_hidden = activation_function(
            self.weights_in_hidden @ input_vector)
        output_vector = activation_function(
            self.weights_hidden_out @ input_hidden)
        return output_vector
```

Using Keras, the coding becomes somewhat easier. Coding a neural network would look something like this:

```
# Import python libraries required in this example:
from keras.layers import Dense, Activation
from keras.models import Sequential
import numpy as np

# Use numpy arrays to store inputs (x) and outputs (y):
x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Define the network model and its arguments.
# Set the number of neurons/nodes for each layer:
model = Sequential()
model.add(Dense(2, input_shape=(2,)))
model.add(Activation('relu'))
model.add(Dense(1))
model.add(Activation('relu'))

# Compile the model and calculate its accuracy:
model.compile(
    loss='mean_squared_error', optimizer='rmse', metrics=['accuracy']
)
```

```
# Print a summary of the Keras model:
model.summary()
```

When you built your training job, you simply had to select a dataset and then select a few training parameters. There was no need for any of the following:

No need to understand what kind of regression algorithm to use
: There are many types of regression analysis techniques, and the use of each method depends upon a number of factors. These factors include the type of target variable, shape of the regression line, and the number of independent variables.

No need to understand "classical machine learning" versus neural networks
: There was no need to comprehend the commonly used neural-network building blocks such as layers, neurons (nodes), objectives, activation functions, or optimizers (see the glossary on GitHub).

No need to understand the training process or any model optimization strategies
: During training, AutoML focuses on optimizing not only the model weights but also the architecture. Selecting the appropriate architecture is done by AutoML.

No need to understand or specify compute resources
: When you chose "one node," AutoML selected the right machine type.



See the glossary on GitHub for definitions of the types of regression algorithms, such as linear regression, logistic regression, ridge regression, lasso regression, polynomial regression, and Bayesian linear regression.

Machine Learning as a Service

AutoML is part of the machine learning as a service (MLaaS) platform offered by cloud providers. The top three cloud providers are Google, Amazon, and Microsoft. If you are not familiar with cloud architecture and services, Figure 3-14 shows the typical cloud "platform pyramid."

Occupying the bottom of the pyramid is IaaS (infrastructure as a service). Think of this layer as the hardware and storage layer, with customers using the cloud provider's servers to handle the actual computation and storage services for storing dataset files, models, containers, and so on. The middle layer is PaaS (platform as a service). Think of this layer as providing the platform (operating systems such as Linux or Windows) that customers use to run their own software. The top layer is SaaS (software as a service). The best example of this is AutoML—you don't have to configure a server or write code, you just open a browser and use it.

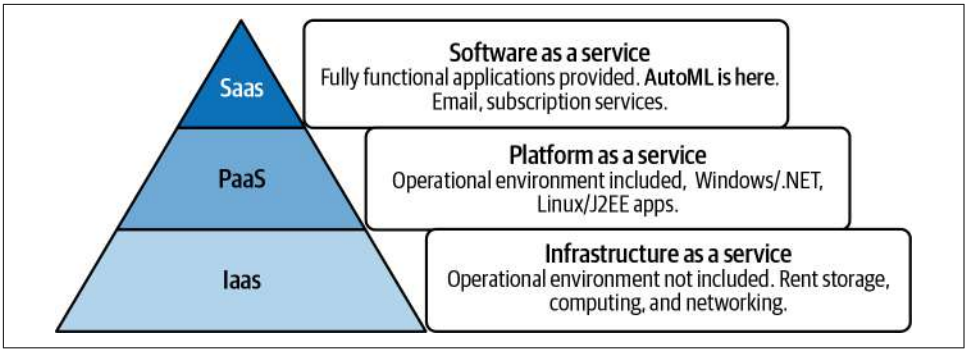


Figure 3-14. Typical cloud "platform pyramid."

Google, Amazon, and Microsoft offer services to support the entire ML workflow, including ML learning algorithm training and tuning, data visualization, data preprocessing, and deep learning. They also provide managed Jupyter Notebooks for using frameworks such as scikit-learn, Keras, TensorFlow, and PyTorch. Table 3-1 shows the benefits of MLaaS.

Table 3-1. MLaaS benefits

Data extraction and exploration	• Extract and clean dataset using Jupyter Notebook or, in some cases, the cloud provider's own data visualization tools. • Apply data transformations and feature engineering to the model, and split the data into training, validation, and test sets. • Explore and visualize data using the GUI tool or Jupyter Notebook.
Model training	• Use AutoML for tabular, image, text, and video data. Automatic featurization and algorithm selection. • Shared notebooks, compute resources, data, and environments. • Custom training using open source platforms, such as PyTorch, TensorFlow, or scikit-learn. • Optimize hyperparameters. • Experiment by running different model versions using different techniques and compare results.
Distributed training	Offer multinode distributed training.
Model evaluation and iteration	Provide evaluation metrics so that you can make adjustments to your data and iterate on your model.
Model explainability	Understand how each feature contributes to model prediction (feature attribution).
Model serving	Deploy your model to production and get predictions.
Model monitoring	Monitors the performance of your deployed model for training-serving skew and prediction drift and sends you alerts when the incoming prediction data skews too far from the training baseline .

AutoML is a valuable tool for businesses and organizations that want to use ML to improve their operations. By automating many of the time-consuming and complex tasks involved in building ML models, AutoML can help businesses and organizations get their models up and running more quickly. Here are some specific examples of how AutoML is being used in businesses today:

Telecom
: AutoML is being used by telecom companies to improve customer churn prediction, fraud detection, and network optimization.

Manufacturing
: AutoML is being used by manufacturing companies to improve product quality, optimize production processes, and predict equipment failure.

Retail
: AutoML is being used by retailers to personalize customer experiences, recommend products, and optimize inventory levels.

Healthcare
: AutoML is being used by healthcare companies to diagnose diseases, predict patient outcomes, and personalize treatment plans.

These are just a few examples of how AutoML is being used in businesses today. As AutoML technology continues to mature, expect to see even more innovative applications of AutoML in the future. Table 3-2 shows the benefits of AutoML.

Table 3-2. AutoML benefits

Improved accuracy	AutoML can help businesses and organizations build more accurate ML models. This is because AutoML can experiment with a wider range of algorithms and hyperparameters than a human data scientist could.
Democratized ML	AutoML makes ML more accessible to nonexperts. This is because AutoML provides a simple, user-friendly interface for building and deploying ML models.
Reduced time to market	AutoML can help businesses and organizations get their ML models up and running more quickly. This is because AutoML automates many of the time-consuming tasks involved in building ML models, such as data cleaning, feature engineering, and model training.
Reduced risk	AutoML can help businesses and organizations reduce the risk of building ML models that are biased or inaccurate. This is because AutoML automates many of the steps involved in model validation and testing.

Figure 3-15 shows Google's Vertex AI solution, Figure 3-16 shows Microsoft's Azure ML Studio solution, and Figure 3-17 shows the AutoML for Amazon SageMaker solution.

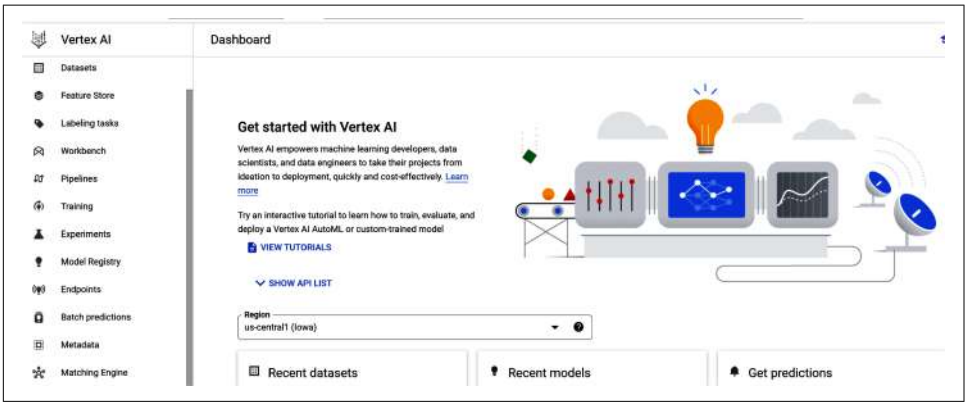


Figure 3-15. Google's Vertex AI interface.

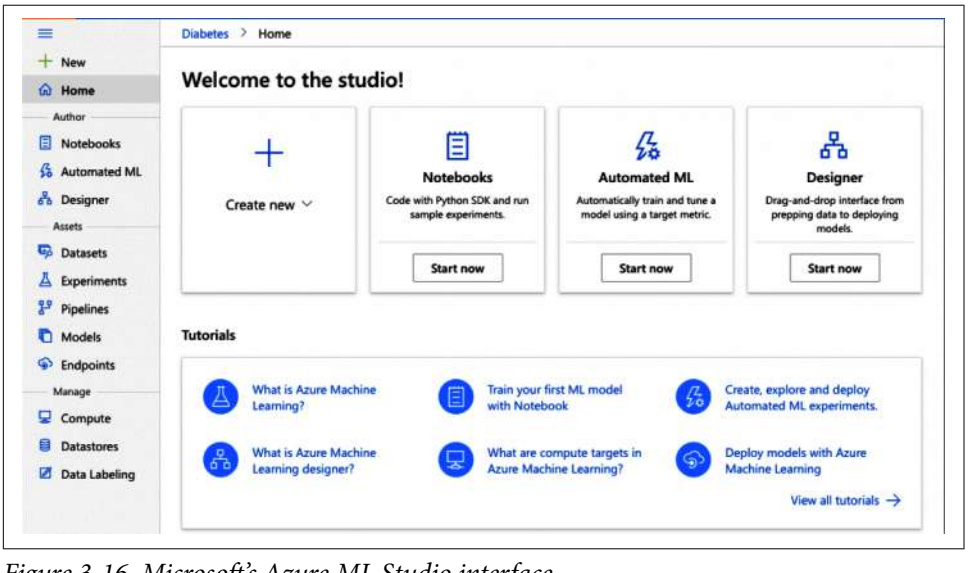


Figure 3-16. Microsoft's Azure ML Studio interface.

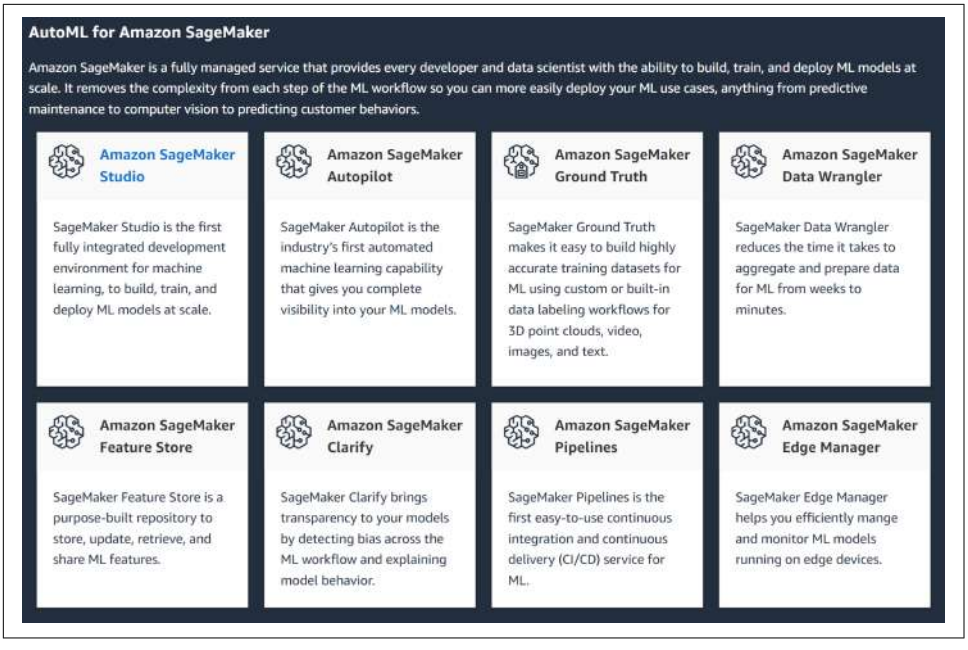


Figure 3-17. AutoML for Amazon SageMaker interface.

Low-Code ML Frameworks

Low-code AutoML requires installation and configuration of libraries, plus some knowledge of Python or some knowledge of Structured Query Language (SQL). *Low-code* is defined here as the following:

- ML frameworks that provide an "abstraction layer" on top of an existing ML framework.
- Databases that allow you to run ML models using SQL or databases that allow you to run Python code that includes ML code.

Table 3-3 shows some examples.

Table 3-3. ML framework examples

Cloud providers (SQL)		Open source
Google	BigQuery	AutoKeras
Amazon	Aurora, Redshift	Auto-sklearn
Microsoft	Azure SQL Server	Auto-PyTorch

SQL ML Frameworks

Data analysts and data scientists typically use SQL for data analysis. They can leverage their existing SQL skills and expertise and apply them to ML—without any ML programming backgrounds. If they know SQL but can't code in Python, Java, or R, they can work within a SQL-ML framework for their ML projects. This is why SQL-ML frameworks are considered low-code. There is not a lot of SQL coding required.

Benefits of using databases/data warehouses for SQL-ML:

Model building with large datasets
: When you can build the ML model where the data "lives," then the ML SQL code stays "close to the data" and reduces latency (data transmission times). This is especially crucial for large datasets using deep learning, where training requires iteration through a percentage of the data for training, validation, and testing.

Backend integration with existing ML systems
: Integrate with the cloud provider's backend ML framework (for example, Google's Vertex AI, Amazon's SageMaker, and Microsoft's Azure).

Common model build statements
: All use the `CREATE MODEL` SQL command and specify training data either as a table or `SELECT` statement. They then compile and import the trained model inside the data warehouse and prepare a SQL inference function that can be immediately used in SQL queries.

Use cases
: Typical use cases include fraud detection, product recommendations, and ad targeting due to low-latency, real-time requirements.



A *database* is a collection of data or information. The term *database* is commonly used to reference both the database itself as well as the database management system (DBMS). *Data warehouses* store large amounts of current and historical data from various sources.

Google's BigQuery ML

Google's BigQuery is a data warehouse. It can provide decision-making guidance through predictive analytics by using its ML tool. You can create and train a model without ever exporting data out of BigQuery. Like Vertex AI, BigQuery ML does not require an environment and dependency setup. BigQuery ML is browser-based and serverless, meaning you don't need a server to run it. If you have data already residing in BigQuery's data warehouse, then you can use that data for your ML projects.

Amazon Aurora ML and Redshift ML

Amazon Aurora is a relational database management system (RDBMS) built for the cloud with full MySQL and PostgreSQL compatibility. Amazon Aurora ML enables you to add ML-based predictions to applications using SQL. When you run an ML query, Aurora calls Amazon SageMaker for a wide variety of ML algorithms.

Redshift ML is a data warehouse. You use SQL statements to create and train Amazon SageMaker ML models using Redshift data and then use these models to make predictions. Redshift ML makes the model available as a SQL function within the Redshift data warehouse.

Open Source ML Libraries

Open source AutoML refers to open source frameworks such as AutoKeras, Auto-sklearn, and Auto-PyTorch that add an additional layer of abstraction on top of existing open source libraries. Typically, you will need to code for the following using a Jupyter notebook:

1. Install the AutoML package.
2. Import the package.
3. Load a dataset.
4. Split the data.
5. Fit the model.
6. Predict.
7. Evaluate.
8. Export the model.

After Step 4, each open source framework has its own way to perform model fit, prediction, and evaluation. Figure 3-18 shows the first four steps.

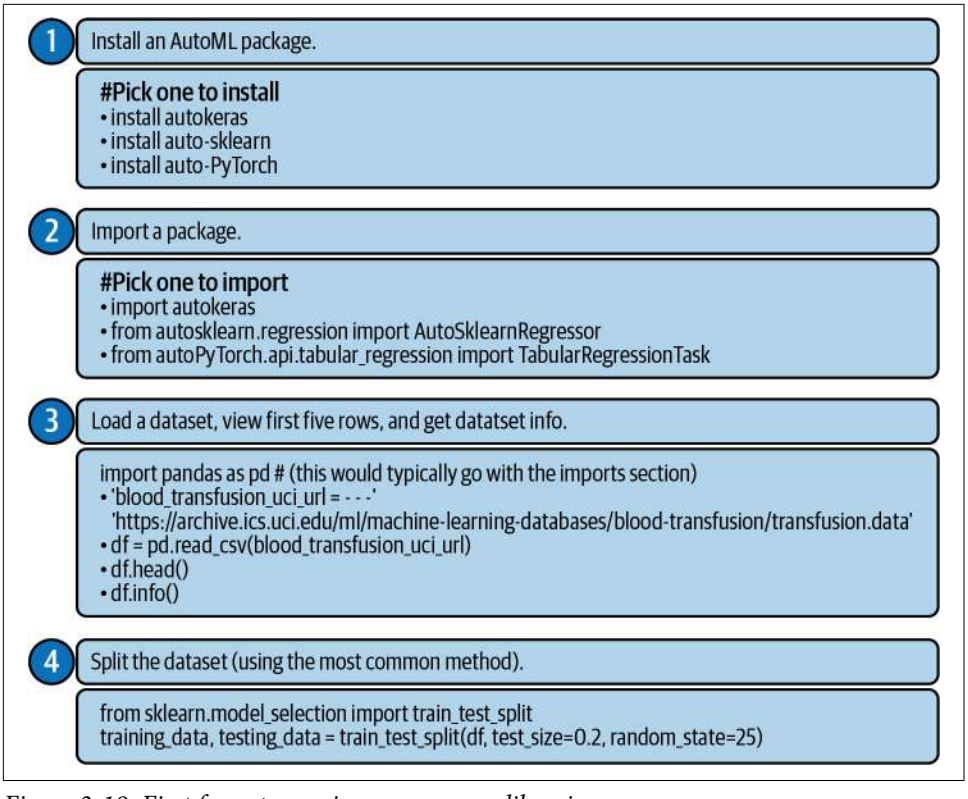


Figure 3-18. First four steps using open source libraries.

AutoKeras

AutoKeras, a Keras-based open source AutoML framework, was developed to allow nonexperts to quickly build neural networks with minimal code. When using AutoKeras, you only need to specify the training data, and AutoKeras performs the data preprocessing independently. For example, if the data has categorical variables, it will convert them into one-hot encoding depending on whether it is a classification or regression task; if the input data contains text, AutoKeras transforms it into an embedding.

Auto-Sklearn

Auto-sklearn is an open source Python package based on the scikit-learn ML library. Auto-sklearn automatically searches for the right learning algorithm for a new ML dataset and optimizes its hyperparameters. This framework only supports sklearn-based models. Auto-sklearn was developed by labs at the University of Freiburg and the University of Hannover.

Auto-PyTorch

In addition to Auto-sklearn, the Freiburg-Hannover AutoML group also developed an AutoML based on PyTorch that focuses on deep learning. Auto-PyTorch is deemed excellent at rapid prototyping. It also supports distributed training.

Summary

Business analysts, data analysts, citizen data scientists, data scientists, software developers, and ML engineers can all use AutoML frameworks to streamline the development process.

First, you load a dataset that includes a target variable and input feature data used for predictions. After the data is loaded, a data profile is generated for each data column. To submit a training job, you select just a few parameters. AutoML then experiments with multiple models and performs model optimization. Results are presented, as well as feature attribution.

Cloud vendors provide MLaaS to accelerate and automate day-to-day ML workflows, tools for integrating models into applications or services, and tools for deploying models into production.

Low-code AutoML requires installation, configuration of libraries, and some knowledge of SQL or Python. Open source AutoML refers to open source frameworks such as AutoKeras, Auto-sklearn, and Auto-PyTorch that add an additional layer of abstraction on top of existing open source libraries.

In Chapter 4, you build an AutoML model to predict advertising media channel sales. First, you explore your data using Pandas. Then you learn how to use AutoML to build, train, and deploy an ML model to predict sales.

CHAPTER 4

Use AutoML to Predict Advertising Media Channel Sales

In this chapter, you build an AutoML model to predict advertising media channel sales. First, you explore your data using Pandas. Then you learn how to use AutoML to build, train, and deploy an ML model to predict sales. You gain an overall understanding of the performance of your model using performance metrics and answer common business questions. Along the way, you'll learn about regression analysis, a common technique used for prediction use cases.

The Business Use Case: Media Channel Sales Prediction

Businesses use advertising media channels to promote their products, services, or brand. Marketers and media planners create marketing campaigns that may run on digital, TV, radio, or in the newspaper. In this scenario, you work as a media planner in the marketing department for a midsize solar energy company. Your firm has a modest media budget and needs to evaluate which channels offer the greatest number of benefits for the least cost. This is a spend optimization problem.

You have been asked to develop a marketing plan that will increase next year's product sales. To accomplish this goal, you need to understand the impact of the media channel product advertising budgets on overall sales. The advertising dataset captures the sales revenue generated with respect to advertisement costs across digital, TV, radio, and newspaper media channels.

Typically, this type of ask from the team lead would go to a data scientist or data analyst. But, although you do not have any coding experience, the marketing team lead has asked you to build a sales predictive model using AutoML, something they want to try for the first time on the team. The goal is to build an ML model to predict

how much sales volume will be generated based on the money spent in each of the media channels.

Business questions include:

- Can the model predict how much sales volume will be generated based on the money spent in each media channel?
- Is there a relationship between advertising budget and sales?
- Which media channel contributes the most to sales?
- Can the model be used to forecast future sales based on the media channel's proposed budget?
- How accurately can the model predict future sales?

The use case is a simple regression problem with just five variables that you can use to answer the preceding five questions.

Project Workflow

Figure 4-1 shows the high-level overview of the typical AutoML no-code workflow from Chapter 3. This workflow is appropriate for your use case.

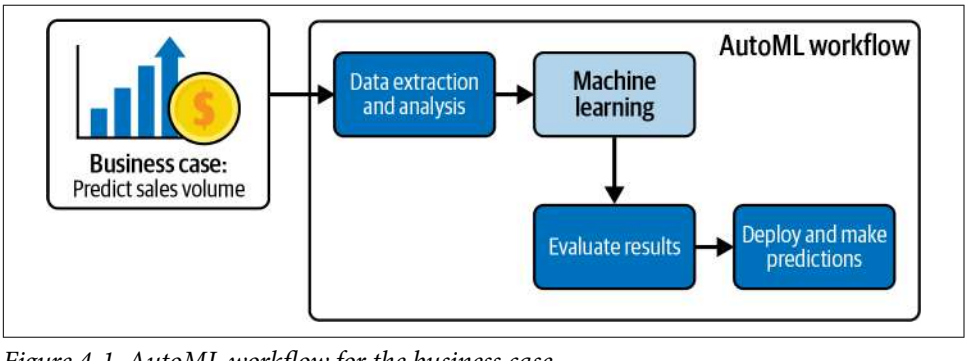


Figure 4-1. AutoML workflow for the business case.

Now that you understand the business use case and objective, you can proceed to data extraction and analysis. Note that this workflow does not include a data preprocessing step. You will get lots of hands-on experience with data preprocessing in later chapters. After data extraction and analysis, you will upload the dataset into the AutoML platform. The advertising budgets of digital, TV, newspaper, and radio data are then fed into the model. You'll then evaluate the AutoML results and then deploy the model to make predictions. After this chapter's hands-on exercise, you will be able to create a strategic marketing plan for your team.

Project Dataset

The dataset is composed of historical marketing channel data that can be leveraged to gain insights for spend allocation and to predict sales. The dataset being used for this chapter, the advertising_2023 dataset, is based on data taken from *An Introduction to Statistical Learning with Applications in R* by Daniela Witten, Gareth M. James, Trevor Hastie, and Robert Tibshirani (Springer, 2021). The advertising dataset captures the sales revenue generated from advertising (in thousands of units) for particular product advertising budgets (in thousands of dollars) for TV, radio, and newspaper media.

For this book, the dataset has been updated to include a digital variable and modified to show the impact of digital budgets on sales. The number of markets has been increased from 200 to 1,200. Thus, the data consists of the advertising budgets for four media channels (digital, TV, radio, and newspapers) and the overall sales in 1,200 different markets. You should feel encouraged to look at other examples of how to work with this dataset to grow your knowledge after completing the exercises in this chapter.

The data is initially supplied in a CSV file, so you will need to spend some time loading the data into Pandas before you can explore it. The dataset contains only numeric variables.

There are five columns in the dataset. Table 4-1 gives the column names, data types, and some information about the possible values for these columns.

Table 4-1. Schema and field value information for the advertising dataset

Column name	Column type	Notes about field values
Digital	Numeric	Amount spent on advertisements in digital
Newspaper	Numeric	Amount spent on advertisements in newspapers
Radio	Numeric	Amount spent on advertisements in radio
TV	Numeric	Amount spent on advertisements in TV
Sales	Numeric	Total media channel sales

Exploring the Dataset Using Pandas, Matplotlib, and Seaborn

Before you begin using AutoML, you follow the workflow discussed in earlier chapters around understanding and preparing data for ML. This section shows you how to load data into a Google Colab notebook using Pandas, an open source Python package that is widely used for data science and data analysis. Once the data is loaded into a DataFrame, you will explore the data. Fortunately, the data has already been cleaned—there are no missing values or strange characters in the dataset. Your

exploratory data analysis is to assist you in validating that the data is clean and to explore relationships between the variables to assist you in answering questions posed from the team. As noted in previous chapters, much of the ML work goes into understanding and preparing the training data—not training the model—because you are relying on AutoML to build the model.

All of the code in this section, including some additional examples, is included in a Jupyter notebook titled Chapter4_Media_Channel_Sales_Notebook in the low-code-ai repository on GitHub.

Load Data into a Pandas DataFrame in a Google Colab Notebook

First, go to *https://colab.research.google.com* and open a new notebook, following the process discussed in Chapter 2. You may rename this notebook to a more meaningful name by clicking on the name as shown in Figure 4-2 and replacing the current name with a new name, say *Advertising_Model.ipynb*.

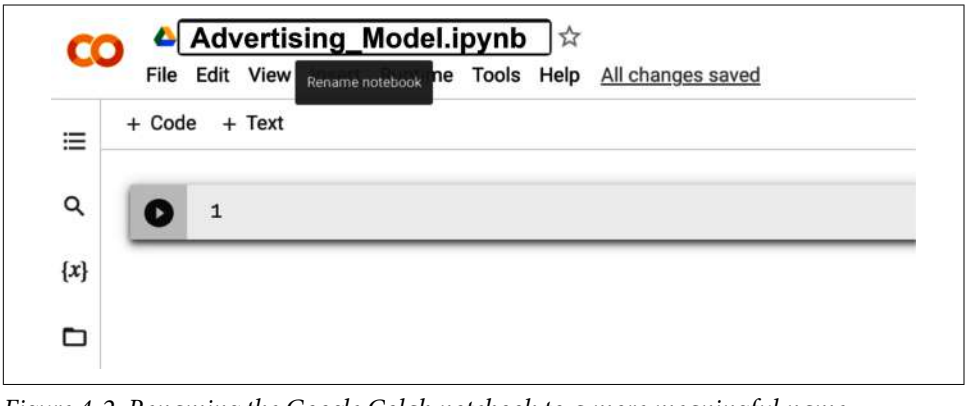


Figure 4-2. Renaming the Google Colab notebook to a more meaningful name.

Now type the following code into the first code block to import the packages needed to analyze and visualize the advertising dataset:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns
%matplotlib inline
```

You saw some of these packages before in Chapter 2 when first exploring the use of Colab notebooks.

Now execute the cell containing the import statements to import the packages. To do this, you click the Run Cell button on the left side of the cell. You can also press Shift + Enter to run the cell.

Now you are ready to import your data. Using Pandas, you can directly import a CSV file into a DataFrame from a location on the internet without having to download the file first. To do this, copy the code from the solution notebook or type in the following code into a new cell and execute the cell:

```
url="https://github.com/maabel0712/low-code-ai/blob/main/advertising_2023.csv?raw=true"

advertising_df=pd.read_csv(url, index_col=0)
```

In general, it is a good idea to look at the first few rows of the DataFrame. Use `advertising_df.head()` to explore the first few rows of the DataFrame. The head Pandas method lets us see the first five rows of our data. By doing this, you can quickly see the features, some of their possible values, and whether they are numerical or not.

An example of a few of the columns is shown in Figure 4-3.

Out[4]:

	digital	TV	radio	newspaper	sales
1	345.15	156.0	37.8	69.2	22.1
2	66.75	46.0	39.3	45.1	10.4
3	25.80	18.3	45.9	69.3	9.3
4	227.25	145.1	41.3	58.5	18.5
5	271.20	165.2	10.8	58.4	12.9

Figure 4-3. A few columns of the first five rows of the DataFrame `advertising_df` printed using the `head()` method.

Explore the Advertising Dataset

Now that the data has been loaded into the DataFrame `advertising_df`, you can begin to explore and understand it. The immediate goal is to get an idea of where there could be issues with the data so that you may resolve those issues before moving forward.

Descriptive analysis: Check the data

First check the data using standard Python methods. To check the data types for your DataFrame, type **`advertising_df.info()`** into a new cell and execute the cell. The information contains the number of columns, column labels, column data types, memory usage, range index, and the number of cells in each column (non-null values).

Figure 4-4 shows an example of the `info()` method output.

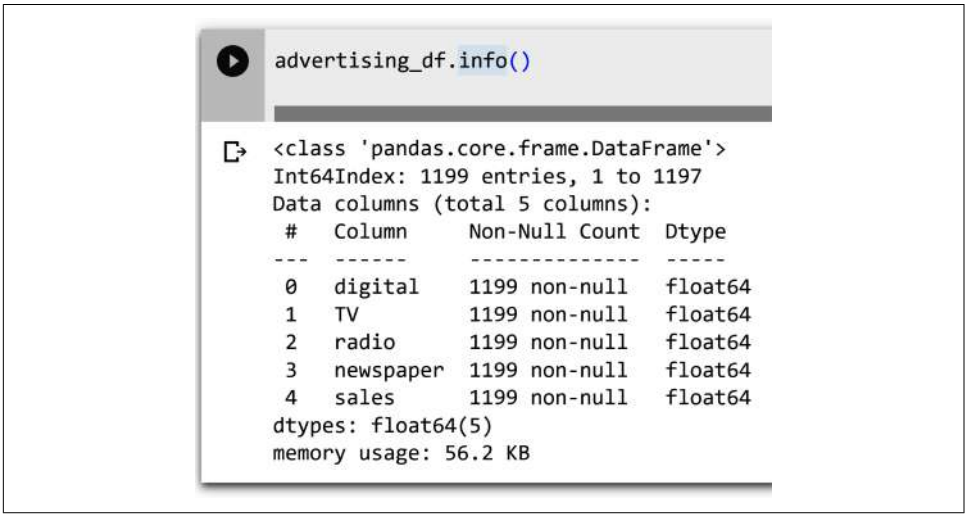


Figure 4-4. Information on the dataset using the `info()` method.

Note that with information on the data type, you can check to make sure that the types inferred by Pandas match up with what was expected from Table 4-1.

Shown in Figure 4-5 is the `describe()` method, which computes and displays summary statistics for the dataset. The `describe` function shows information about the numerical variables of our dataset. You can see the mean, maximum, and minimum values of each of these variables, along with their standard deviation. Type **`advertising_df.describe()`** into a new cell and execute the cell.

```
advertising_df.describe()
```

	digital	TV	radio	newspaper	sales
count	1199.000000	1199.00000	1199.000000	1199.000000	1199.000000
mean	135.472394	146.61985	23.240617	30.529942	14.005505
std	135.730821	85.61047	14.820827	21.712507	5.202804
min	0.300000	0.70000	0.000000	0.300000	1.600000
25%	24.250000	73.40000	9.950000	12.800000	10.300000
50%	64.650000	149.70000	22.500000	25.600000	12.900000
75%	256.950000	218.50000	36.500000	45.100000	17.400000
max	444.600000	296.40000	49.600000	114.000000	27.000000

Figure 4-5. Summary statistics for all columns in the advertising dataset.

Shown in Figure 4-6 is the output of the `.isnull()` method. Type **`advertising_df.isnull().sum()`** into a new cell and execute the cell. The output shows all of the columns in the DataFrame with associated zeros. If there were null values, the number of null values for the column would be shown.

```
advertising_df.isnull().sum()
```

```
digital      0
TV           0
radio        0
newspaper    0
sales        0
dtype: int64
```

```
advertising_df.isnull().values.any()
```

```
False
```

Figure 4-6. Determining null values using the `isnull()` method.

Explore the data

Exploratory data analysis (EDA) is the first step of any ML project. You need to explore your data before building any ML model. The goal is to take a look at the raw data, explore it, and gather relevant insights from the information derived from the data. Doing this also helps make models better, as you'll be able to spot any "dirty data" issues—such as missing values, strange characters in a column, etc.—that may impact performance.

Heat maps (correlations). A *heat map* is a way of representing the data visually. The data values are represented as colors in the graph. The goal of the heat map is to provide a colored visual summary of information. Heat maps show your relationships (correlations) between variables (features). *Correlation* is a statistical measure that shows the extent to which two or more variables move together. Shown in Figure 4-7 is the output of a correlation method that plots correlation values on the grid. Type the following code into a new cell and execute the cell:

```
plt.figure(figsize=(10,5))
sns.heatmap(advertising_df.corr(),annot=True,vmin=0,vmax=1,cmap='viridis')
```

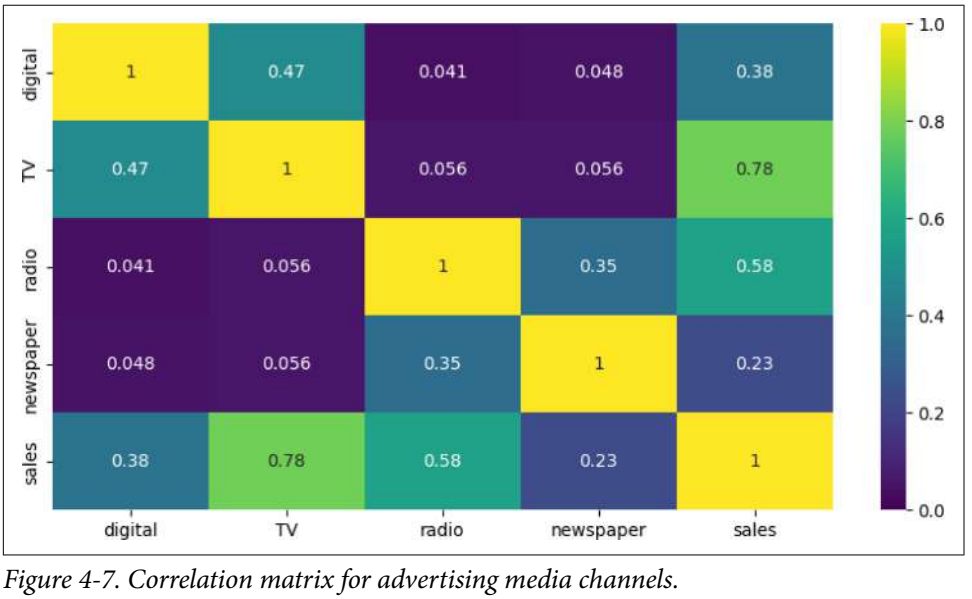


Figure 4-7. Correlation matrix for advertising media channels.

The results from a correlation matrix can be used in a variety of ways, including:

Identifying relationships between variables
: The correlation coefficient between two variables can tell you how strongly they are related. A correlation coefficient of 0 means there is no relationship, whereas a correlation coefficient of 1 means there is a perfect positive relationship. A correlation coefficient of –1 means there is a perfect negative relationship. *Note that the stronger relationships are between sales and television (0.78), followed by sales and radio (0.58)*. This information can be used to develop targeted marketing campaigns that are more likely to improve sales.

Selecting variables for inclusion in a model
: When you are building a predictive model, you need to select the variables that are most likely to be predictive of the outcome. For example, should you include newspapers (with a 0.23 correlation to sales) as a feature for inclusion in the model to predict sales?

Detecting multicollinearity
: Multicollinearity occurs when two or more predictor variables in a regression model are highly correlated. For example, if both TV and radio were highly correlated (meaning that both had a value >0.7 instead of 0.056 as shown in Figure 4-7), it would indicate multicollinearity. Since it is harder to numerically distinguish predictors with a strong collinear relationship from one another, it is more difficult for a regression algorithm to determine the degree of influence or weight one of them should have on sales.



As a warning, the results of a correlation matrix may change due to sample size or outliers in the dataset.

Scatterplots. Scatterplots are used to determine relationships between two numerical variables. They can help you see if there is a direct relationship (positive linear relationship or negative linear relationship, for example) between two variables. Also, they can help you detect if your data has outliers or not. Figure 4-8 shows a scatter of the digital feature and the sales target. Type the following code into a new cell and execute the cell:

```
advertising_df.plot(kind='scatter', x=['digital'], y='sales')
```

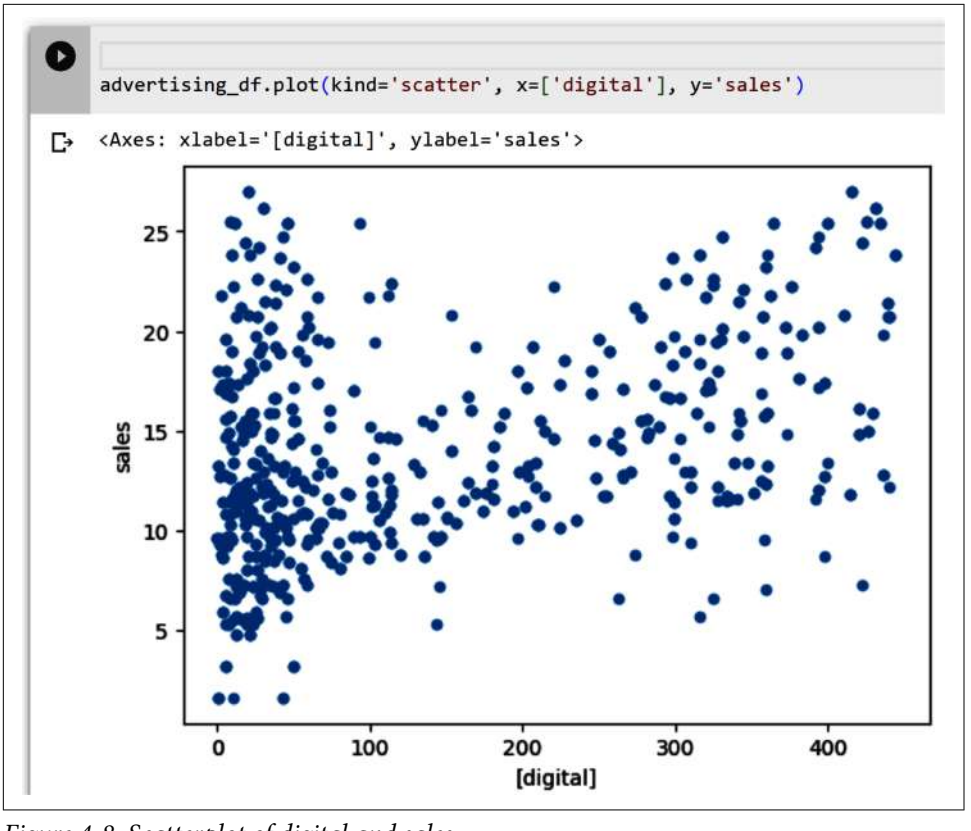


Figure 4-8. Scatterplot of digital and sales.

You want to explore the scatterplots for each variable with the predicted variable `sales`. You can plot each feature as a scatterplot separately (as you did earlier), or you can plot them so all of the relationships are shown in one plot. Type all of the following code into a new cell and execute the cell (Figure 4-9 shows the output):

```
plt.figure(figsize=(18, 18))

for i, col in enumerate(advertising_df.columns[0:13]):
    plt.subplot(5, 3, i+1) # each row three figure
    x = advertising_df[col] #x-axis
    y = advertising_df['sales'] #y-axis
    plt.plot(x, y, 'o')

    # Create regression line
    plt.plot(np.unique(x), np.poly1d(np.polyfit(x, y, 1)) (np.unique(x)),
        color='red')
    plt.xlabel(col) # x-label
    plt.ylabel('sales') # y-label
```

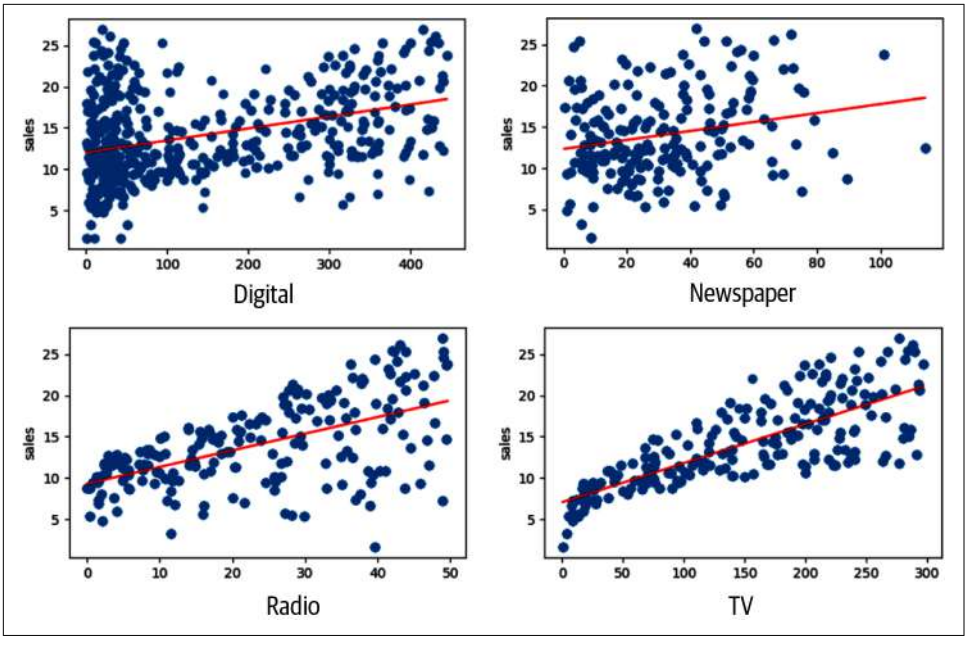


Figure 4-9. Scatterplots of all features and sales targets.

Note that TV and sales have a strong linear relationship—as it appears to show that for an increase in TV budget, there is a positive impact on sales volume. There does not appear to be a strong relationship between newspaper and sales. Recall the correlation values of 0.23 for this relationship. This is very different from the relationship between TV and sales (0.78).

Histogram distribution plot. A common approach to visualizing a distribution is the histogram. A *histogram* is a bar plot where the axis representing the target variable is divided into a set of discrete bins, and the count of observations falling within each bin is shown using the height of the corresponding bar.

Type this code into a new cell and run the cell:

```
sns.displot(advertising_df, x="sales")
```

Figure 4-10 shows the data values from the sales column. The plot looks somewhat like a bell curve that is slightly skewed to the left. The most common sales amount is $11,000 dollars.

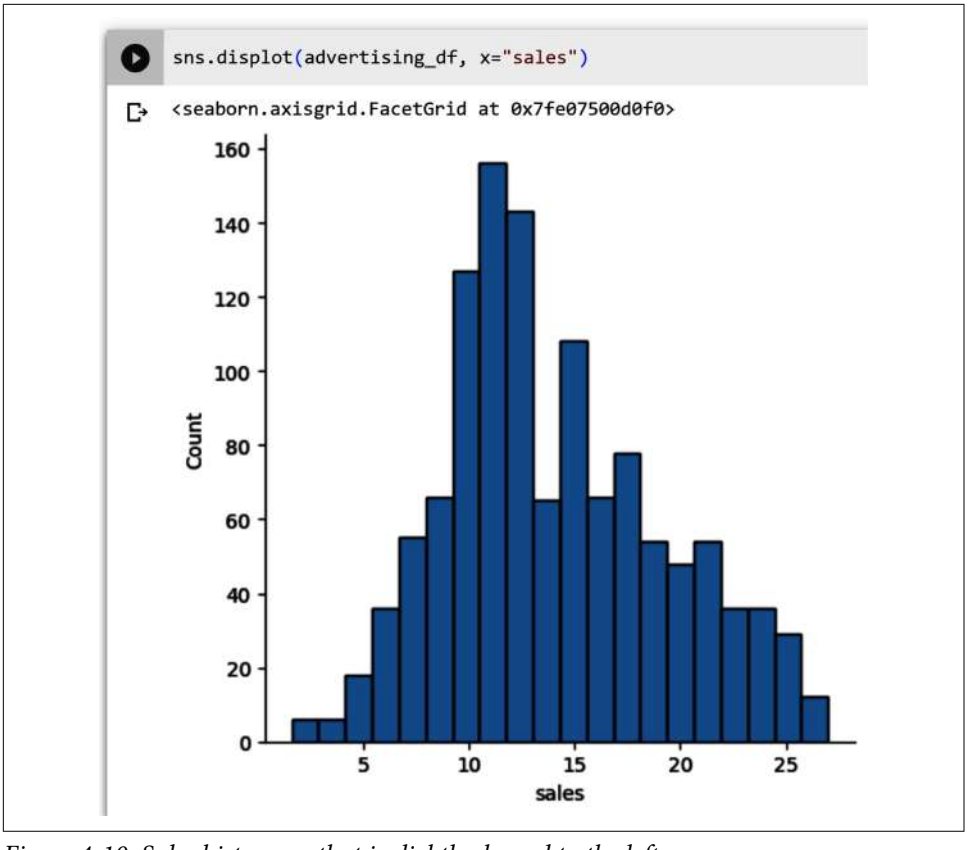


Figure 4-10. Sales histogram that is slightly skewed to the left.

What about the other features—are they skewed left or right or do they have a "normal distribution," like a bell curve? You can type the preceding code for each individual feature or use the following code to see all of the features together. Type the following code into a new cell and execute the cell (Figure 4-11 shows the output):

```
lis = ['digital', 'newspaper', 'radio','TV']
plt.subplots(figsize=(15, 8))
index = 1
for i in lis:
    plt.subplot(2, 2, index)
    sns.distplot(advertising_df[i])
    index += 1
```

As you saw in Figure 4-10, sales have somewhat of a normal distribution. However, in Figure 4-11, digital appears to be skewed left, and TV, radio, and newspaper are not normally distributed. Standardizing these features so they are normally distributed before feeding them into your ML model would generate better results.



However, your role is not that of a data scientist. Do not worry about understanding these concepts. Discussing each transformation required for the features is beyond the scope of this chapter. Chapter 7 is where you perform transformations on a dataset.

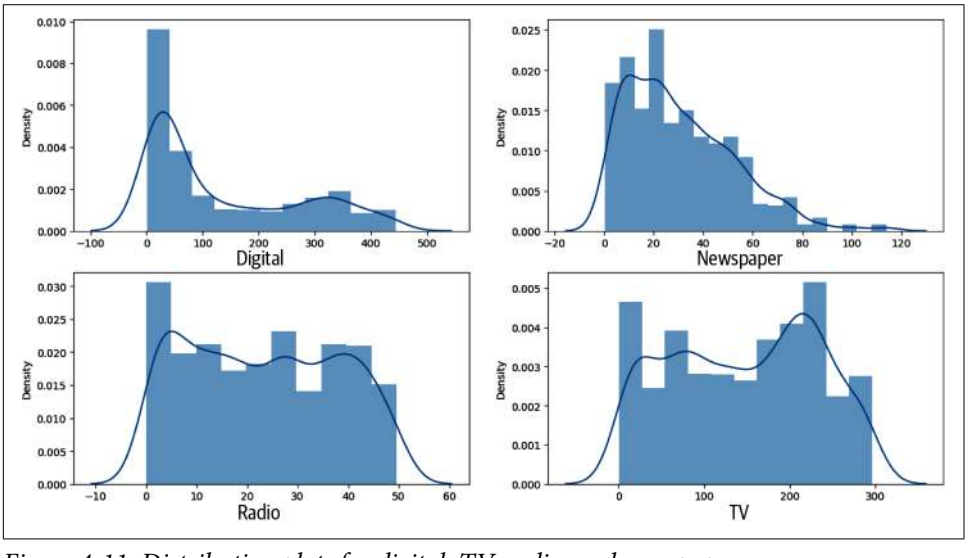


Figure 4-11. Distribution plots for digital, TV, radio, and newspaper.

Export the advertising dataset

After you have checked and explored the dataset, it is time to export the file so that it can be uploaded into your AutoML framework. Type the following code into a new cell and execute the cell. The first line imports the operating system that will allow you to make a directory called *data* (lines two and three):

```
import os
if not os.path.isdir("/content/data"):
    os.makedirs("/content/data")
```

After the directory has been created, type the following code into a new cell and execute the cell. The first line of code creates a CSV file format of the advertising DataFrame and places it in the *content/data* directory you made in the previous step:

```
advertising_df.to_csv('/content/data/advertising.csv',
    encoding='utf-8', index=False)
```

Figure 4-12 shows the newly created directory called *data* with the file *advertising.csv*.

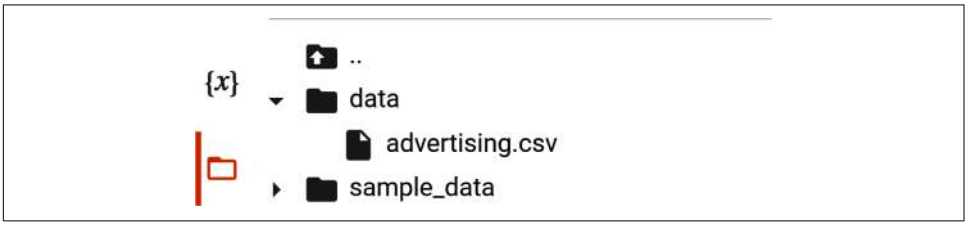


Figure 4-12. Newly created data directory with advertising.csv file.

As a best practice, validate that you can see the contents of the newly exported file in the newly created directory. Type **`!head /content/data/advertising.csv`** into a new cell and execute the cell. Check that the output shown in Figure 4-13 is the same as yours.

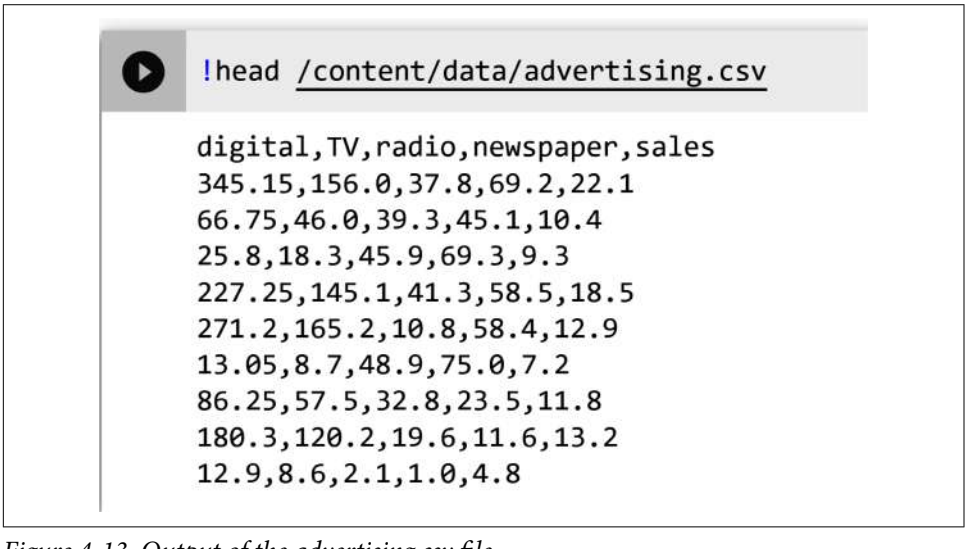


Figure 4-13. Output of the advertising.csv file.

Now that you have verified that the file has been properly exported, you can download it to your computer.

Right-click the *advertising.csv* file in the newly created *data* directory and select Download as shown in Figure 4-14. The file will download to your desktop. You are now ready to upload the file for AutoML use.

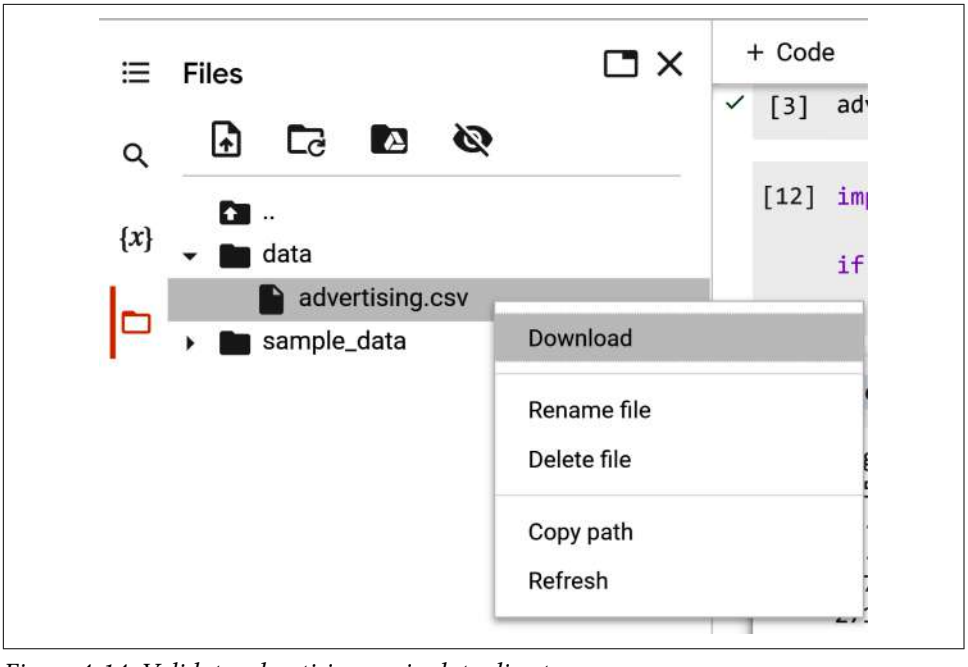


Figure 4-14. Validate advertising.csv in data directory.

In the next section, you build a code-free model based on the training data file you just exported.

Use AutoML to Train a Linear Regression Model

The AutoML projects for this book will be implemented using Google's Vertex AI, the GUI-based AutoML and custom training framework the authors are most familiar with. Note that the top three major cloud vendors (Google, Microsoft, and AWS) all offer AutoML tutorials. Each of these three major cloud vendor's guides can be found in their documentation. Many cloud vendors offer a trial period to explore their products without cost.

Given that Google offers a step-by-step tutorial on AutoML, some introductory steps are excluded.

Figure 4-15 shows a high-level overview of the AutoML no-code workflow for your business use case.

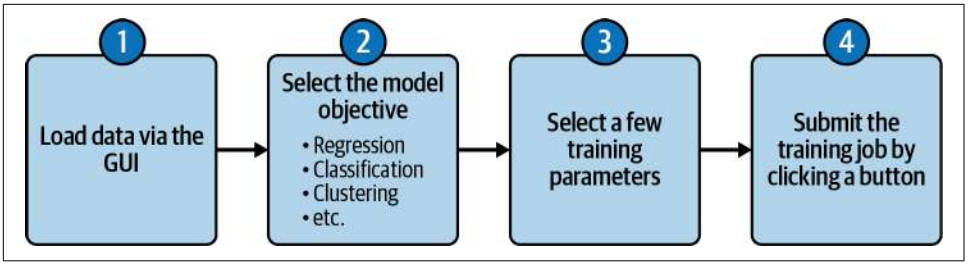


Figure 4-15. AutoML no-code workflow for your use case.

No-Code Using Vertex AI

Figure 4-16 shows the Vertex AI Dashboard. To create an AutoML model, turn on the Vertex AI API by clicking the Enable All Recommended APIs button. From the left-hand navigation menu, scroll down from Dashboard and select Datasets.

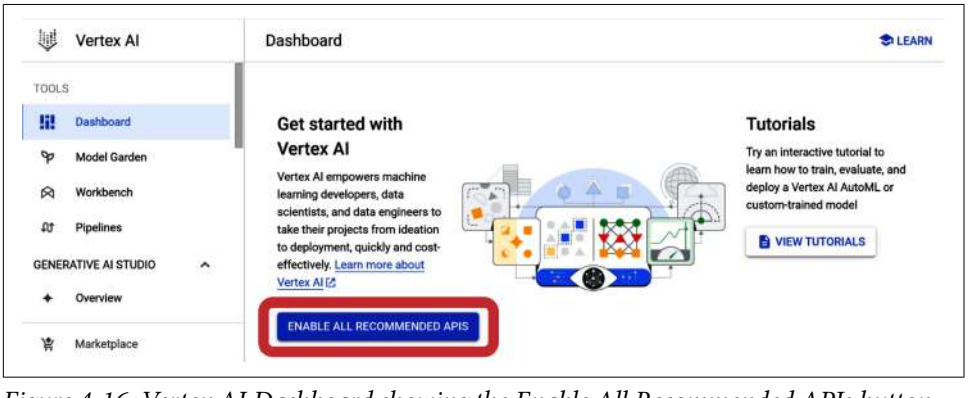


Figure 4-16. Vertex AI Dashboard showing the Enable All Recommended APIs button.

Create a Managed Dataset in Vertex AI

Vertex AI offers different AutoML models depending on data type and the objective you want to achieve with your model. When you create a dataset you pick an initial objective, but after a dataset is created you can use it to train models with different objectives. Keep the default region (us-central1), as shown in Figure 4-17.

Select the Create button at the top of the page and then enter a name for the dataset. For example, you can name the dataset *advertising_automl*.

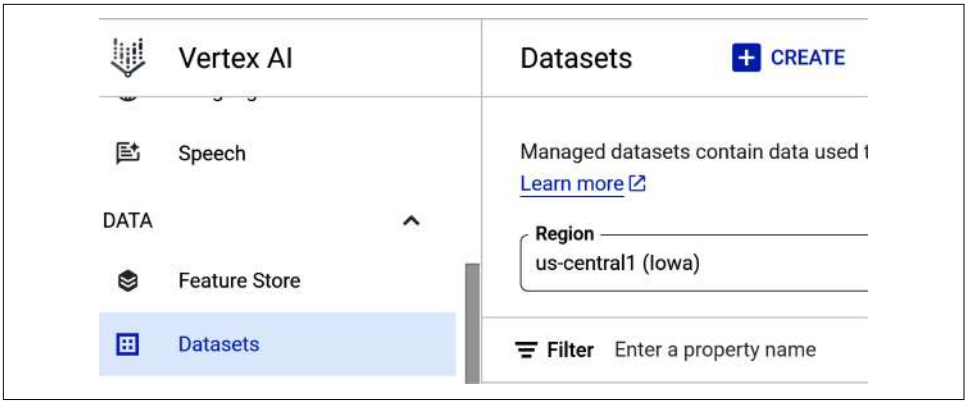


Figure 4-17. Vertex AI Dataset navigation that allows you to create a dataset.

Select the Model Objective

Figure 4-18 shows Regression/classification selected as the model objective under the Tabular tab. Given that you want to predict a target column's value (sales), this is the appropriate selection.

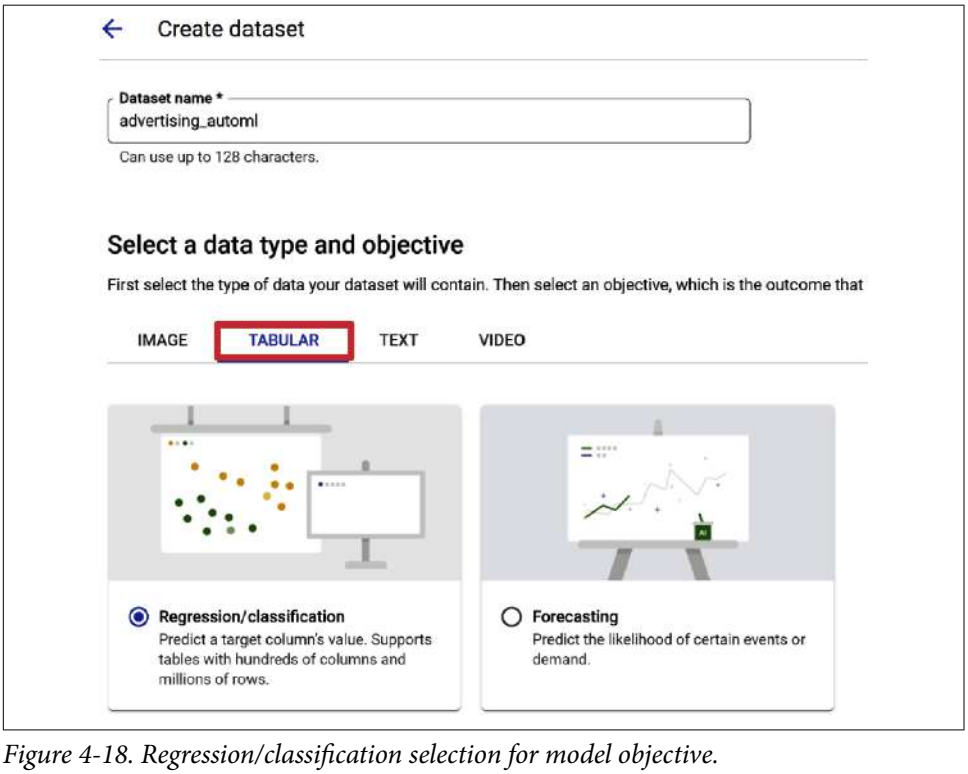


Figure 4-18. Regression/classification selection for model objective.

You selected Regression/classification as your objective. Let's cover some basic concepts to help you with future use cases. *Regression* is a supervised ML process. It is similar to classification, but rather than predicting a label for a classification, such as classifying spam from your email inbox, you try to predict a continuous value. *Linear* regression defines the relationship between a target variable (y) and a set of predictive features (x). If you need to predict a number, then use regression. In your use case, linear regression predicts a real value (sales) using some independent variables given in the dataset (digital, TV, radio, and newspaper).

Essentially, linear regression assumes a linear relationship with each feature. The predicted values are the data points on the line, and the true values are in the scatterplot. The goal is to find the best fitting line so that when new data is input the model can predict where the new data point will be in relation to the line. The "evaluation" of how good that fit is includes an evaluation criteria—which is covered in "Evaluate Model Performance" on page 98.

Figure 4-19 shows a "best-fit" line based on your dataset, where the model tries to fit the line to your data points, which are the dark scatters.

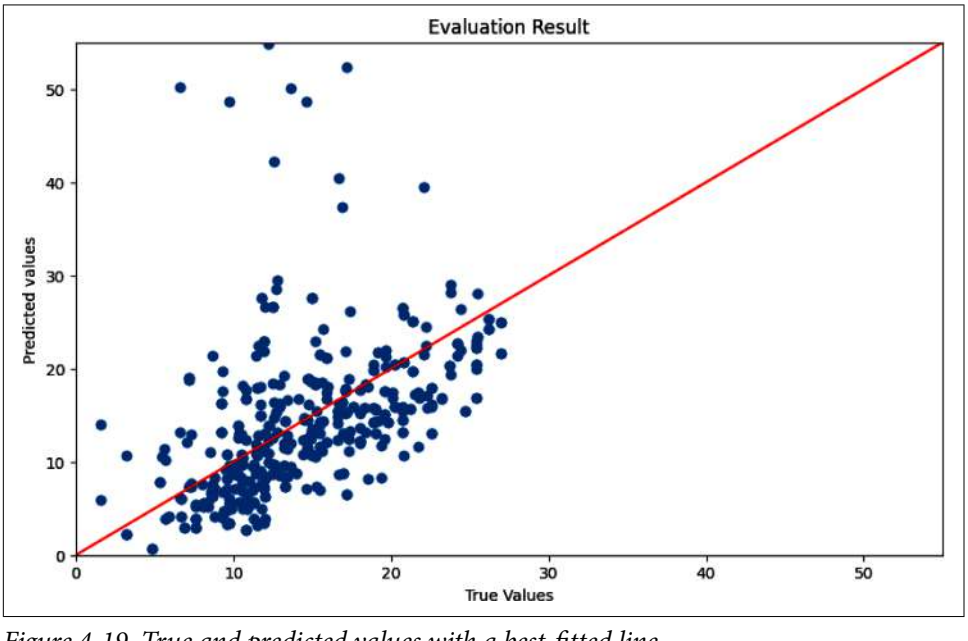


Figure 4-19. True and predicted values with a best-fitted line.

After making the Regression/classification selection, scroll down and click the Create button. You are now ready to add data to your dataset. Vertex AI–managed datasets are available for a variety of data types, including tabular, image, text, and video data.

Figure 4-20 shows the data source upload options—upload CSV from your computer, select CSV files from Cloud Storage, or select a table or view from BigQuery (Google's data warehouse).

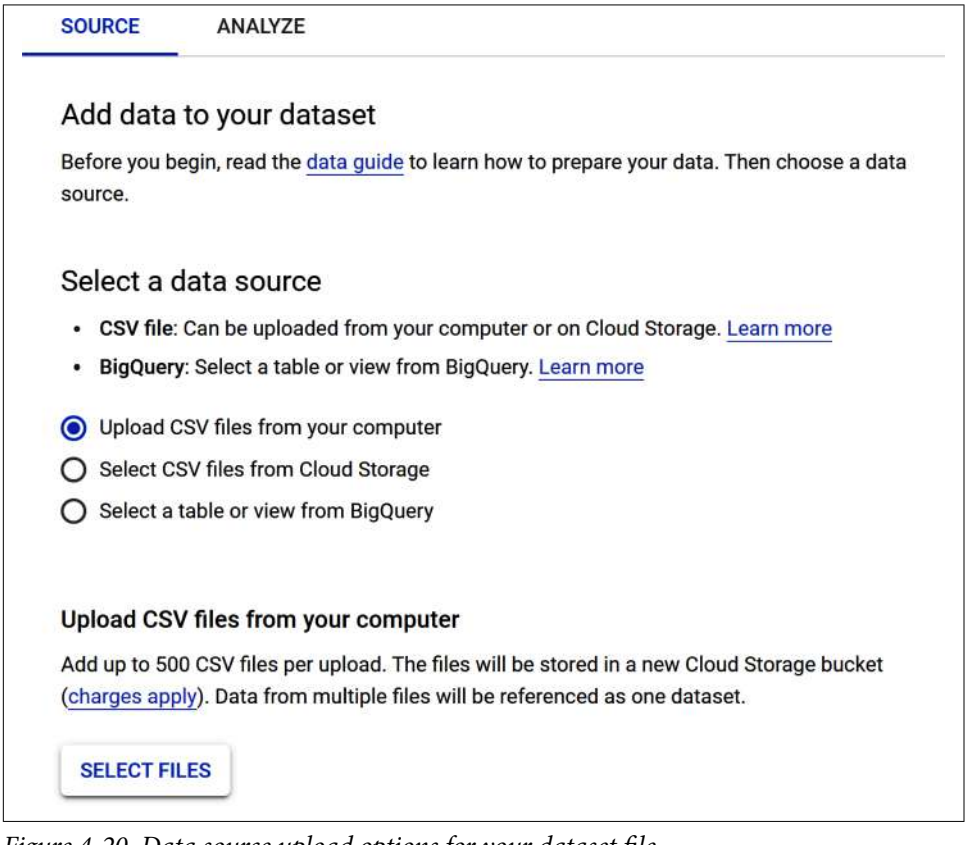


Figure 4-20. Data source upload options for your dataset file.

To upload your advertising dataset, select "Upload CSV files from your computer." Find the file on your local computer and load it.

Scroll down the page and review the Select a Cloud Storage Path section, which requires that you store the file in a cloud storage bucket. Why do you need to store the file in a cloud storage bucket? Two reasons: (1) when training a large-scale ML model, you may need to store terabytes or even petabytes of data; and (2) cloud storage buckets are scalable, reliable, and secure.

Figure 4-21 shows that the file *Advertising_automl.csv* has been uploaded and a cloud storage bucket has been created to store the uploaded file. To see the step-by-step process of creating the storage bucket and the entire exercise, see the PDF entitled *Chapter 4 AutoML Sales Prediction* in the repository.

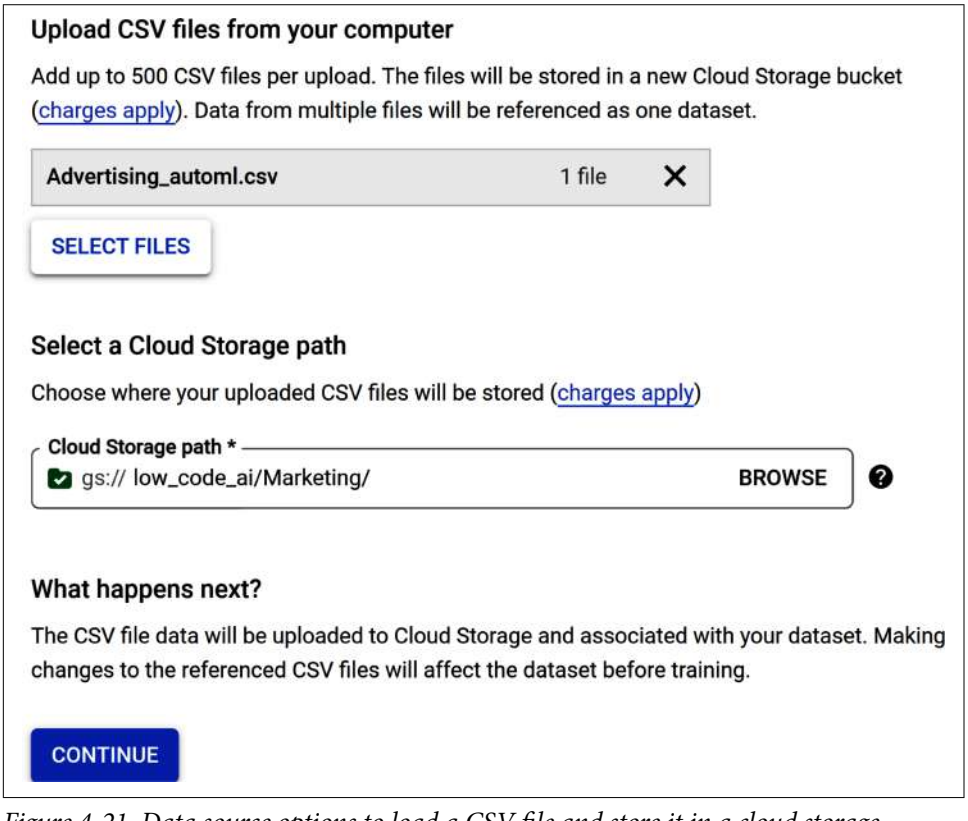


Figure 4-21. Data source options to load a CSV file and store it in a cloud storage bucket.

Some frameworks will generate statistics after the data loads. Other frameworks help minimize the need to manually clean data by automatically detecting and cleaning missing values, anomalous values, and duplicate rows and columns. Note that there are a few additional steps that you can employ, such as to review the data after it has loaded to check for missing values and view data statistics.

Figure 4-22 shows the output of the Generate Statistics window. Note there are no missing values and the number of distinct values for each column is shown.

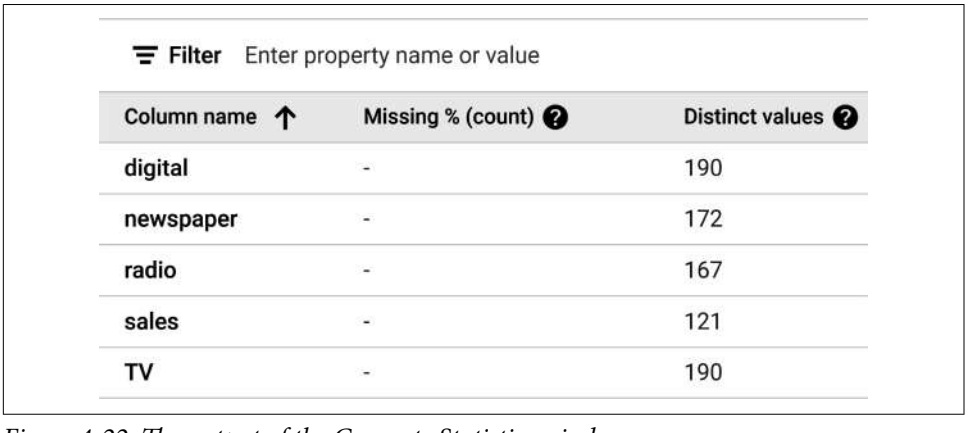


Figure 4-22. The output of the Generate Statistics window.

AutoML presents a data profile of each feature. To analyze a feature, click the name of the feature. A page shows the feature distribution histograms for that feature.

Figure 4-23 shows the data profile for sales. Note that the mean is 14.014, which is very close to the numeric value you received when you typed in the `advertising_df.describe()` code earlier in the chapter as you were exploring the dataset.

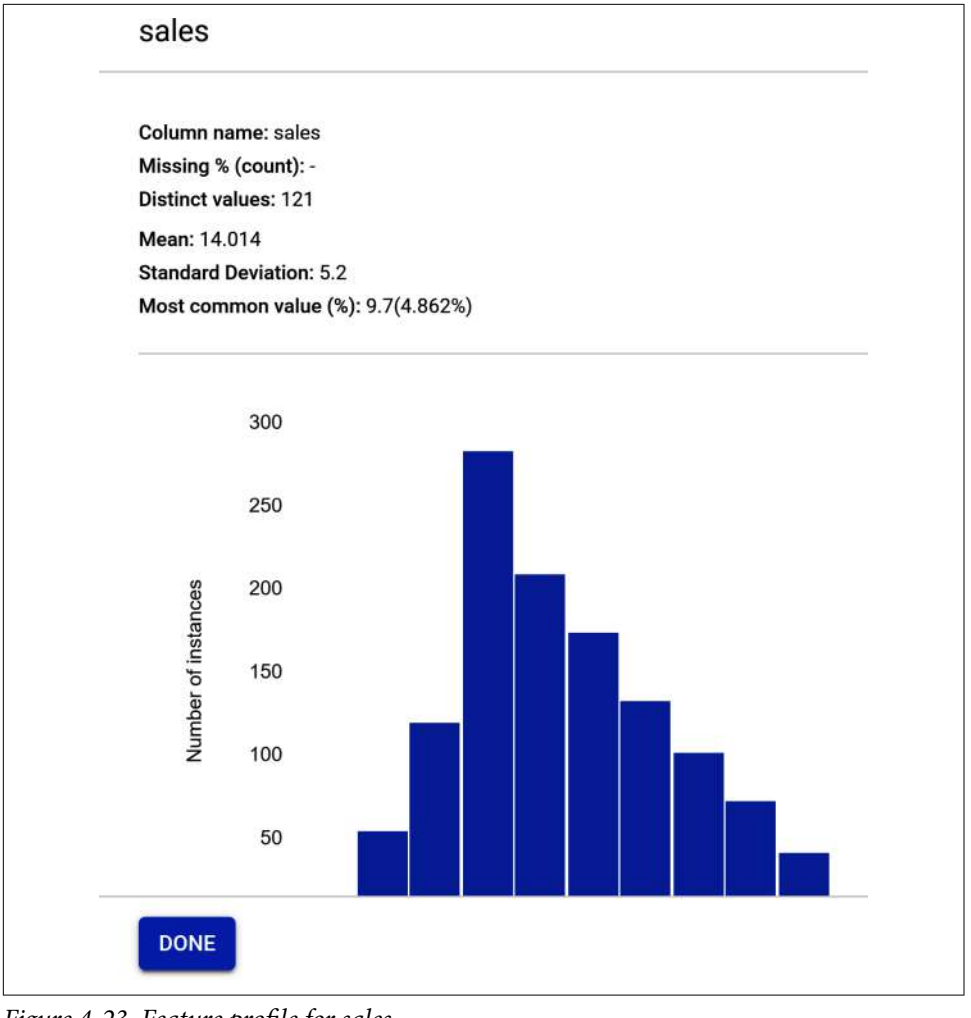


Figure 4-23. Feature profile for sales.

Build the Training Model

Figure 4-24 shows that the model is now ready to train. Select Train New Model under the "Training jobs and models" section. Select Other and not AutoML on Pipelines. AutoML on Pipelines is a feature that allows you to specify the type of ML model you want to build and other parameters. It is beyond the scope of the book.

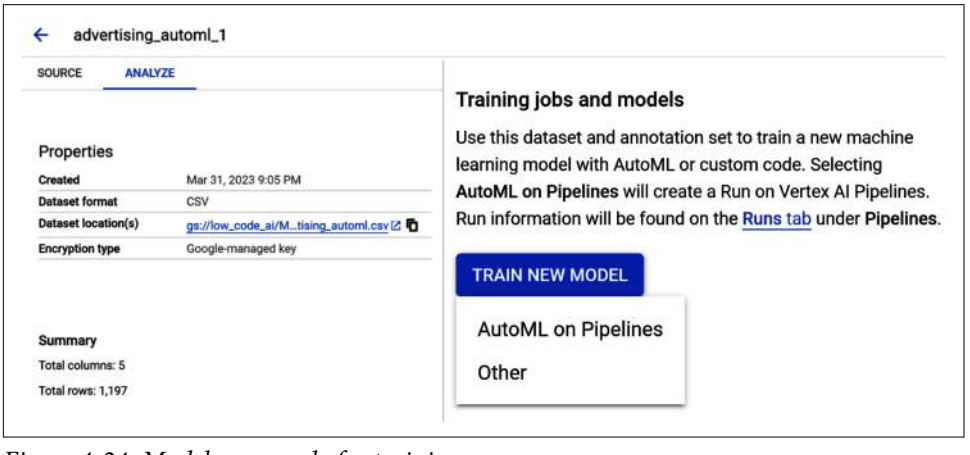


Figure 4-24. Model now ready for training.

The "Train new model" window appears. There are four steps:

1. Select the training method
2. Configure model details
3. Determine training options
4. Select compute and pricing

In Step 1, under the Objective, select the drop-down and choose Regression. Under "Model training method," select AutoML (as shown in Figure 4-25). Click Continue.

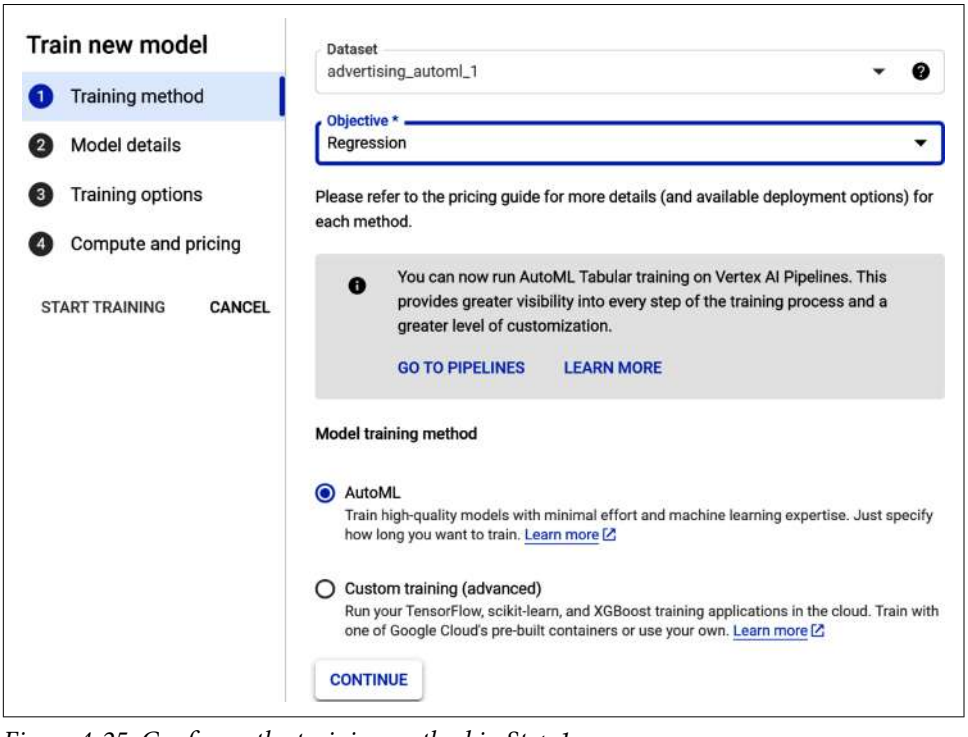


Figure 4-25. Configure the training method in Step 1.

In Step 2, under "Model details," name your model and give it a description. Under "Target column," select sales from the drop-down (as shown in Figure 4-26). Click Continue.

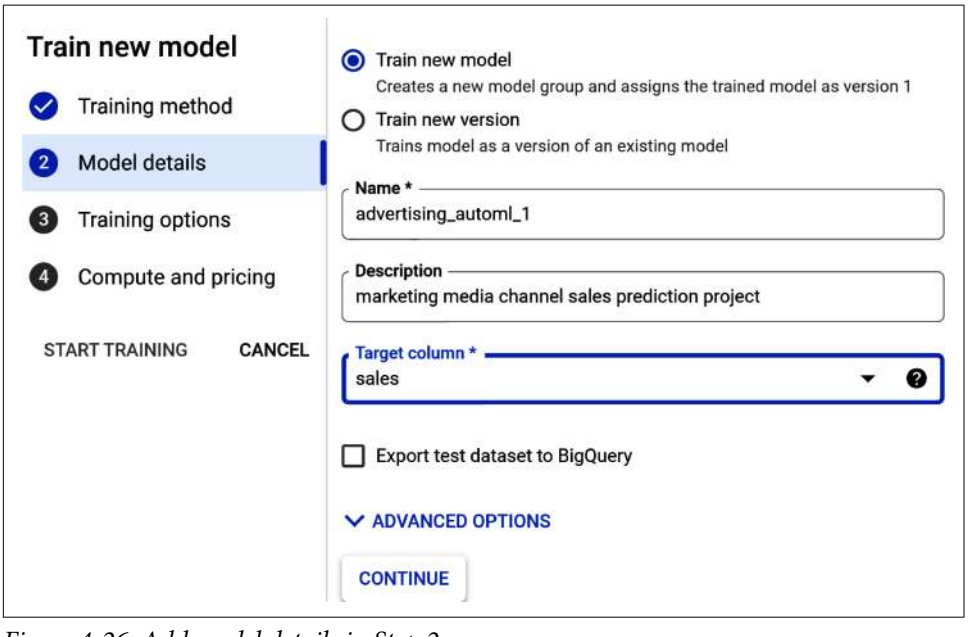


Figure 4-26. Add model details in Step 2.

In Step 3, review the training options. Note that any data transformations (or data processing) such as standardization are handled automatically (as shown in Figure 4-27). Click Continue.

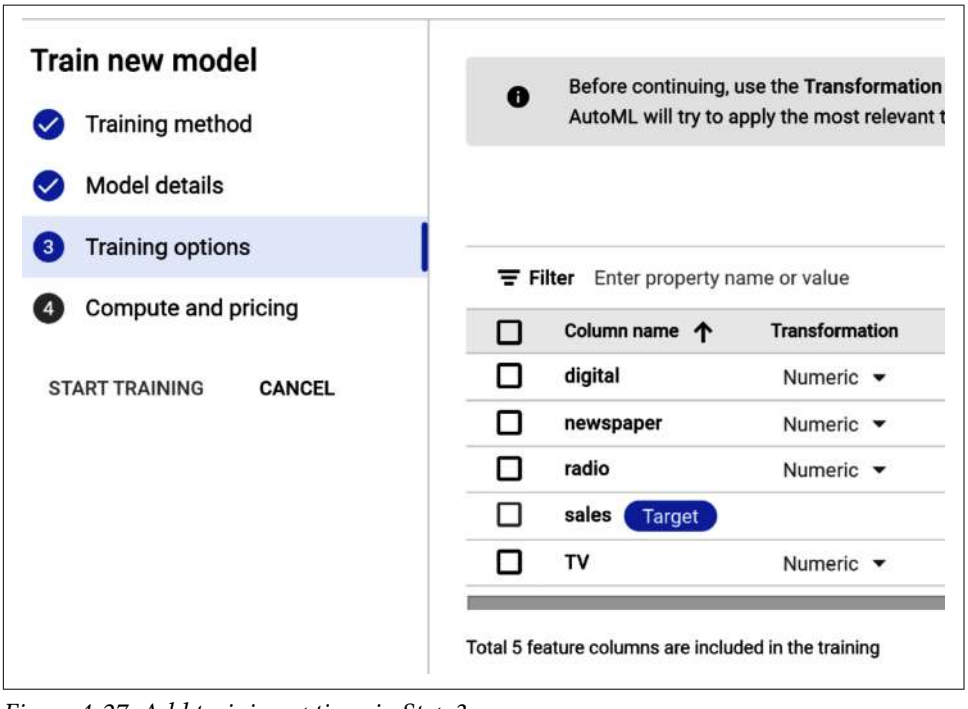


Figure 4-27. Add training options in Step 3.

In Step 4, you see "Compute and pricing" (as shown in Figure 4-28). The time required to train your model depends on the size and complexity of your training data. A *node hour* is one hour's usage of one node (think virtual machine) in the cloud, spread across all nodes. Enter the value **3** in the Budget field for maximum number of node hours—this is just an estimate. You pay only for compute hours used; if training fails for any reason other than a user-initiated cancellation, you are not billed for the time. You are charged for training time if you cancel the operation.

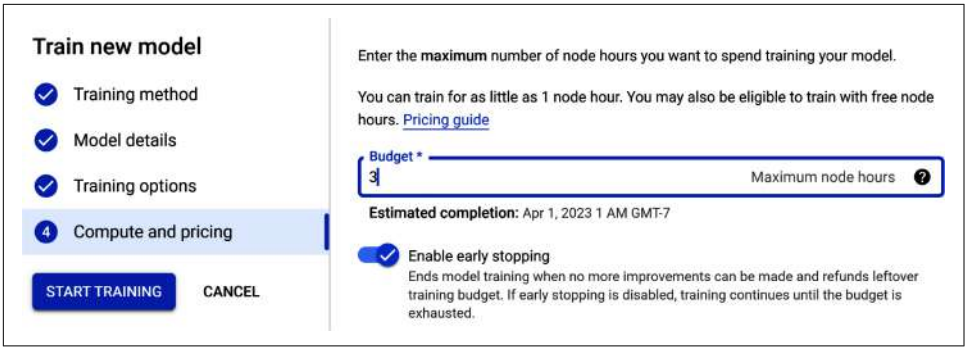


Figure 4-28. Select compute and pricing in Step 4.

Also under "Compute and pricing" is early stopping. When you enable this option, this means that training will end when AutoML determines that no more model improvements can be made. If you disable early stopping, AutoML will train the model until the budget hours are exhausted.

Once all the parameters are entered, you start the training job. Click Start Training.

After model training, the model is registered in the model registry (as shown in Figure 4-29).

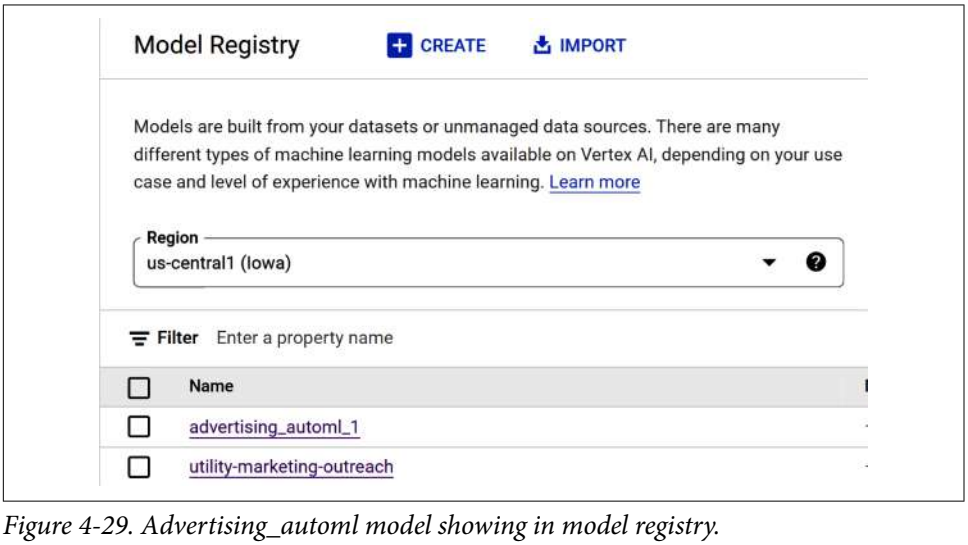


Figure 4-29. Advertising_automl model showing in model registry.



As previously mentioned, training can take up to several hours, depending on the size of your data and type of model objective you choose. Image and video data types may take much longer to process than a structured data type such as a CSV file. The number of training samples also impacts training time.

Also, AutoML is time intensive. AutoML algorithms need to train a variety of models, and this training process can be computationally expensive. This is because AutoML algorithms typically try a large number of different models and hyperparameters, and each model needs to be trained on the entire dataset. AutoML algorithms then need to select the best model from the set of trained models, and this selection process can also be time-consuming. This is because AutoML algorithms typically need to evaluate the performance of each model on a holdout dataset, and this evaluation process can be computationally expensive.

Evaluate Model Performance

Figure 4-30 shows the model training results.

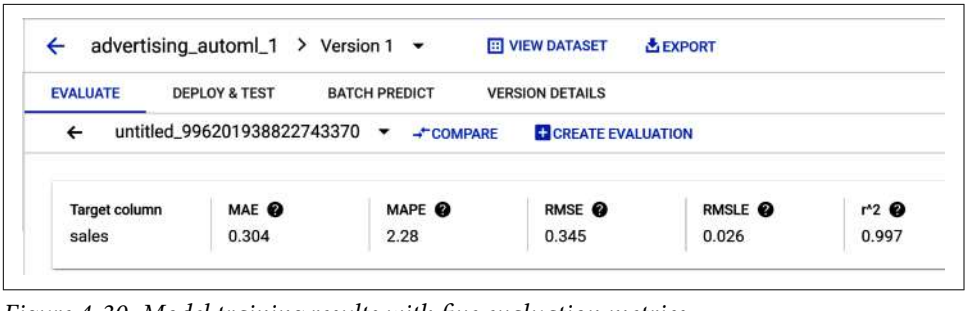


Figure 4-30. Model training results with five evaluation metrics.

There are a few factors that a practitioner should consider when weighing the importance of different linear regression evaluation metrics:

The purpose of the model
: The purpose of the model will determine which evaluation metrics are most important. For example, if the model is being used to make predictions, then the practitioner may want to focus on metrics such as mean squared error (MSE) or root mean squared error (RMSE). However, if the model is being used to understand the relationship between variables, then the practitioner may want to focus on metrics such as R-squared or adjusted R-squared.

The characteristics of the data
: The characteristics of the data will also affect the importance of different evaluation metrics. For example, if the data is noisy (e.g., contains unwanted information or errors), then the practitioner may want to focus on metrics that are robust to noise, such as mean absolute error (MAE). However, if the data is not noisy, then the practitioner may be able to focus on metrics that are more sensitive to changes in the model, such as MSE.

The practitioner's preferences
: Ultimately, the practitioner's preferences will also play a role in determining the importance of different evaluation metrics. Some practitioners may prefer metrics that are easy to understand, while others may prefer metrics that are more accurate. There is no right or wrong answer, and the practitioner should choose the metrics that are most important to them.

Here are common linear regression evaluation metrics:

R-squared
: R-squared is a measure of how well the model fits the data. It is the square of the Pearson correlation coefficient between the observed and predicted values. It is calculated by dividing the sum of squared residuals (the difference between the predicted and the actual values) by the total sum of squares. A higher R-squared value indicates a better fit. R-squared ranges from 0 to 1, where a higher value indicates a higher-quality model. Your R^2 should be around 0.997.

Adjusted R-squared
: Adjusted R-squared is a modified version of R-squared that takes into account the number of independent variables in the model. It is calculated by dividing the sum of squared residuals by the total sum of squares minus the degrees of freedom. A higher adjusted R-squared value indicates a better fit, but it is less sensitive to the number of independent variables than R-squared.

Mean squared error (MSE)
: MSE is a measure of the average squared error between the predicted values and the actual values. A lower MSE value indicates a better fit. Figure 4-31 shows the loss visualized in a table and graph.

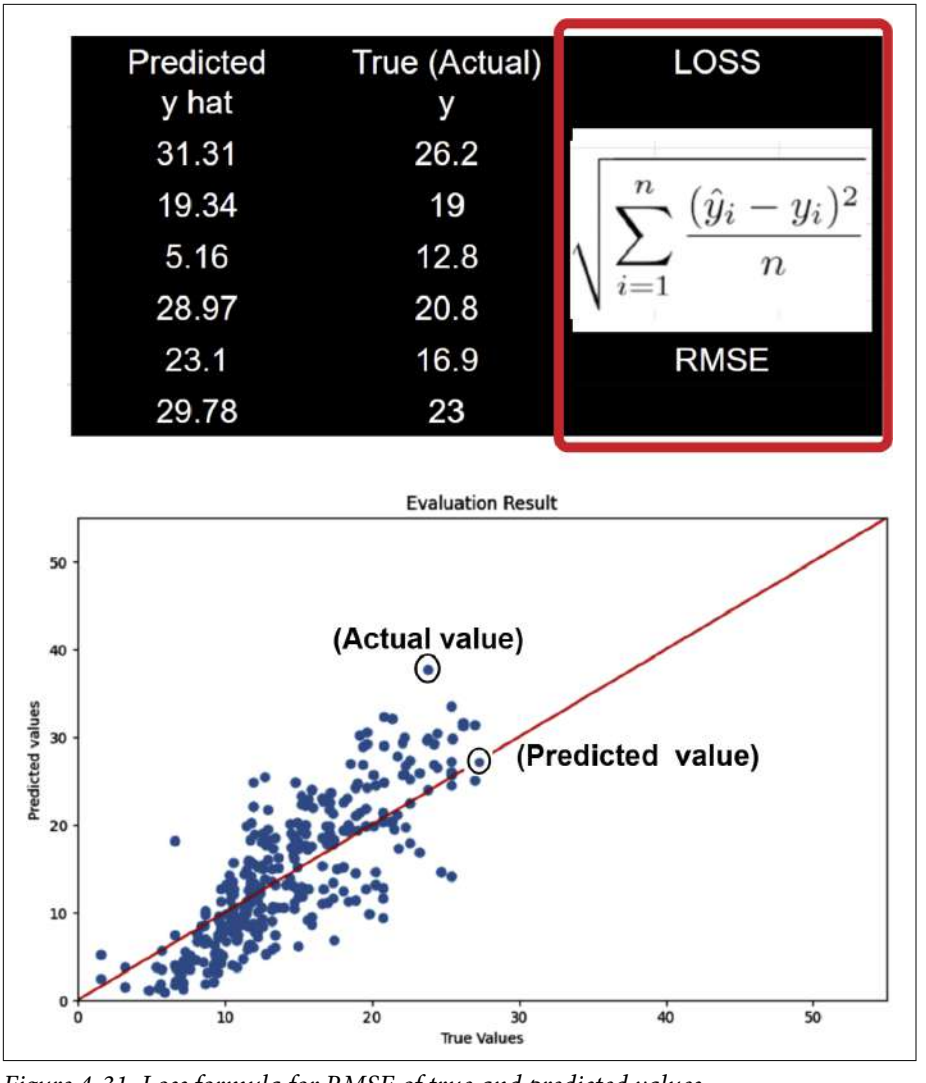


Figure 4-31. Loss formula for RMSE of true and predicted values.

Root mean squared error (RMSE)
: RMSE is the square root of MSE. It is a more interpretable version of MSE. A lower RMSE value indicates a better fit and a higher-quality model, where 0 means the model made no errors. Interpreting RMSE depends on the range of values in the series. Your RMSE should be around 0.345.

Root mean squared log error (RMSLE)
: Interpreting RMSLE depends on the range of values in the series. RMSLE is less responsive to outliers than RMSE, and it tends to penalize underestimations slightly more than overestimations. Your RMSLE should be around 0.026.

Mean absolute error (MAE)
: MAE is a measure of the average absolute error between the predicted values and the actual values. A lower MAE value indicates a better fit. Your MAE should be around 0.304.

Mean absolute percentage error (MAPE)
: MAPE ranges from 0% to 100%, where a lower value indicates a higher-quality model. MAPE is the average of absolute percentage errors. Your MAPE should be around 2.28.

Model Feature Importance (Attribution)

Model feature importance tells you how much each feature impacted model training. Figure 4-32 shows attribution values expressed as a percentage; the higher the percentage, the more strongly the correlation—that is, the more strongly that feature impacted model training. Feature attribution allows you to see which features contributed most strongly to the resulting model training shown in Figure 4-32.

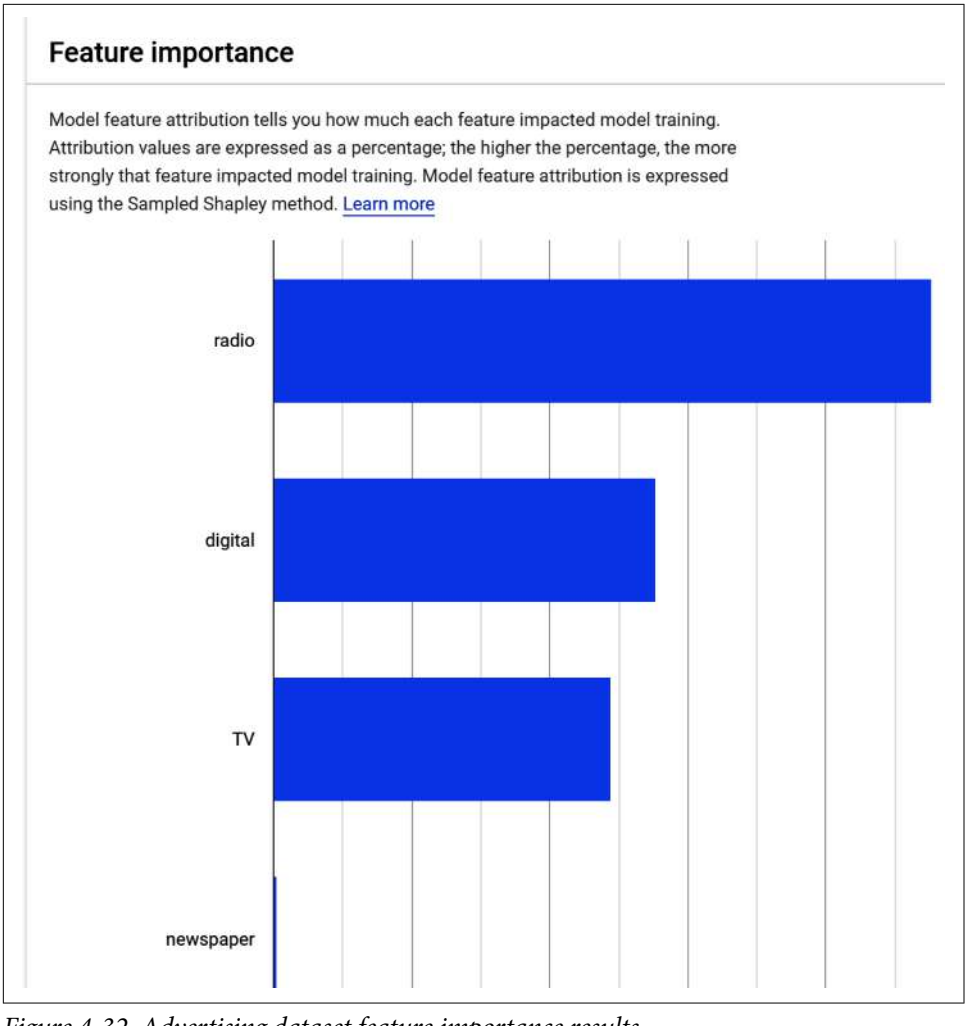


Figure 4-32. Advertising dataset feature importance results.

If you were to hover over the newspaper feature shown in Figure 4-32, you would see that its contribution to model training is 0.2%. This supports what you discovered during the EDA phase you completed earlier—the relationship between sales and newspaper advertising spend is the weakest. These results mean that advertising on *radio*, *digital*, and *TV* contribute the most in *sales*, and *newspaper* advertisements have little effect on *sales*.

Get Predictions from Your Model

To deploy your model, you will need to test it. You can deploy to your environment to test your model without building an application you would need to deploy to the cloud. After you train an ML model, you need to deploy the model so that others can use it to do inferencing. Inferencing in machine learning is the process of using a trained ML model to make predictions on new data.

There are four steps, but for this chapter you'll only need the first two. For this exercise, it is not necessary to configure model monitoring or model objectives. Model monitoring adds an additional charge for logging, while model objectives require you to choose from a variety of model objectives, depending on the type of model you are training and the application you are using it for. Here are the four steps:

1. Define your endpoint.
2. Configure model settings.
3. Configure model monitoring.
4. Configure model objectives.



What are endpoints and deployments? In ML, an *endpoint* is a service that exposes a model for online prediction. A *deployment* is the process of making a model available as an endpoint. An endpoint is an HTTPS path that provides an interface for clients to send requests (input data) and receive the inferencing (scoring) output of a trained model. Endpoints are typically used to make predictions in real time. For example, you could use an endpoint to predict the likelihood of a customer clicking on an ad or the risk of a loan defaulting.

Deployments are typically used to make a model available to a larger audience. For example, you could deploy a model to a production environment so that it can be used by your customers or employees.

Figure 4-33 shows the deploy and test page. To deploy your model, go to Model Registry, select Deploy and Test, and select your model.

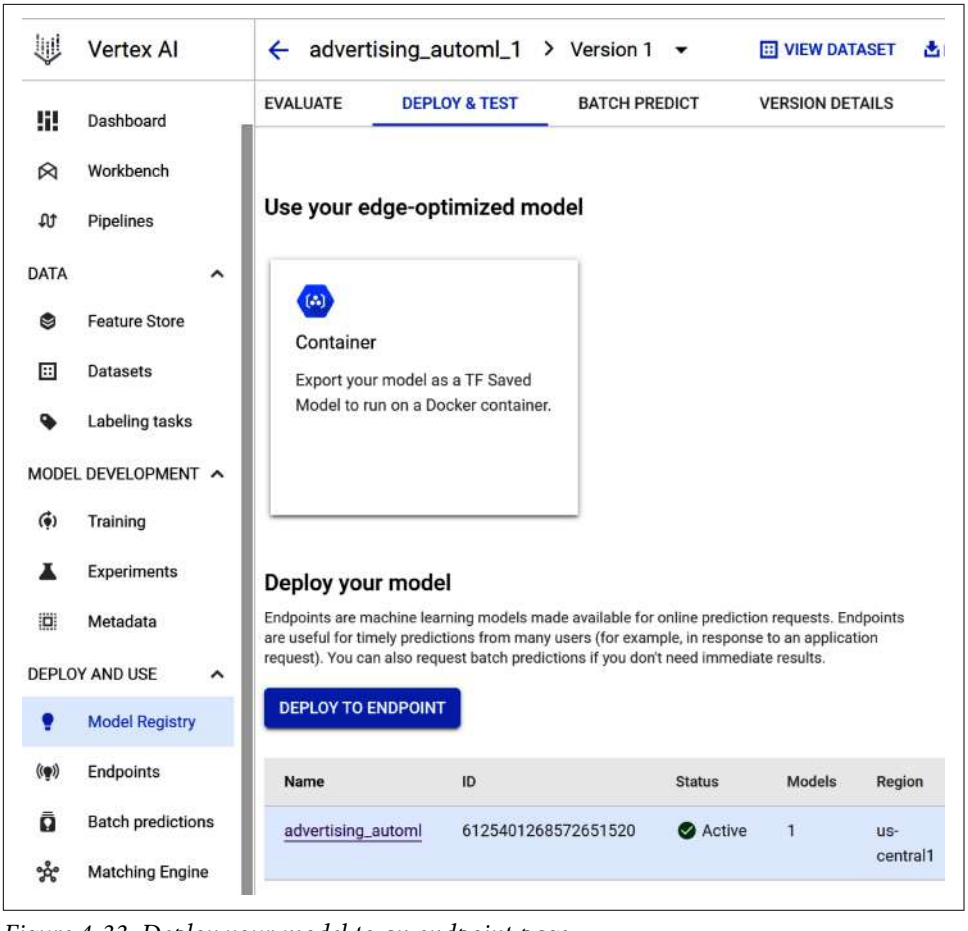


Figure 4-33. Deploy your model to an endpoint page.

In Step 1, you define your endpoint. You select a region and determine how your endpoint will be accessed.

In Step 2, you add the model and add traffic split. A *traffic split* in Vertex AI is a way to distribute traffic between multiple models that are deployed to the same endpoint. This can be useful for a variety of purposes, such as:

A/B testing
: Traffic splitting can be used to A/B test different models to see which one performs better.

Canary deployments
: Traffic splitting can be used to deploy a new model to a small percentage of users before deploying it to a larger audience. This can help to catch any problems with the new model before they affect too many users.

Rollouts
: Traffic splitting can be used to roll out a new model to users gradually. This can help to mitigate the impact of any problems with the new model.

In Step 3, you choose how compute resources will serve the predictions to your model (shown in Figure 4-34). For this exercise, use the minimum number of compute nodes (virtual machine servers). Under "Machine type," select Standard.

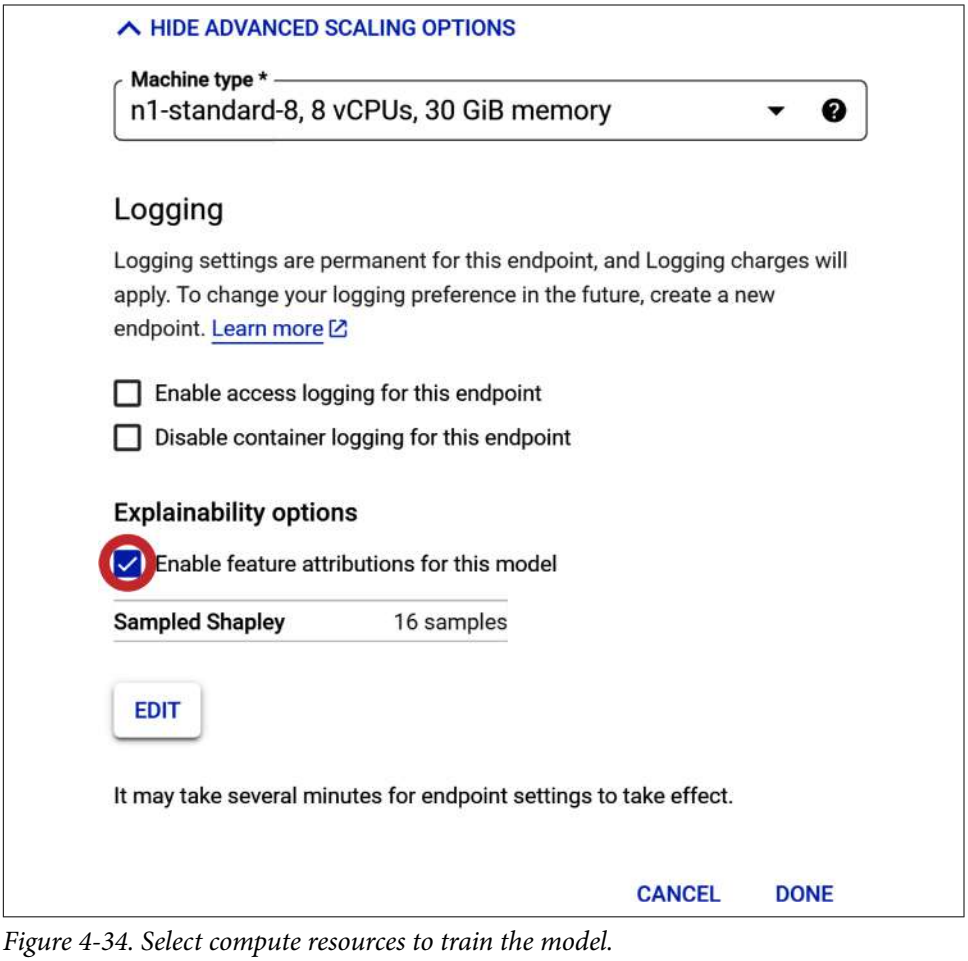


Figure 4-34. Select compute resources to train the model.

Note: "Machine types" differ in a few ways: (1) number of virtual central processing units (vCPUs) per node, (2) amount of memory per node, and (3) pricing.

There are a few factors to consider when choosing compute resources for a prediction model:

Size and complexity of the model
: The larger and more complex the model, the more compute resources it will need. (This is more applicable to custom coding neural networks.)

Number of predictions that will be made
: If you expect to make a large number of predictions, you will need to choose a compute resource that can handle the load.

Latency requirements
: If you need to make predictions in real time or with very low latency, you will need to choose a compute resource that can provide the necessary performance. Note: low latency in machine learning refers to the time it takes for an ML model to make a prediction once it receives a new data point.

Cost
: Compute resources can vary in price, so you will need to choose one that fits your budget.

Once you have considered these factors, you can start to narrow down your choices. Here are a few examples of compute resources that can be used to serve prediction models:

CPUs
: Central processing units (CPUs) are the most common type of compute resource and are a good choice for models that are not too large or complex.

GPUs
: Graphics processing units (GPUs) are more powerful than CPUs and can be used to speed up the training and inference of large and complex models.

TPUs
: Tensor processing units (TPUs) are specialized hardware accelerators that are designed for ML workloads. They are the most powerful option and can be used to train and serve the most demanding models.

As part of Step 2 under "Model settings," there is a Logging setting. If you enable endpoint logging, charges will apply. Thus, for this exercise, please do not enable it.

The next setting is Explainability options, which do not carry a charge. Check "Enable feature attributions for this model."

Step 3 is "Model monitoring." Do not enable it for this project (as shown in Figure 4-35).

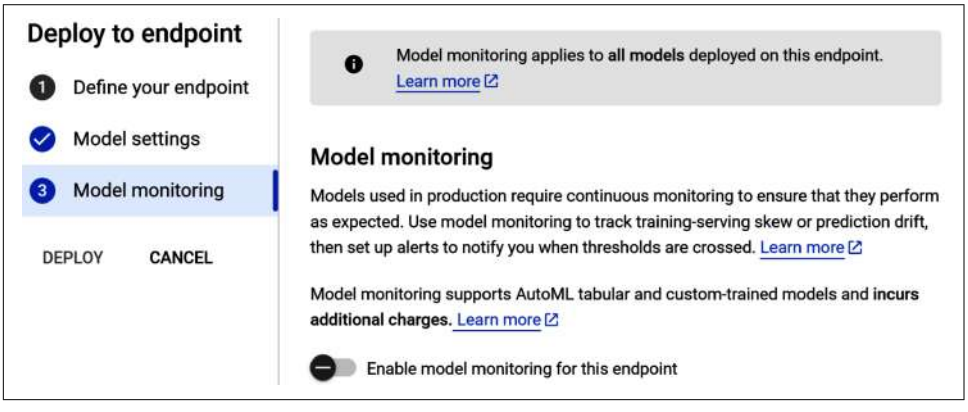


Figure 4-35. "Model monitoring" configuration window.

Now that all configurations are made, the Deploy button should be highlighted. Click Deploy to deploy your model to the endpoint (as shown in Figure 4-36).

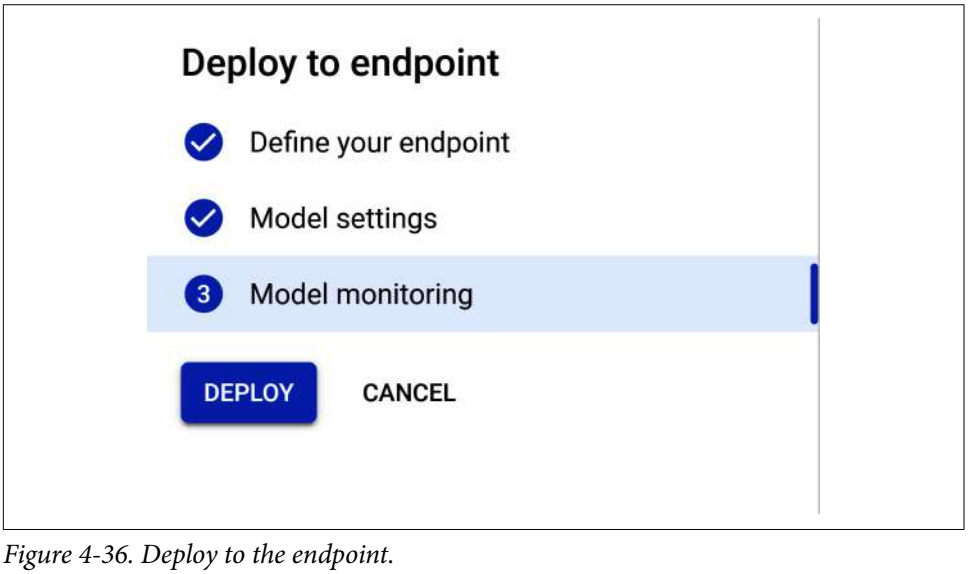


Figure 4-36. Deploy to the endpoint.

After the endpoint is created and the model deployed to the endpoint, you should receive an email regarding the endpoint deployment status. If deployment was successful, you can start making predictions. There are four steps:

1. Go to Model Registry.
2. Select your model.
3. Select the model's version.
4. Scroll down until you see the "Test your model" page.

Figure 4-37 shows the "Test your model" page. Note, this page could be an app or web page that looks like this—where you and your team input media channel values and predict sales volume.

Click the Predict button.

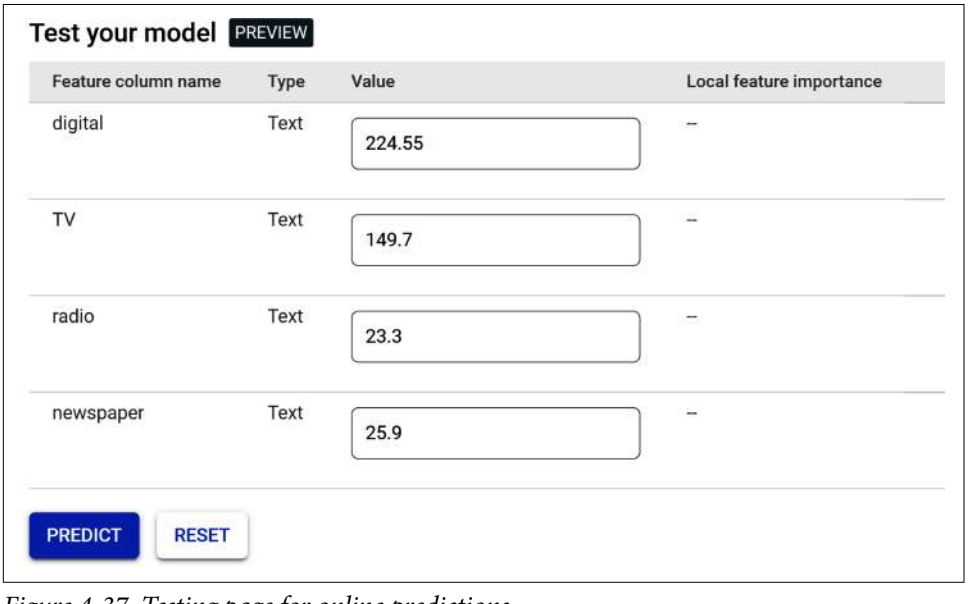


Figure 4-37. Testing page for online predictions.

After clicking the Predict button, you will get a prediction for your label (sales), as shown in Figure 4-38.

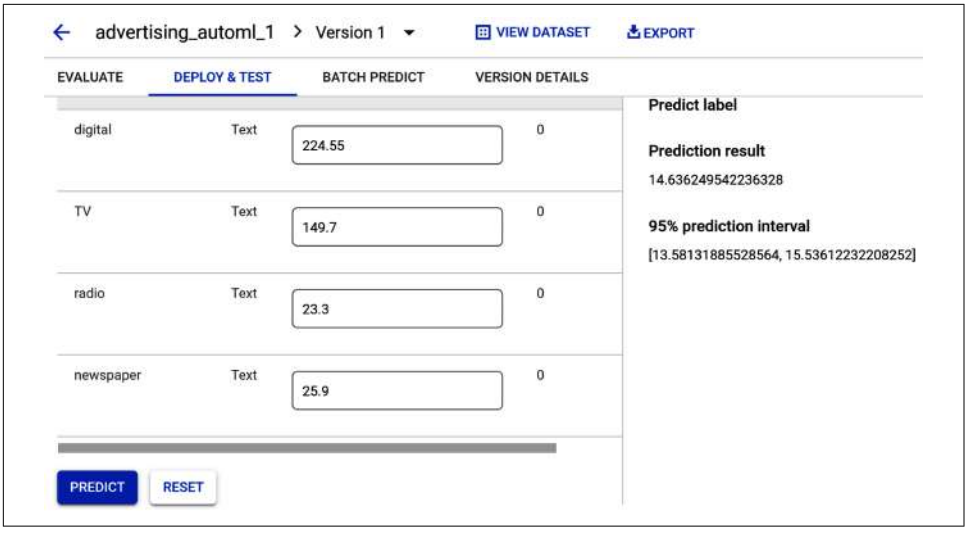


Figure 4-38. Prediction of sales volume based on initial values.

Regression models return a prediction value. Figure 4-38 shows a sales prediction result value of 14.63—which is very close to the mean from the sales histogram (shown in Figures 4-10 and 4-23). The prediction interval provides a range of values that the model has 95% confidence to contain the actual result. So, since the sales prediction result is 14.63, and the prediction interval is a range between 13.58 and 15.53, you can be 95% certain that any prediction result will fall within this range.

The Prediction Interval in Linear Regression

Prediction intervals are often used in regression analysis. A prediction interval is different from a confidence interval—though both are statistical measures that provide an estimate of the range of values within which a true value is likely to live. A *confidence interval* focuses on past or current events and is used to estimate a population parameter (e.g., standard deviation, mean).

A *prediction interval* is used to predict the value of a future observation, given what has already been observed. It provides an estimated range of values that may contain the value of a single new observation, based on previous data.

If you create a regression model, you can use it to develop a prediction interval that can determine where the next data point sampled may appear. Prediction intervals are wider than confidence intervals. This is because prediction intervals account for the uncertainty associated with predicting an individual value, as opposed to a population parameter.

> Prediction intervals can be used to make decisions about future observations. For example, a company might use prediction intervals to decide whether to invest in a new product or service or to increase or decrease an advertising media channel budget.

Now, let's answer those business questions.

The goal was to build an ML model to predict how much sales volume will be generated based on the money spent in each of the media channels.

Can the model predict how much sales volume will be generated based on the money spent in each media channel?

Yes. Since Vertex AI allows you to input values for each media channel, you can now make decisions about future budget allocation. For example, your company's strategic media plan may now include increasing the digital channel budget based upon the results obtained from a prediction.

Is there a relationship between advertising spend and sales?

Yes. There is a positive linear relationship between advertising spend and sales for digital, TV, and radio. Newspaper spend has a weak relationship to sales.

Which media channel contributes the most to sales?

TV contributes more to sales than the other media channels. How? The scatterplot you built during the EDA section, and your review of the Vertex AI feature's attribution bar chart after the model was trained, show the contribution of TV to sales.

How accurately can the model predict future sales?

The regression model returns a prediction value when media channel values are input into the Predict window to predict sales volume. The prediction results showed a sales prediction value and prediction interval. Prediction intervals can be used to make decisions about future observations, so you can be 95% certain that any future sales prediction result will fall within that range.



Do not forget to *undeploy* your model once you are finished with this chapter. Deployed models incur cost even when they are not being used so that they are always available to return quick predictions. To undeploy the model, go to Vertex AI Endpoints, click the endpoint name, then click the "More actions" three-dot menu, and finally click "Undeploy model from endpoint."

Summary

In this chapter, you built an AutoML model to predict advertising media channel sales. You explored your data using Pandas, creating heat maps, scatterplots, and histograms. After you exported the data file, you uploaded it into Google's Vertex AI framework. Then you learned how to use Google Cloud's AutoML to build, train, and deploy an ML model to predict sales. You gained an overall understanding of the performance of your model using performance metrics and answered common business questions. You used the model to make online predictions and do a bit of budget forecasting. You are now ready to present to your team!

CHAPTER 5

Using AutoML to Detect Fraudulent Transactions

In this chapter, you build a Vertex AI AutoML model to predict whether a financial transaction is fraudulent or not. You will clean and explore the dataset in a Google Colab notebook environment before creating a managed dataset on Vertex AI as you did in Chapter 3. Once you have created a managed dataset, you will use AutoML to create a classification model to predict if a transaction is fraudulent or not. Along the way, the chapter discusses classification models in general and the corresponding metrics that are commonly used to evaluate them.

The overall workflow of this chapter is very similar to what you worked through in Chapter 4 for the problem of predicting advertising media channel sales. For this reason, in many places in this chapter you will see more concise details where the conversations would be very similar. If you get stuck in these sections, please refer back to Chapter 4 for more details.

The Business Use Case: Fraud Detection for Financial Transactions

Your task in this chapter, as mentioned, is to build a model to predict whether a financial transaction is fraudulent or legitimate. Your new company is a mobile payment service that serves hundreds of thousands of users. Fraudulent transactions are fairly rare and are usually caught by other protections. However, the unfortunate truth is that some of these are slipping through the cracks and negatively impacting your users. Your company can rectify these after the fact, but there is a fear of losing customers due to them having to report these transactions. The goal is to improve the fraud-detection software that your company is using by leveraging machine learning (ML) to build a bespoke model.

One complicating factor will be that the corresponding dataset will be highly unbalanced. The vast majority of transactions are going to be legitimate transactions, so a simple model that predicts that all transactions are legitimate would be equally accurate and useless. You will need to leverage other metrics to better understand your model's performance along the way.

This task might normally be one that is passed to a data scientist to create some sort of advanced model (such as an autoencoder), but you have been tasked to quickly get together a benchmark model that could be used to prototype other parts of the fraud-detection system. That may seem like a hopeless task, but remember in the previous project you were able to quickly create such a prototype using AutoML for media channel sales prediction. So, you should feel confident that you are up to the challenge!

Project Workflow

The project workflow in this chapter, illustrated in Figure 5-1, will be similar to that in the previous chapter. For this reason, some details of the process will be omitted to avoid repetition, but feel free to refer back to the previous chapter as needed.

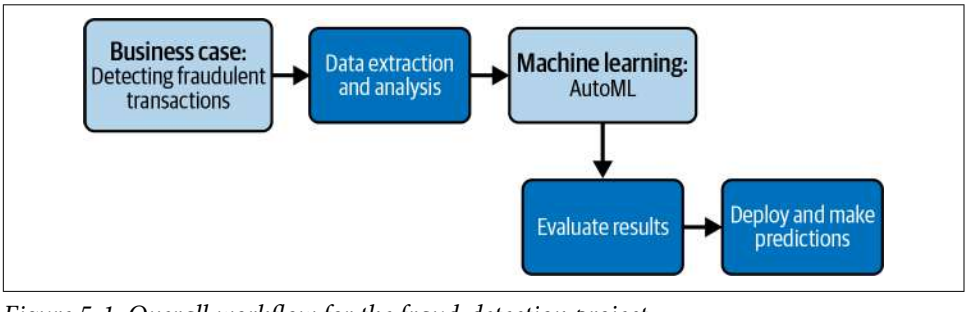


Figure 5-1. Overall workflow for the fraud-detection project.

Now that you understand the business use case and objective, you can proceed to data extraction and analysis as in your previous project. After the data extraction and analysis steps, you will upload the dataset into the AutoML platform. The various features (soon to be introduced) will be fed into the model. You'll evaluate the AutoML results and then deploy the model to make predictions. After this activity is completed, you will have the benchmark model ready for the engineering team to start developing a better fraud-detection pipeline. And who knows, this model may actually be the one that goes into production.

Project Dataset

The project dataset consists of transaction data that has been simulated to replicate user behavior and fraudulent transactions. This has been done using PaySim. PaySim is an open source tool developed by a group of researchers who at the time were studying scalable resource-efficient systems for big data analytics.[1]

Since financial transaction data can be difficult to use without exposing user information, your company decided to use this simulated data. Data analysts at your company have confirmed that the dataset that has been shared is really similar in distribution to the actual data that your company sees in its application, so you can move forward assuming that the data is representative of the real-world data that your company wants to leverage at prediction time.

The dataset has been provided as a CSV file in Google Cloud Storage (download at *https://oreil.ly/n1y1X*). In your version of the dataset, there are 10 columns. Table 5-1 gives the column names, data types, and some information about the possible values for these columns.

Table 5-1. Schema and field value information for the customer churn dataset

Column name	Column type	Notes about field values
`step`	Integer	Represents number of hours since simulated data was generated
`type`	String	Type of transaction
`amount`	Float	Amount of transaction
`nameOrig`	String	Anonymized name of customer who originated transaction
`oldbalanceOrg`	Float	Initial balance before the transaction for the originator
`newbalanceOrig`	Float	New balance after the transaction for the originator
`nameDest`	String	Anonymized name of the customer who owns the destination account
`oldbalanceDest`	Float	Initial balance before the transaction for the destination account
`newbalanceDest`	Float	New balance after the transaction for the destination account
`isFraud`	Integer	1 if the transaction was fraudulent and 0 otherwise

1 For more details, see "PaySim: A Financial Mobile Money Simulator for Fraud Detection" by E. A. Lopez-Rojas, A. Elmir, and S. Axelsson (in The 28th European Modeling and Simulation Symposium-EMSS, Larnaca, Cyprus, 2016).

Exploring the Dataset Using Pandas, Matplotlib, and Seaborn

All of the code in this section, including some additional examples, is included in a Jupyter notebook in the low-code-ai repository on GitHub. You will walk through how to create this notebook from the very beginning, but the notebook in GitHub is great to use as a resource for later independent work or in case you get stuck.

As in Chapter 4, you will be loading a CSV file into a Jupyter Notebook environment and analyzing and exploring your data using Pandas, Matplotlib, and Seaborn to get a better understanding of your features. You will use the information you gain from this process to select the best set of features for your model in AutoML.

Loading Data into a Pandas DataFrame in a Google Colab Notebook

First, go to *https://colab.research.google.com* and open a new notebook, following the process discussed in Chapters 2 and 4 for notebook creation in Google Colab. You may rename this notebook to a more meaningful name by clicking on the name and replacing the current name with a new one, say, *Fraud_Detection_Model.ipynb*.

Next, type the following code into the first code block to import the packages needed to analyze and visualize the financial transactions dataset and execute the cell:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

%matplotlib inline
```

Now that the packages are imported, the next step is to load the data into a Pandas DataFrame:

```
url = ('https://storage.googleapis.com/' +
'low-code-ai-book/financial_transactions.csv')

transaction_df = pd.read_csv(url)
```

Your data is now loaded into the DataFrame. It is always a good idea to take a peek at a few rows of data before moving forward. To look at the first few rows of data, add the following code to a new cell and execute that cell:

```
transaction_df.head()
```

Take a look now at the first five rows of data in the DataFrame. This will give you some insight into what the different columns look like. What do you notice about the data? Tables 5-2 and 5-3 show the output from this line of code.

Table 5-2. The first six columns of output from the `transaction_df.head()` statement

step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig
1	PAYMENT	9839.64	C1231006815	170136.0	160296.36
1	PAYMENT	1864.28	C1666544295	21249.0	19384.72
1	TRANSFER	181.00	C1305486145	181.0	0.00
1	CASH_OUT	181.00	C84003671	181.0	0.00
1	PAYMENT	11668.14	C2048537720	41554.0	29855.86

Table 5-3. The last five columns of output from the `transaction_df.head()` statement

nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
M1979787155	0.0	0.0	0	0
M204428225	0.0	0.0	0	0
C553264065	0.0	0.0	1	0
C38997010	21182.0	0.0	1	0
M1230701703	0.0	0.0	0	0

Note that so far the `step` column has a value of `1` for all of the rows you're looking at. There are many reasons this could be the case. For example, all rows could have the same value, the data could be grouped up in some way by step, or it could just be a coincidence since we are only looking at five rows. In this case, it is actually that the data is sorted by `step` in ascending order, though this is not clear at all from only five rows of data.

You may also notice a pattern with the `type` column with the `oldbalanceDest` and `newbalanceDest` columns. In these rows, whenever there is a `PAYMENT` or `TRANSFER` transaction type, the `oldbalanceDest` and `newbalanceDest` columns are both zero. This could be a coincidence, but it is something you should explore in the dataset later on.

In the last column in the DataFrame, you should notice something odd. There is a column, `isFlaggedFraud`, that you did not see in the original schema. There are a lot of reasons this could be the case, but this is another reason why basic data exploration can be useful. If you run across this sort of scenario in practice, you may have to reach back out to the person or team who shared the data so that you can validate whether the column should be there, and if so what it represents.

In this case, you learned that this was a column that gave the output of the previous model's prediction if the transaction was legitimate or fraudulent. Why would this be a bad feature for your model to use? Because you may not have `isFlaggedFraud` at prediction time. Ideally, this is coming from a model that you will deprecate once a new and better model has been deployed. However, this discovery is still useful. You can use the previous model as a benchmark to compare against to be sure that your new model shows improvement.

Exploring the Dataset

As you saw in Chapter 4, AutoML will do a lot of the work of describing your features and performing feature engineering before training your model, but it is important that you still take the time to understand the data you are using for training your model. ML is very sensitive to the quality of your data. If there are problems with your data, they will not magically go away in the process of using AutoML.

Descriptive analysis

First, use the `.info()` method on your DataFrame to get a quick understanding of the amount of data you are working with and the data types of the columns. Add the following code into a cell and execute it to do so:

```
transaction_df.info()
```

You should see that there are 6,362,620 rows of transaction data with the 11 columns that you expect. The data types all match the expected types (with the `object` type corresponding to the String data type in the Pandas DataFrame).

As before, the easiest way to get a quick look at the descriptive statistics of your dataset in Pandas is to use the `.describe()` method. Create a new cell in your notebook, add the following code, and execute the cell to see the descriptive statistics (as seen in Figure 5-2):

```
transaction_df.describe()
```

```
1  transaction_df.describe()
```

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06
mean	2.433972e+02	1.798619e+05	8.338831e+05	8.551137e+05	1.100702e+06	1.224996e+06	1.290820e-03	2.514687e-06
std	1.423320e+02	6.038582e+05	2.888243e+06	2.924049e+06	3.399180e+06	3.674129e+06	3.590480e-02	1.585775e-03
min	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.560000e+02	1.338957e+04	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
50%	2.390000e+02	7.487194e+04	1.420800e+04	0.000000e+00	1.327057e+05	2.146614e+05	0.000000e+00	0.000000e+00
75%	3.350000e+02	2.087215e+05	1.073152e+05	1.442584e+05	9.430367e+05	1.111909e+06	0.000000e+00	0.000000e+00
max	7.430000e+02	9.244552e+07	5.958504e+07	4.958504e+07	3.560159e+08	3.561793e+08	1.000000e+00	1.000000e+00

Figure 5-2. Rendered output of `transaction_df.describe()`.

It seems like there are no null values by looking at the count for each row. The `step` field varies between 1 and 743 fairly uniformly. The `amount`, `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest`, and `newbalanceDest` vary over a large range, but this is reasonably to be expected of financial transactions.

The `isFraud` column shows an interesting feature of the dataset. The `mean` of a field with values of only 0 and 1 corresponds to the proportion of values that are equal to 1. In this case, about 0.129% of transactions were indeed fraudulent, or about 1 in every 800 transactions. This represents a fairly unbalanced dataset that can cause issues when attempting to build a classification model. You will explore these issues and possible solutions in later sections, but for now continue to explore the data.

The `isFlaggedFraud` column has about double the number of rows flagged as fraudulent transactions compared with the actual amount of fraudulent transactions. Is this a bad thing? Not necessarily. You want to be sure you are catching the fraudulent transactions, and if you flag a few extra transactions and recognize them as legitimate transactions upon further consideration, then it is a small price to pay for capturing fraudulent transactions. However, if you flag too many additional transactions, then you will be wasting a significant amount of time and resources exploring legitimate transactions. These are both things you will need to consider when evaluating models in the near future.

There are a few columns missing from your descriptive analysis, though. Notice that you only have statistics for the numeric features (the columns of type integer and float), but you're missing data from the `type`, `nameOrig`, and `nameDest` columns. The `describe()` method will only return the statistics for numerical features if there is a mix of numerical and categorical features. To see descriptive statistics for categorical features, limit the scope to just those columns. To do that, add the following code to a new cell and execute that cell:

```
cols = ['type', 'nameOrig', 'nameDest']
transaction_df[cols].describe()
```

You are defining a list of columns you want to see the descriptive statistics for, and then using the notation `transaction_df[cols]`, only considering those columns for the `.describe()` method (Figure 5-3).

```
1  cols = ['type','nameOrig', 'nameDest']
2  transaction_df[cols].describe()
```

	type	nameOrig	nameDest
count	6362620	6362620	6362620
unique	5	6353307	2722362
top	CASH_OUT	C1902386530	C1286084959
freq	2237500	3	113

Figure 5-3. Code and output of the `describe()` method for categorical columns.

Once again, we see that there is a value for every row for each of these columns by comparing the `count` with the number of rows in the DataFrame. As expected, there are five unique values for the `type` column, but notice that the `nameOrig` column is almost in a one-to-one relationship with the transactions. In particular, there are 6,353,307 values for `nameOrig` and 6,362,620 total transactions. This is not a good sign for this being a useful feature for building a good model. Why? Because if there is a feature that identifies a row, or nearly identifies a row, then you risk the model only looking at that value and not learning more complex relationships between the features. This seems to be less of an issue for the `nameDest` column, but it's still a risk given the large number of unique values.



AutoML will perform feature engineering and selection for you, but if you can give it a better set of features to start with, you will still see better performance. For example, you could re-create the `nameOrig` and `nameDest` features as a Boolean-valued column where the value is `True` if the value is repeated more than once and `False` otherwise. Is this the right approach? Maybe. To verify this, you should reach out to domain experts and experiment with different feature transformations to see what works best. You will learn more about feature engineering as a whole when working through upcoming chapters.

Exploratory analysis

Before visualizing certain features of the data, it is worth exploring how well the previous model is performing. One easy way to do this is by creating a new column, which you will call `isCorrect`, which is a simple 1 if `isFraud` and `isFlaggedFraud` are the same and 0 otherwise. To compute this new column and count the number of times that the old model correctly predicted fraudulent transactions, execute the following code in a new cell:

```
transaction_df['isCorrect'] = (
    transaction_df['isFraud'] == transaction_df['isFlaggedFraud']
)
transaction_df['isCorrect'].sum()
```

You should see the result of `6354423`, but what does that actually mean? In Python, the Boolean data type has two possible values: `True` and `False`. When you use the `sum` function, it treats the `True` value as 1 and the `False` value as 0. The result that you see then is the number of `True` values from the statement `transaction_df['isFraud'] == transaction_df['isFlaggedFraud']`. This statement returns `True` when the columns have the same value and `False` otherwise. Recall that there were `6362620` rows of data total, which means that 99.87% of transactions were correctly flagged.

So, the old model was a really good model, right? Most of the transactions are not fraudulent, and those are the ones that you are most worried about. But consider another related question: How many fraudulent transactions did it correctly predict? You can figure that out by asking where the `isFraud` column has a value of `1` and the `isCorrect` column also has a value of `1`. Execute the following code in a new cell to see the results:

```
(transaction_df['isFraud']*transaction_df['isCorrect']).sum()
```

Why does that line of code tell you the number of fraudulent transactions that were successfully flagged? The `isFraud` column has a value of 1 only if the transaction was fraudulent, and the `isCorrect` column is `True` (which is treated as `1` for arithmetic) if the prediction was correct. So, multiplying the values of these columns will only give 1 if both columns have a value of 1—otherwise that product will be 0. Adding up the number of 1s gives the total number of correctly flagged fraudulent transactions.

The number of correctly marked fraudulent transactions was 16. By using the code `transaction_df['isFraud'].sum()`, you see that the total number of fraudulent transactions was 8,213. That means only 0.19% of fraudulent transactions were successfully flagged. This model, which looked great when we saw the overall accuracy, did very poorly when we asked a slightly different, maybe more relevant question. This is an example of a metric known as *recall*. When selecting the model objective and considering evaluation metrics later in the chapter, you will explore how to interpret the different available metrics in AutoML.

The next way that you can visualize the data is to create a bar chart for categorical features to understand the percentage of fraudulent transactions by value. For example, for the `type` feature, there are five values. To create such a bar chart, use the following code in a new cell to create the visualization:

```
transaction_df.groupby('type')['isFraud'].mean().plot.bar()
```

This line of code groups the rows by the value of `type` and takes the mean of the `isFraud` column. Recall that since `isFraud` has a value of 1 or 0, this is equivalent to the percentage of values that are 1. You then use the `plot.bar()` method to plot the results as a bar graph, as shown in Figure 5-4.

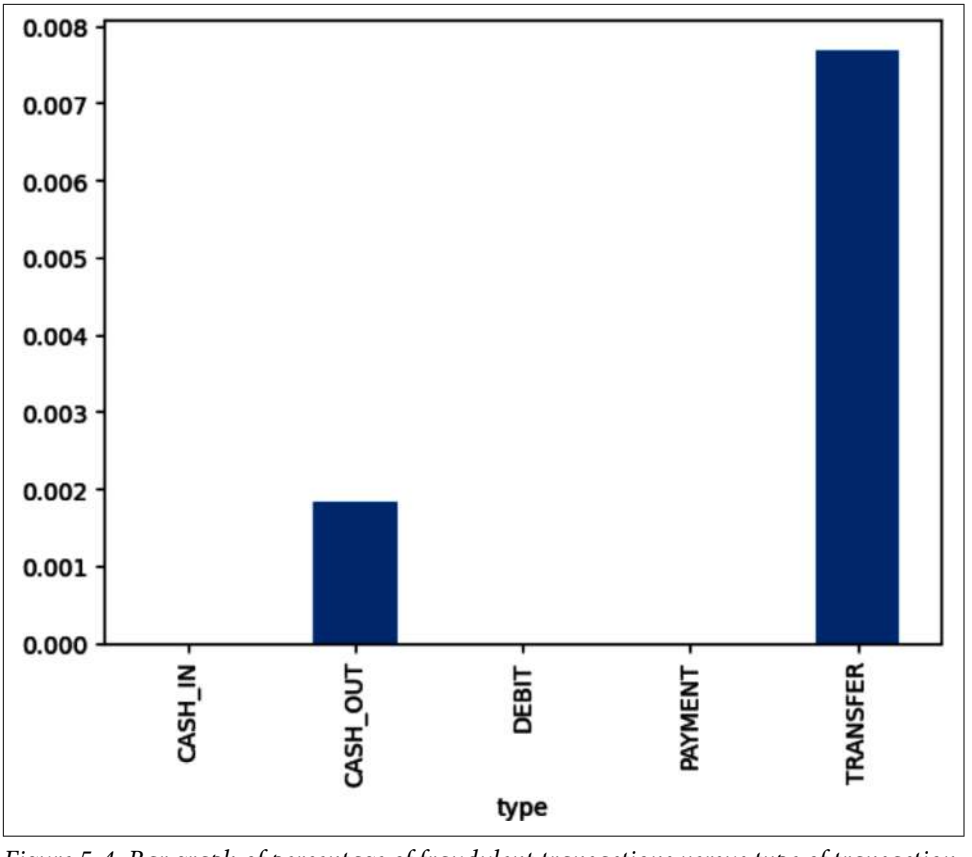


Figure 5-4. Bar graph of percentage of fraudulent transactions versus type of transaction.

You should notice something interesting immediately. There are only two transaction types in your dataset that have fraudulent transactions: `CASH_OUT` and `TRANSFER`. You can confirm this with the following line of code:

```
transaction_df.groupby('type')['isFraud'].value_counts()
```

That outputs the number of times each value combination appears. The output you should see will be similar to Table 5-4.

Table 5-4. Output from the `value_counts()` function

type	isFraud	
CASH_IN	0	1399284
CASH_OUT	0	2233384
	1	4116
DEBIT	0	41432
PAYMENT	0	2151495
TRANSFER	0	528812
	1	4097

In this case you can confirm that fraudulent transactions only appear for the transaction types mentioned. That means—as long as this dataset is really representative of the transactions your company sees—this could be a very useful feature. However, if fraudulent transactions of new types start appearing, your model will almost certainly miss them because it never saw such an example.



It will be important to monitor the transaction types for fraudulent transactions over time and consider retraining the model as needed. This conversation around model monitoring and continuous training is beyond the scope of this book, but it's certainly a concept worth being aware of.

Another useful way to visualize data with categorical labels is to bucketize the numeric features and see for each bucket what percentage of rows is fraudulent versus legitimate. *Bucketization*, or *discretization*, is the process of splitting a numeric variable into value ranges or "buckets" and assigning each element to one of the buckets. You did this in Chapter 4 when you created histograms as well. You can do this for exploring the amount of the transaction and the new and old balance features for the originator and destination accounts.

For example, suppose you want to visualize the percentage of different ranges of the `amount` feature:

```
transaction_df['amountBkts'] = pd.qcut(transaction_df['amount'], 10)
transaction_df.groupby('amountBkts')['isFraud'].mean().plot.bar()
```

Before looking at the output, take a moment to parse that first line of code. You are creating a new column, `amountBkts`, in your DataFrame that will contain the bucket information. The `pd.qcut()` function assigns buckets based on quantiles. You are bucketizing the `amount` column into 10 buckets. The first bucket contains the data from the 0th to the 10th percentile, the second contains the data from the 10th to the 20th percentile, and so on (see Figure 5-5).

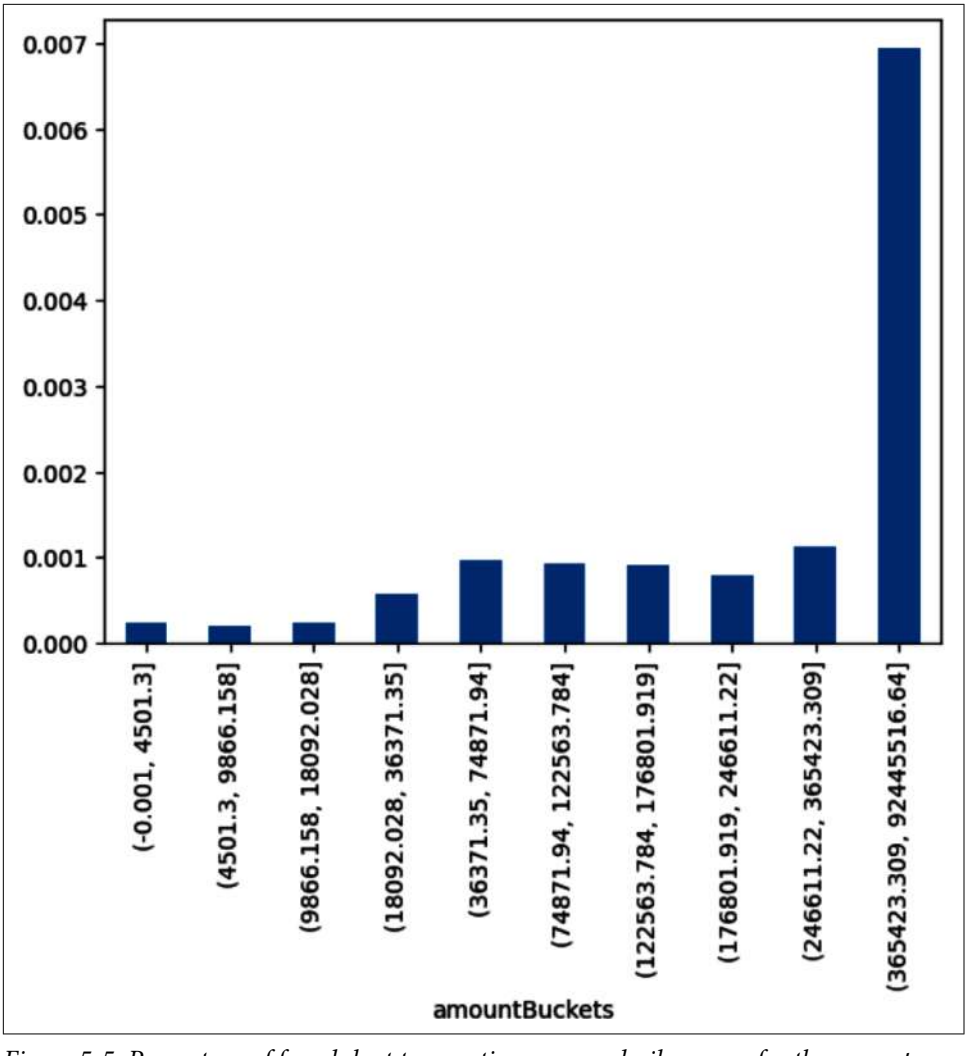


Figure 5-5. Percentage of fraudulent transactions versus decile ranges for the `amount` column.

You can see a general trend here that hints at something you may have intuitively considered. The larger the transaction amount, the more likely it is that the transaction is fraudulent. In particular, you see that there is a sevenfold jump in the fraudulent transaction rate for transactions with an amount above the 90th percentile.

To look at the same visualization for the `oldbalanceOrg` column, add the following code into a new cell and execute that cell:

```
transaction_df['oldbalanceOrgBkts'] = pd.qcut(transaction_df['oldbalanceOrg'],
                                              10, duplicates='drop')
transaction_df.groupby('oldbalanceOrgBkts')['isFraud'].mean().plot.bar()
```

There is a new argument, `duplicates='drop'`, in the `pd.qcut()` function. This argument is here to merge duplicate buckets into a single bucket. In this case, the 30th percentile of the `oldbalanceOrg` column is 0, so the first three buckets would be identical. These buckets are merged into the 30th to 40th percentile bucket instead.

As you can see in Figure 5-6, there is definitely some sort of relationship between `oldbalanceOrg` and `isFraud`. The overall trend is that the higher the old balance in the originator account, the higher the chance of fraud—but only up to a certain point. This chance of fraud decreases once the value is in the 90th to 100th percentile bucket.

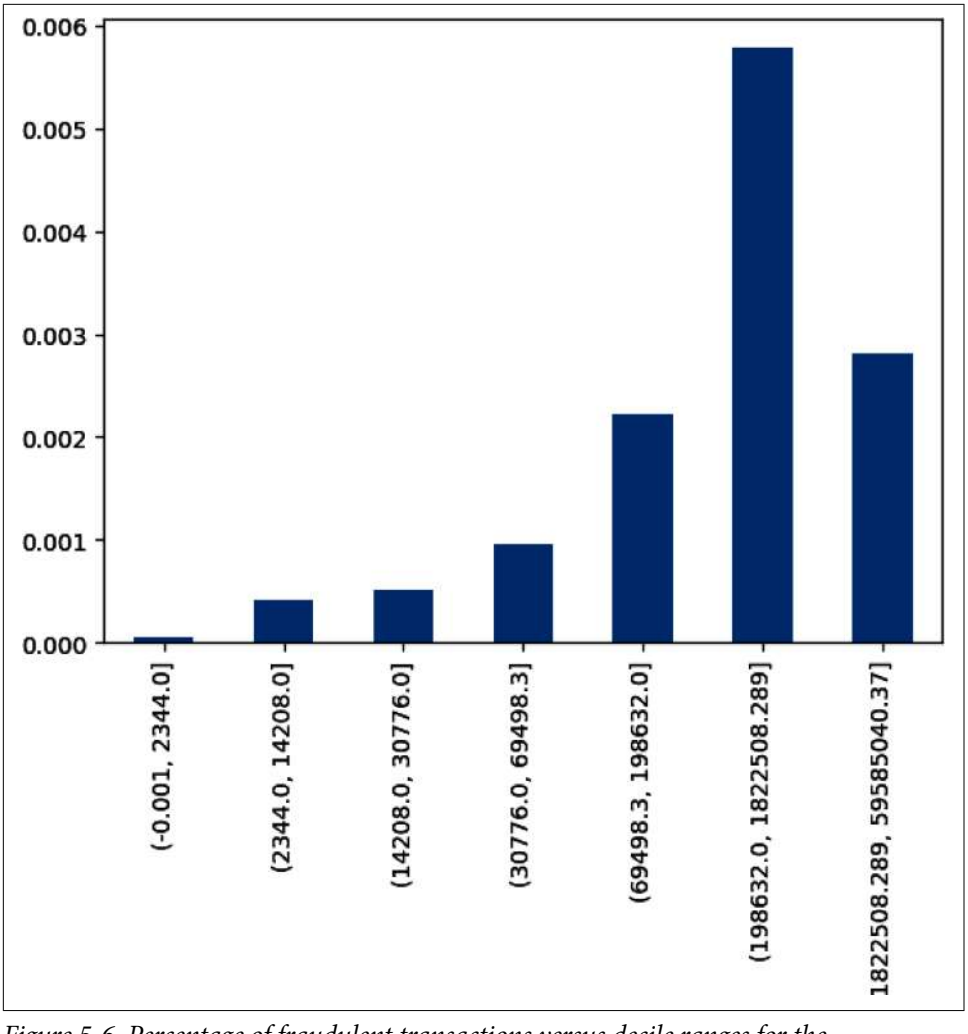


Figure 5-6. Percentage of fraudulent transactions versus decile ranges for the `oldbalanceOrg` column.

As an exercise, explore the relationship between the other numeric features and the label in a similar fashion. The code for these examples is available in the GitHub notebook, and the corresponding visualizations can be found in Figures 5-7, 5-8, and 5-9.

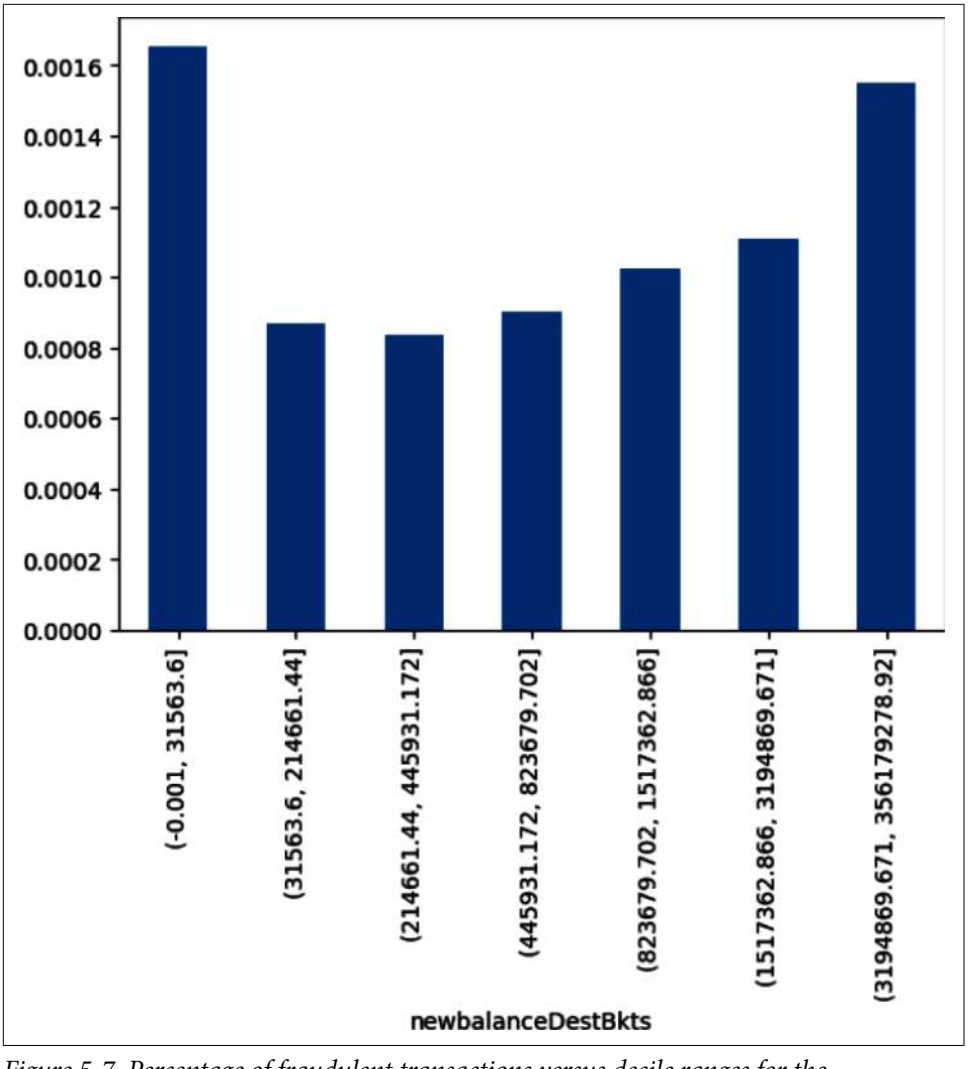


Figure 5-7. Percentage of fraudulent transactions versus decile ranges for the `newbalanceDest` *column.*

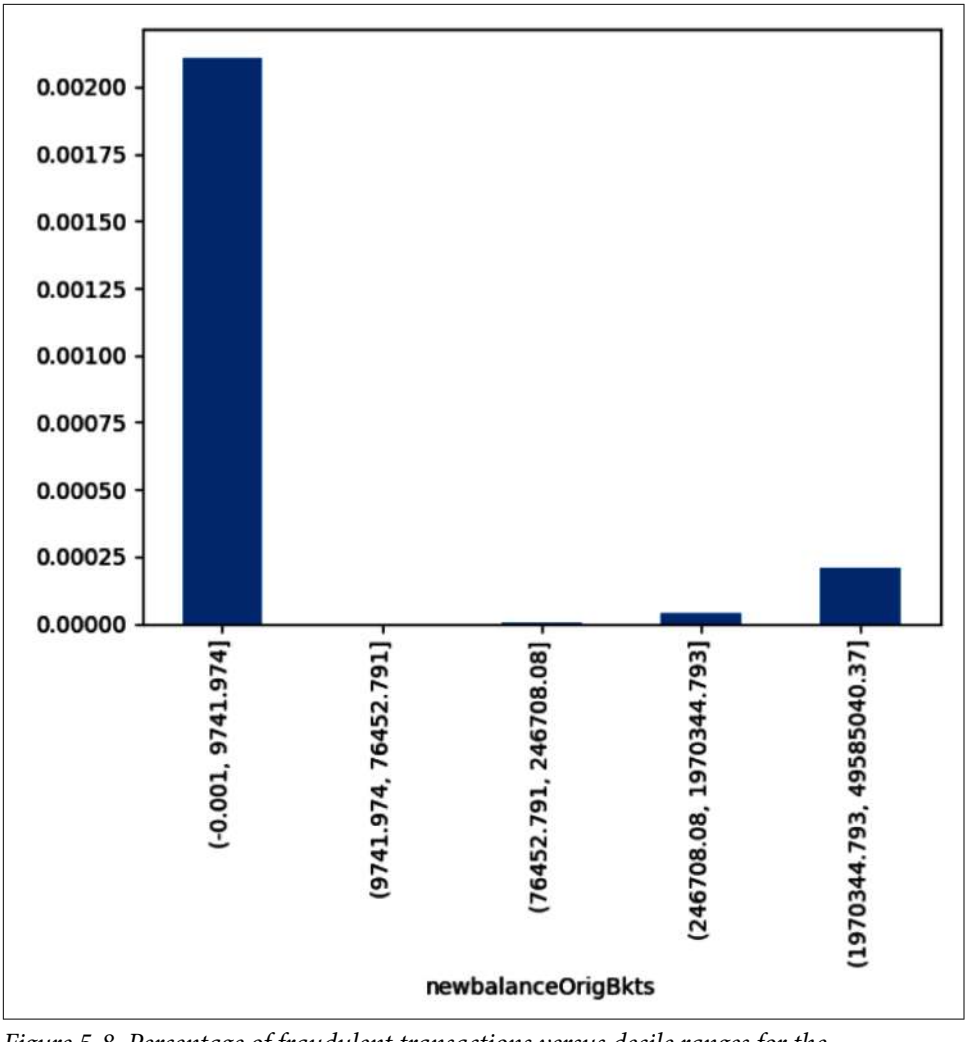


Figure 5-8. Percentage of fraudulent transactions versus decile ranges for the `newbalanceOrig` column.

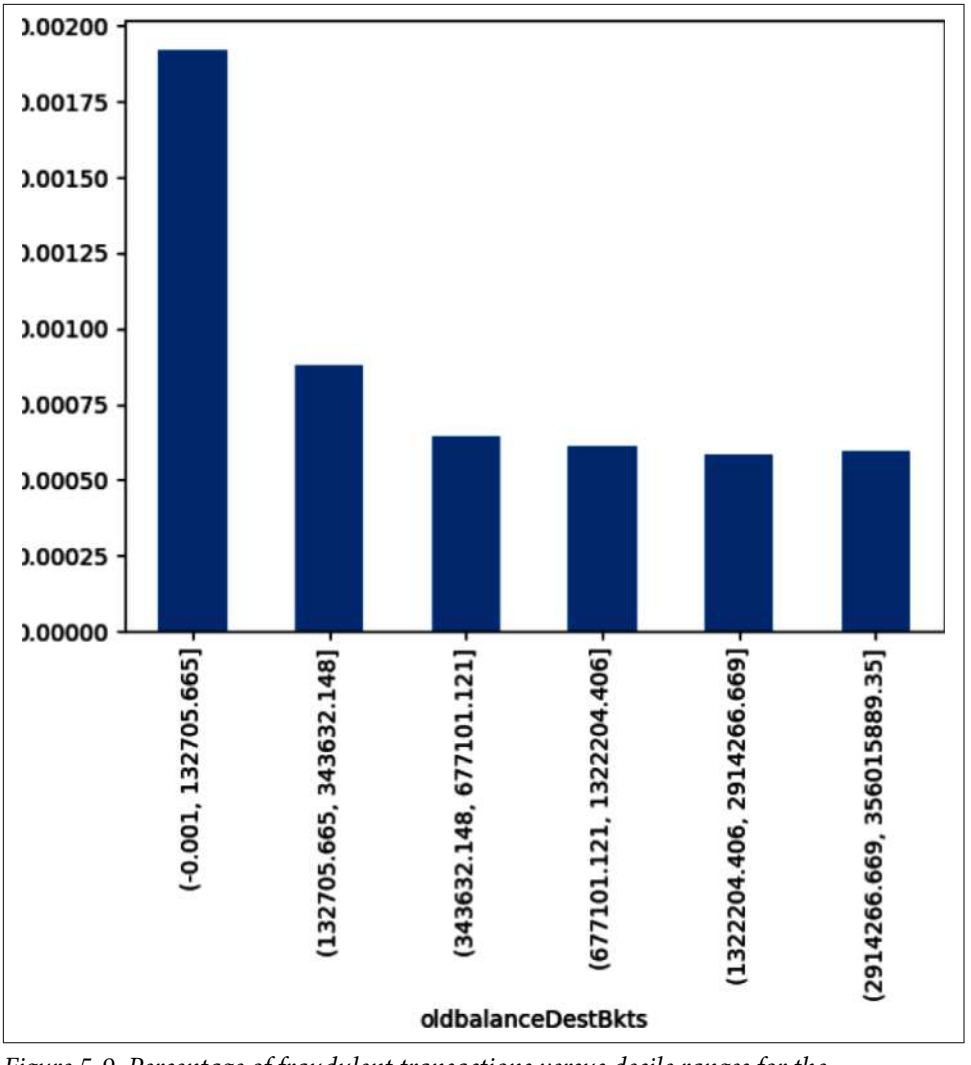


Figure 5-9. Percentage of fraudulent transactions versus decile ranges for the `oldbalanceDest` column.

There are other ways to explore transactions. For example, you know that only two types of transactions historically have been fraudulent. You can create a scatterplot for one of these types, say `CASH_OUT`, where you plot the `newbalanceDest` versus the `oldbalanceDest` and then color the points depending on whether they were fraudulent or not. Use the following code to create this visualization:

```
cashout_df = transaction_df.query("type == 'CASH_OUT' & newbalanceDest < 1e8")
cashout_df.plot.scatter(x='oldbalanceDest',
                        y='newbalanceDest',
```

```
    c='isFraud',
    colormap='YlOrRd',
    alpha=0.1)
```

The first line applies two filters. The first filter pulls rows with the `type` of `CASH_OUT` and all transactions where the `newbalanceDest` is less than `1e8` or 100,000,000. The second line is about building the scatterplot. We set the `x` and `y` columns, the column for coloring the points (`isFraud`), and a color map `YlOrRd` to make it easier to see the points. This specific color map assigns a darker color (red) to `isFraud=1` and a lighter color (yellow) to `isFraud=0`. For other color maps, see the documentation for Matplotlib. Finally, since you are plotting many points, you set `alpha=0.1` to add a little bit of transparency to the points. This makes the visualization easier to parse (see Figure 5-10).

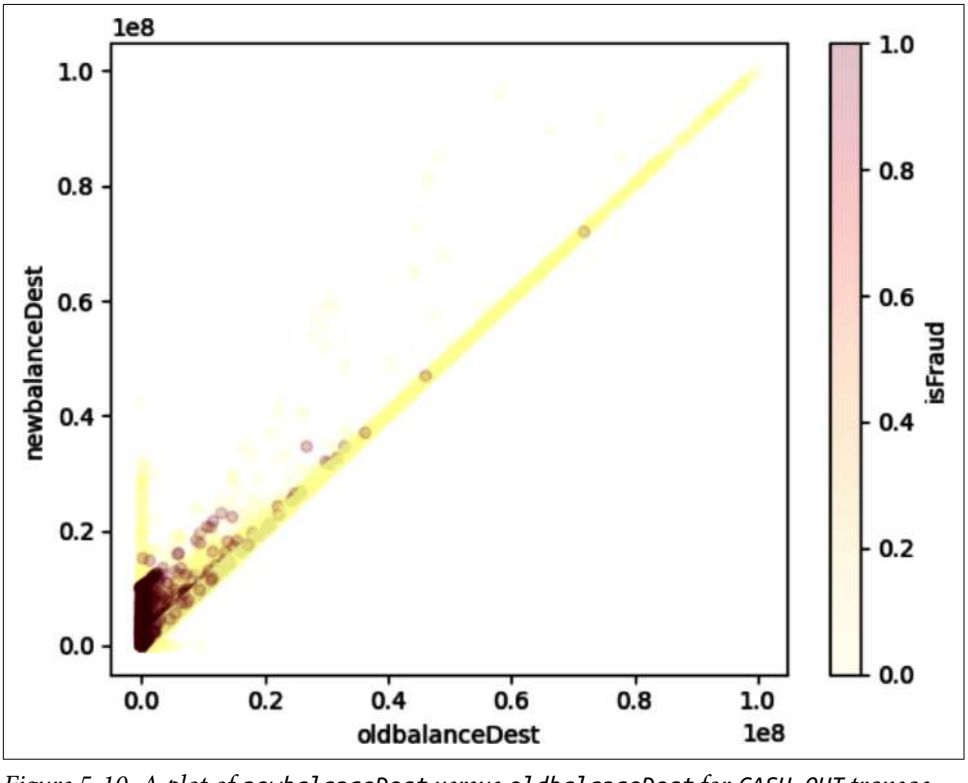


Figure 5-10. A plot of `newbalanceDest` versus `oldbalanceDest` for `CASH_OUT` transaction type colored by the value of `isFraud`.

There seems to be a region where the points are closer to red than yellow, or where there are more fraudulent transactions than legitimate transactions. This is a good sign that there is some sort of cross-correlation between these two columns. This relationship can be captured in a few different ways, such as *feature crossing*, which

AutoML will use as part of the feature selection and engineering process. Feature crossing is the process of creating synthetic features by concatenating, or crossing, two or more preexisting categorical features. You will explore manually performing this process in Chapter 8 when working with custom code models.

As an exercise, continue to visually explore the dataset using the tools you have learned about in this chapter and the previous chapter. See if you can find any interesting new insights that you may have not expected.

Exporting the Dataset

After exploring the dataset, you are ready to export it into a form that will ultimately be usable by Vertex AI AutoML. You will see a reminder here of how to do this, but note that the data is already stored in a CSV file in Google Cloud Storage and stored in a table on BigQuery, so for the next section you don't need to load the exported dataset. If you are interested in learning more about loading data into BigQuery, this will be one of the steps in the project you work through in Chapter 6.

To export data from a Pandas DataFrame to a CSV file, you can use the following code:

```
transaction_df.to_csv('transactions.csv', encoding='utf-8', index=False)
```

It is always a good practice to check the contents of the CSV file after it has been written, and you can use the `!head` command to do this:

```
!head transactions.csv
```

To download the file, follow the process from Chapter 4. Additionally, Google Colab has a function in the `google.colab` package that can also download the file for you. To use this function, import the function from the `google.colab` package and download the file to your local machine:

```
from google.colab import files
files.download('transactions.csv')
```



Though downloading files programmatically is a very convenient capability of Google Colab, this is also why it is extremely important to go through any notebook that you are running and be sure that you understand what code is being run in the notebook. You do not want to have unexpected files downloaded to your machine that could be malicious in nature, and the easiest way to avoid this is to carefully read through the code being executed.

Classification Models and Metrics

Before you begin to train the model, let's ask what exactly classification models are and how to properly evaluate them. More mathematical details will be found later in Chapter 7, but with at least a basic understanding of classification models, you can comprehend results and detect troublesome behavior from your model.

A *classification model* is a model that returns a categorical output, like *cat, dog, fish*, or *fraudulent*. In a classification model, you know up front which classes you are trying to distinguish. These classes can be numbers as well; for example, a rating or a problem where numeric labels have been used in place of categorical labels. In the case of your problem in this chapter, those classes are `0` and `1`, representing *legitimate* and *fraudulent* transactions respectively.

In practice, classification models do not return the predicted class per se, but rather return a probability for all of the possible classes. Usually, the predicted class is simply the most likely class. In the case of binary classification, where there are two possible classes, that may not be the case. For example, suppose you only classify a transaction as *fraudulent* if there is at least a 50% chance of the transaction being fraudulent from the output of the model. You may be missing cases of fraud. On the other hand, if you set that *threshold* too low, say 5%, then you may end up flagging way too many transactions as fraudulent and thus spending too many resources exploring those transactions further. As you may guess, figuring out where exactly to place the threshold is both a modeling problem and a business problem.

To help determine where to place this threshold, you can leverage different evaluation metrics to make the decision based on your business needs. Accuracy is a simple and easy-to-understand metric that is commonly used for classification models. However, you have already seen the danger in using accuracy as your main evaluation metric. After all, the original model was highly accurate, over 99%, but barely caught any of the fraudulent transactions. Accuracy is an important metric nonetheless, but it does not always give the complete picture.

These sorts of issues tend to arise when you have unbalanced classes, that is, when one class is much more common than other classes. In your case here, recall that only 0.129% of transactions were fraudulent. The ratio of fraudulent to legitimate is nearly 1:800. This scenario means that certain issues can easily be masked by metrics like accuracy. In scenarios such as fraud detection, where the *positive* class is very rare, accuracy quickly becomes unreliable on its own.

You can use metrics like recall and precision together with accuracy to get a clearer picture of your model's performance. *Recall* can be thought of as the true positive rate. In this example of fraudulent transactions, the recall represents the percentage of fraudulent transactions that the model successfully predicts. *Precision* can be thought of as the probability the model is right when it predicts a positive case. In the case

of the fraudulent transactions problem, precision represents the percentage of the predicted fraudulent transactions that were indeed fraudulent. If your goal is to proactively flag fraudulent transactions for review, recall would be a very important metric to consider. You would likely be willing to sacrifice a little bit of accuracy or precision to improve recall. Of course, this is still something you want to balance, as there is a resource cost to reviewing a large number of transactions.

Often, recall and precision are computed in terms of the confusion matrix. The *confusion matrix* breaks down the predictions into a table based on the predicted class and the actual class. An example of this is shown in Table 5-5.

Table 5-5. Confusion matrix for a general problem

	Predicted positives	Predicted negatives
Actual positives	True positives (TP)	False negatives (FN)
Actual negatives	False positives (FP)	True negatives (TN)

Recall is defined as $\frac{TP}{(FN + TP)}$, or the percentage of actual positives that was predicted as positive. Precision is defined as $\frac{TP}{(FP + TP)}$, or the percentage of predicted positives that was actually positive. As you may expect, there is a balancing act between precision and recall in practice. Having a lower threshold for the positive class (say, *fraudulent transaction*) will lead to more false positives and fewer false negatives, thus lower precision and higher recall. On the other hand, having a higher threshold for the positive class will lead to more false negatives and fewer false positives, thus higher precision and lower recall.



Which is more important: precision or recall? Many times, the answer to this question comes down to a business decision. What are your priorities? For example, if your company wants to capture fraudulent transactions as much as possible but is willing to check some legitimate transactions as well, you will likely want to favor recall as your evaluation metric.

How do you know you are finding the right balance between precision and recall? One metric is known as the *F1-score*, which is the *harmonic mean* of precision and recall. In general, harmonic means are often considered the "best average" for ratios. For two numbers it is easy to write down. Let P be the precision and R be the recall. Then the F1-score can be computed using the following formula:

$$\text{F1-score} = \frac{2(P \times R)}{P + R}$$

The F1-score tends to be skewed more toward the smaller of precision and recall. For that reason, this can be a good metric for ensuring that you don't sacrifice too much of one metric to optimize another.

Precision and recall heavily depend on the set threshold, but also on the underlying model as well. How do you decide on the underlying model when your metrics (including accuracy) depend so heavily on the threshold? There are metrics that depend only on the underlying model, such as the area under the receiver operator characteristic curve (ROC AUC) and the area under the precision-recall curve (PR AUC). Both of these metrics vary between 0 and 1. The closer in value to 1 your metric is, the better the model. A deep discussion on these metrics is beyond the scope of this chapter, but an intuitive understanding will be helpful in practice.

The ROC curve plots the recall (or the true positive rate) versus the false positive rate, which is defined as the percentage of negative examples that were classified as positive. The different points on this curve correspond to different thresholds, thus the curve as a whole sees all of the different possible thresholds. The area under this curve, the ROC AUC, can be thought of as the chance that the model (independent of threshold) gives a higher probability that a positive example is indeed positive, versus a negative example. This is a really useful metric for classification problems, but often it is not as useful for situations where the dataset is unbalanced or there is a bigger cost to a false negative than a false positive. One could reasonably argue that this is the situation that you are in here, where missing a fraudulent transaction will likely be more costly than having to check a legitimate one.

Another metric is the PR curve. The idea is the same as the ROC curve, but here you plot precision versus recall instead. The PR curve tends to be a stronger metric in the case of unbalanced datasets. Why is this? Well, both the precision and recall are focused on the positive class, and if you set the positive class to be the rarer class, then your metrics are based on the success of predicting this class rather than the dataset as a whole.

In the following section, when evaluating the model that you train in AutoML, you will see examples of both the ROC curve and the PR curve.

Using AutoML to Train a Classification Model

Now that you have explored your data, you are ready to train a model using AutoML. Similarly to Chapter 4, you will create a managed dataset on Vertex AI and then explore the dataset statistics. Then you will train the model and check model performance. Since classification metrics in general may be newer to you, you will spend some time studying those before moving on to understanding feature importances and serving predictions with your model.

Many of these steps were covered in detail in Chapter 4 for a regression problem. For that reason you will see a lighter treatment of some topics in this chapter and be referred back to Chapter 4 for more details.

Creating a Managed Dataset and Selecting the Model Objective

Go to *console.cloud.google.com* and then go to Vertex AI (either using the side menu or the search bar). From there, select Datasets, and once on the Datasets page click the Create button. Replace the autogenerated name with `fraud_detection`. Similarly to the project from Chapter 4, you first need to choose a data type and objective. Since you are working with tabular data and want to solve a classification problem, you should choose Tabular and then "Regression/classification" as shown in Figure 5-11.

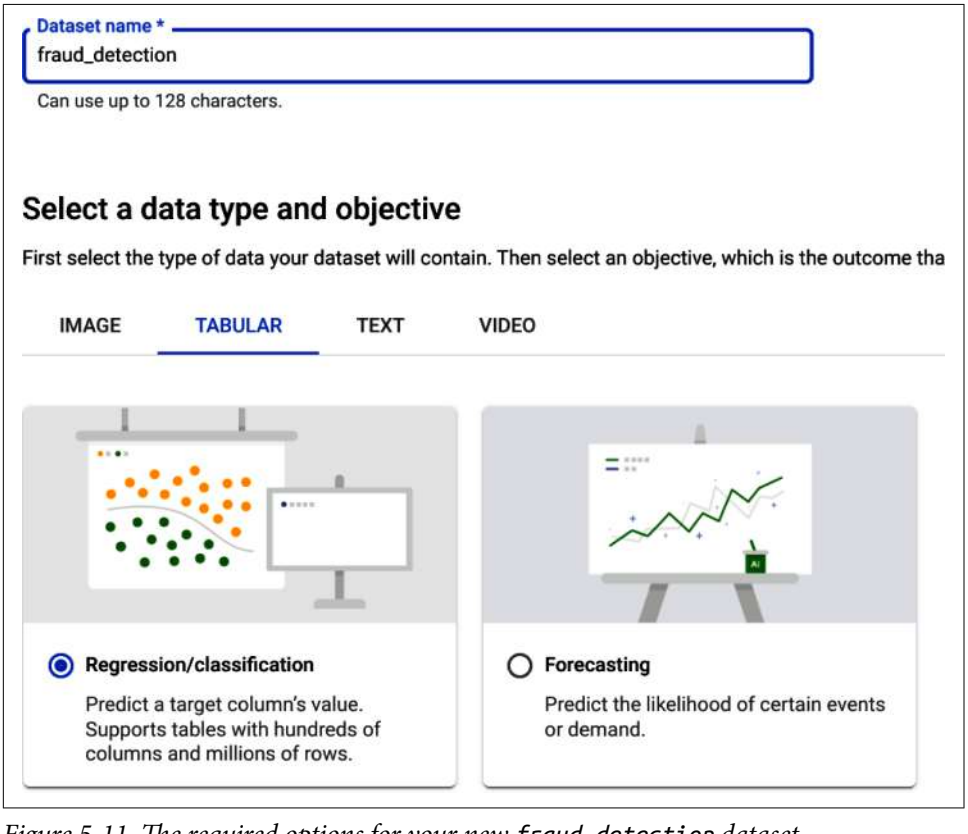


Figure 5-11. The required options for your new `fraud_detection` dataset.

Click the Create button and you will be taken to the next page, where you can specify the source data. Here you have two choices: the CSV file in Google Cloud Storage, which you used in your notebook exploration, or a table in BigQuery that contains the exact same data that was prepared for you in advance. The data stored in BigQuery will come with additional schema information—for example, the data types of each column—that the CSV does not contain, so you will use this approach.

Under "Select a data source," click the radio button for "Select a table or view from BigQuery" and then type in the following BigQuery path and hit the Continue button:

```
ma-low-code-ai.low_code_ai.financial_transactions
```

It will take around 15–20 minutes to load and preprocess the data. AutoML does a lot of additional preprocessing to analyze and prepare the data for the upcoming training job. Afterward you will be ready to explore the data in the Vertex AI Datasets UI.

Exploring Dataset Statistics

Once the data has been loaded and prepared, you can move on to analyzing the dataset. Click the Generate Statistics button. It will take a few minutes to generate the statistics. Once they are generated, you can click on the various columns and explore the statistics. Based on your analysis from before, there should not be any surprises, but it is always worth taking an additional look before moving forward. For example, if you click the `type` column you will see the statistics shown in Figure 5-12. You will see that there are no missing values; there are the five distinct values that you saw before and the breakdown of the number of times each value appears. The new part here should be the nice visualization to be able to quickly visually parse the portion of the total number of values each value comprises.

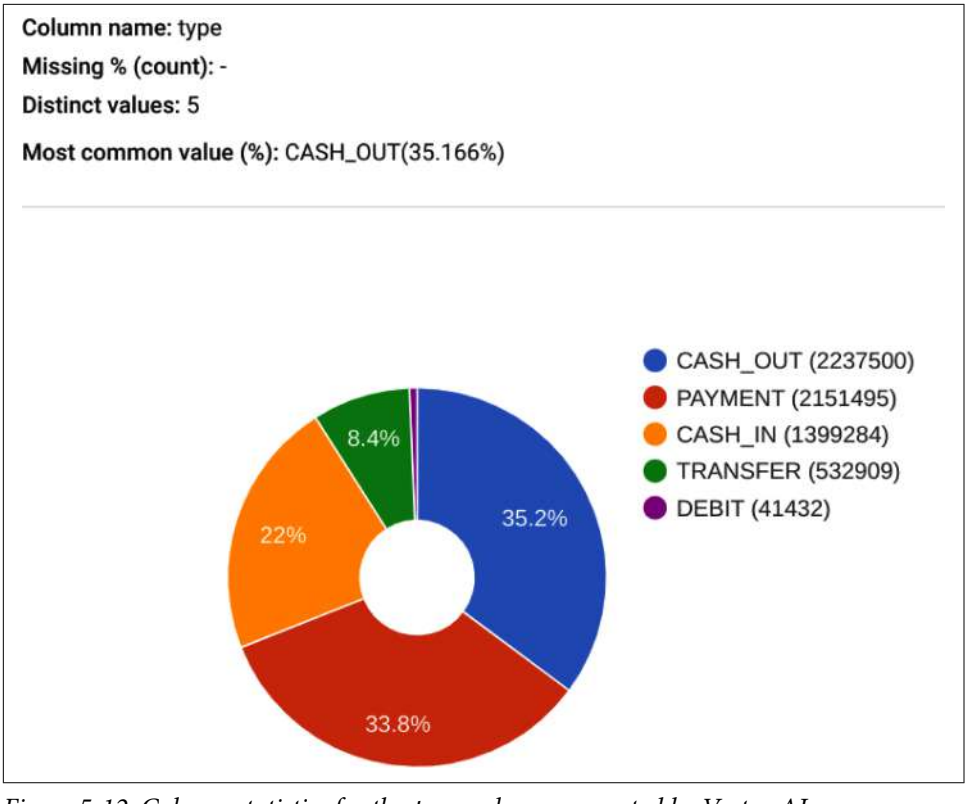


Figure 5-12. Column statistics for the `type` column generated by Vertex AI.

Before moving on to the next section, go through each column and ensure that everything looks how you expect based on the analysis you did before. Pay attention to the data types and the statistics and ensure that there are no surprises. You will not find any such surprises this time around, but developing this habit is good, and it is a best practice to be sure before you begin any training using AutoML.

Training the Model

To train your model, click the Train New Model button and select the Other option (as shown in Figure 5-13). Be sure that Classification is chosen as the objective and AutoML is selected as the model training method, and hit the Continue button.

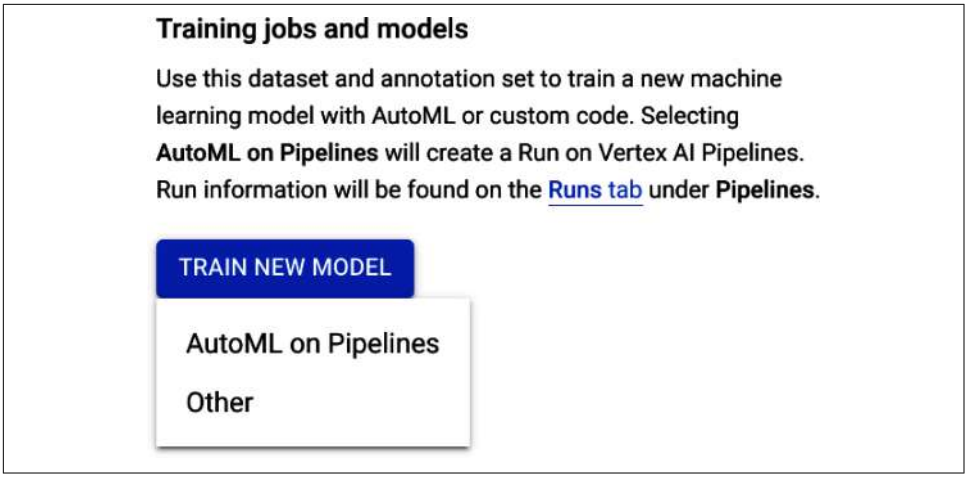


Figure 5-13. Options for training a new model in Vertex AI AutoML.

Next, on the "Model details" page, select "Train new model" and set the name of the model as `fraud_detection`. Also, choose `isFraud` as the target column. Your inputs should look like those shown in Figure 5-14. Hit the Continue button to go to the next page.

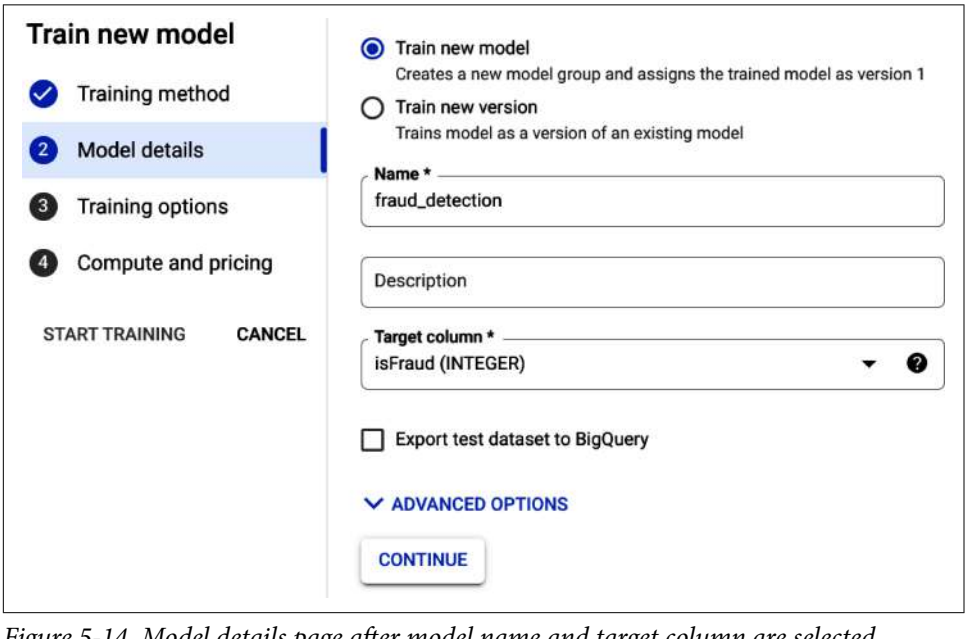


Figure 5-14. Model details page after model name and target column are selected.

On the "Training options" page, choose which features to use to train and ensure that the features are treated correctly as either categorical or numeric. On the right side of the screen, click the minus signs beside the `isFlaggedFraud`, `nameDest`, and `nameOrig` features. This will remove these features from the training process. Ensure that the `step` feature is treated as categorical as well. This will be important because this represents a time period and not a numeric value. In particular, `step` being 10 should not be counted as twice as influential on the model as `step` being 5. See Figure 5-15 to see what your page should look like before hitting the Continue button.

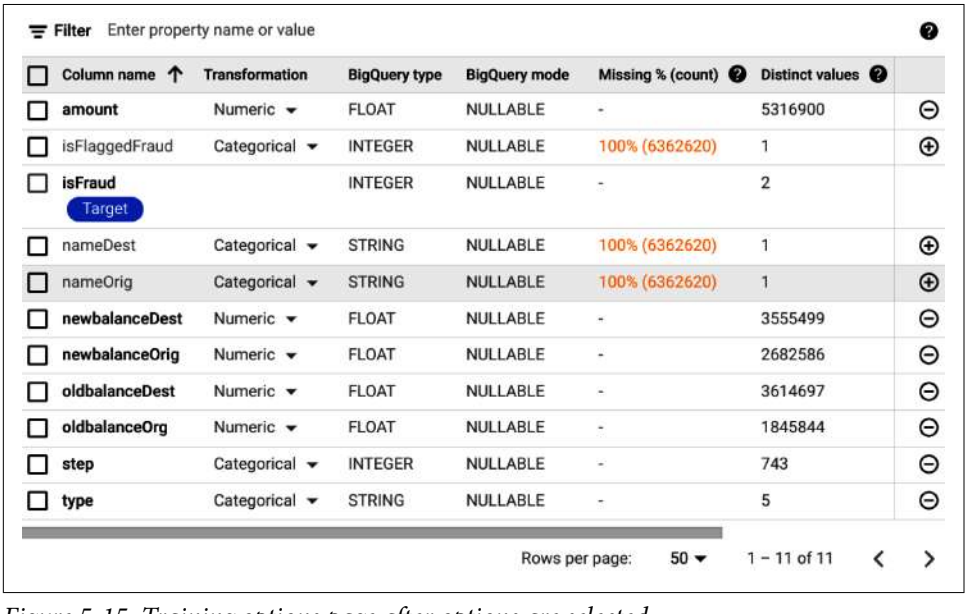


Figure 5-15. Training options page after options are selected.

On the final "Compute and pricing" page, you will set the training budget. As a note, the pricing is computed per hour of training performed. For up-to-date pricing, please see the Vertex AI documentation. With this in mind, set the budget to one hour. Be sure Early Stopping is selected since it will terminate the model training process once improvement is no longer being seen. This is very useful if you set a higher budget but ultimately end up not needing that time. Once you are done, click Start Training.

After model training is completed, you will receive an email notification, and the model is registered in the model registry.

Evaluating Model Performance

Once your model has completed the training process, you are ready to view the evaluation metrics. Navigate, either via the side menu or the search bar, to Vertex AI Model Registry. Click on your model, `fraud_detection`, and then click the corresponding version you want to view. If you are following this chapter, that will be version `1`.

Since you care most about trying to predict fraudulent transactions, click the 1 class (highlighted in Figure 5-16) to see how your model performed on the test dataset set aside by AutoML. Note that the evaluation metrics for your model may differ from the one shown in this chapter due to randomness in the model training process.

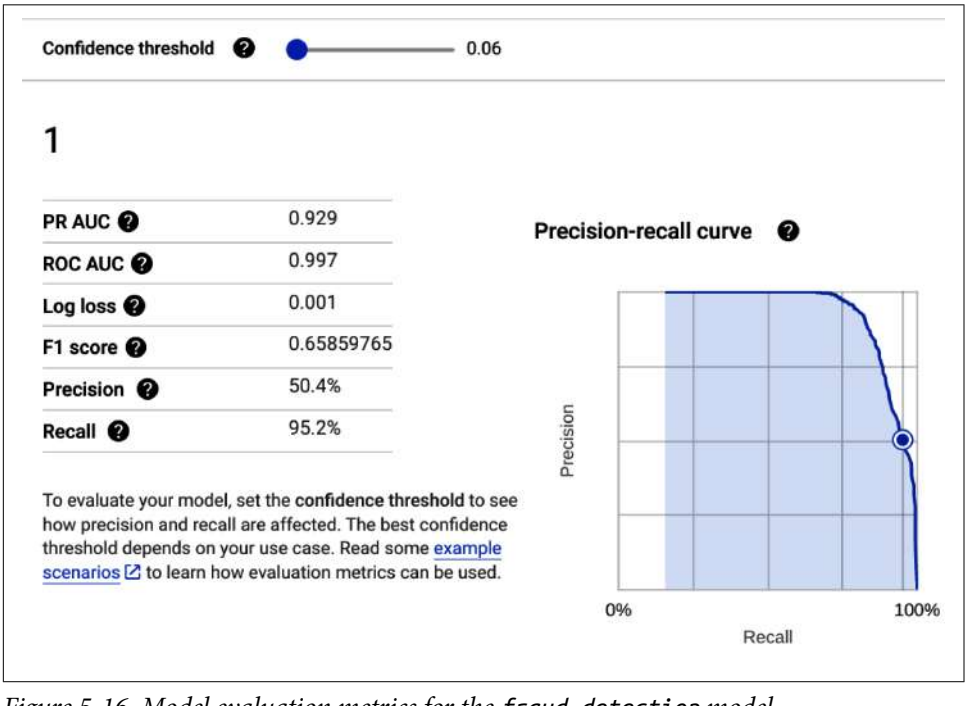


Figure 5-16. Model evaluation metrics for the `fraud_detection` model.

Note the slider for Confidence threshold. You can move this around to see how it affects the precision and recall. Suppose, for example, your company is willing to have up to 50% of flagged transactions be legitimate so that they can capture as much fraud as possible while still being able to have the resources to check the flagged transactions. For the model represented in Figure 5-16, the threshold that corresponds to 50% precision is 0.06. That is, if the model predicts that there is more than a 6% chance of fraud, you would want to flag it as fraud and check it out. Note that the recall in this scenario is 95.2%—in other words, only 1 in 20 fraudulent transactions would be expected to be missed in this scenario.

The best confidence threshold will depend on your business needs and goals, so there is no single correct answer here.

Model Feature Importances

If you click "All labels" on the side of the UI and then scroll down to the bottom of the page, you can see the feature importances for your model. Recall from Chapter 4 that feature importances measure features' impact on model predictions. Note that, as with the evaluation metrics, the exact importances you see for your model may differ from the importances that are shown in this chapter.

The feature importances for this model are shown in Figure 5-17. The `oldbalanceOrg` and `newbalanceOrig` features were the most important features, and `amount` and `type` were the third and fourth, respectively. Recall, when you were exploring your data in the notebook environment, these features seemed to have the clearest patterns between the feature value and the percentage of fraudulent transactions. In general, these are good to check to be sure that nothing is possibly odd with what your model learned. It is not uncommon for a model to notice patterns that you may not have noticed, but it should be a red flag if your model seems to have learned something nonsensical. For example, if a model to predict taxi fare learned that the distance of the ride was the least important feature, in some circumstances this could be a reason to explore your data more closely. Feature importances are very useful both to stakeholders, to understand how a model makes predictions, and to data scientists and ML engineers, to understand how to debug models in the case that surprising or nonsensical predictions are being made.

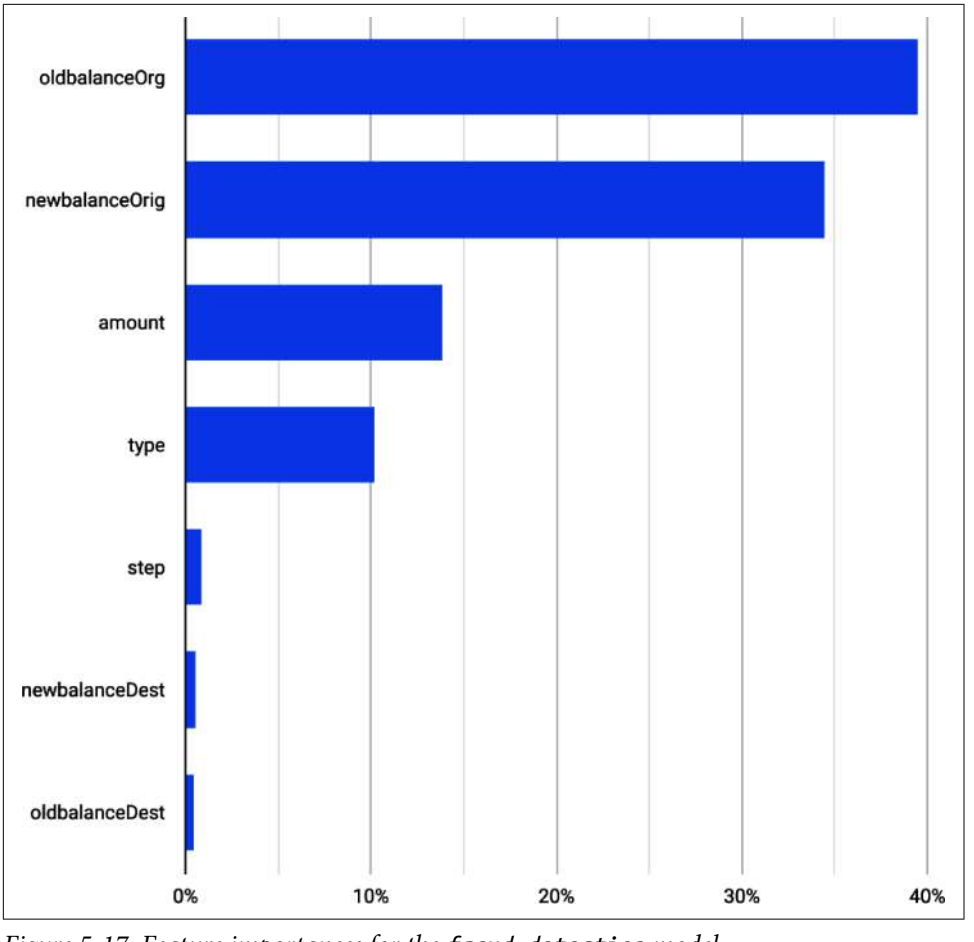


Figure 5-17. Feature importances for the `fraud_detection` model.

Getting Predictions from Your Model

Now that you have seen the evaluation metrics for your model, you are ready to deploy the model for prediction. If you get stuck in the process for deploying your model, refer back to Chapter 4, as the process in this chapter will be identical.

Complete the following steps to deploy your model, leaving all unmentioned options as the default value:

1. On the "Define your endpoint" page, name the endpoint `fraud_endpoint`.
2. On the "Model settings" page, select `n1-standard-2, 2vCPUs, 7.5GiB memory` as the Machine type. Using a smaller machine type will lower the cost of the deployed model and meet your needs in this example.
3. On the "Model settings" page, under Explainability options, click the checkbox for "Enable feature attributions for this model."
4. On the "Model monitoring" page, disable model monitoring, as you will not need it here.

After completing these steps, click Deploy. The model will take a few minutes to deploy. Once the model has finished deploying, you can test the model on the bottom of the page.

Use the following values to test your model, noting that once again, your exact results will vary from the model being shown in Figure 5-18:

`step`	`14`
`type`	`CASH_OUT`
`amount`	`1000000`
`oldbalanceOrg`	`1000000`
`newbalanceOrig`	`0`
`oldbalanceDest`	`0`
`newbalanceDest`	`0`

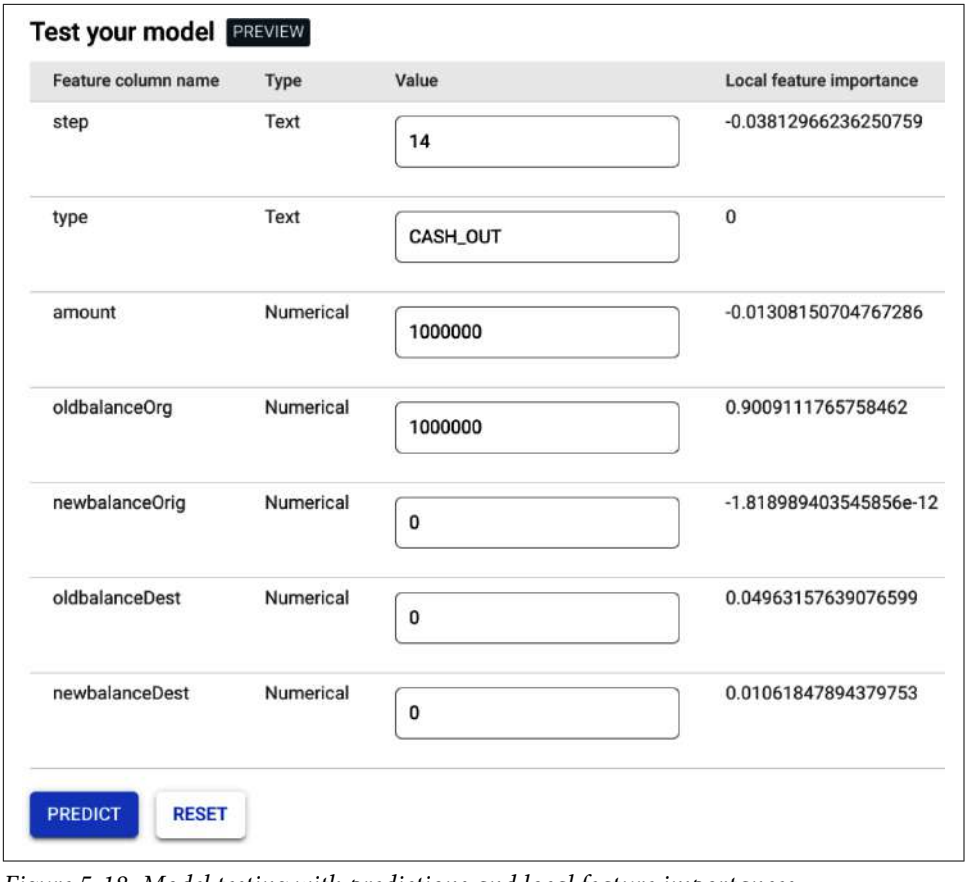


Figure 5-18. Model testing with predictions and local feature importances.

Once you are done, click the Predict button. To see the predicted probability of a fraudulent transaction from your model, change the "Selected label" to 1.

For the model shown in Figure 5-18, the "Confidence score" is the predicted probability. In this case, it gives a 91% chance of the transaction being fraudulent. Based on the conversation from before around evaluation metrics and thresholds, this transaction would be marked as fraudulent. You can also see the local feature importances. The highest feature importance value corresponds to the `oldbalanceOrg` feature. In this case, there is a `CASH_OUT` transaction for the exact balance of the account with 1,000,000 dollars in it. Based on your model, there is a high chance that this transaction is fraudulent.

As an exercise, explore different combinations of feature values and see what the predicted probabilities and corresponding feature attributions are.



Do not forget to *undeploy* your model once you are finished with this chapter. Deployed models incur cost even when they are not being used so that they are always available to return quick predictions. To undeploy the model, go to Vertex AI Endpoints, click the endpoint name `fraud_endpoint`, click the "More actions" three-dot menu (as shown in Figure 5-19), and finally click "Undeploy model from endpoint."

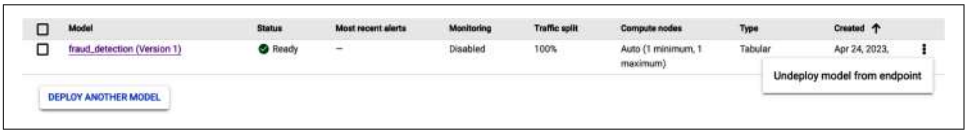


Figure 5-19. The location of the "Undeploy model from endpoint" option.

Summary

You have now trained both a regression model and a classification model over the past two chapters using AutoML! In this chapter you explored your data in a Google Colab environment, then uploaded that data to a Vertex AI managed dataset before training the classification model using AutoML. The recall of the original model was significantly lower than the recall of the new model, meaning that this new model will ultimately lead to better flagging of transactions and saving your customers' accounts.

This is just the beginning of your ML journey, though. In many cases, you will want the ability to have more control over your model training process. In practice, this means moving toward more low-code and custom code solutions. The next chapter covers how to explore data using SQL in BigQuery and train ML models using SQL in BigQuery ML.

CHAPTER 6

Using BigQuery ML to Train a Linear Regression Model

In this chapter you learn how to build a linear regression model and a neural network model from scratch to forecast power plant production. You perform this task using SQL for data analysis, Jupyter Notebook for data exploration, and BigQuery Machine Learning (BigQuery ML) for training the ML model. Along the way, you learn new techniques for understanding your data in preparation for ML and how to apply this knowledge in improving your model performance.

The Business Use Case: Power Plant Production

Your goal in this project will be to predict the net hourly electrical energy output for a combined cycle power plant (CCPP) given the weather conditions near the plant at the time.

A CCPP is composed of gas turbines, steam turbines, and heat recovery steam generators. The electricity is generated by gas and steam turbines, which are combined in one cycle, and is transferred from one turbine to another. While the vacuum is collected from the steam turbine, the other three ambient variables (temperature, ambient pressure, and relative humidity) affect the gas turbine performance.

The dataset in this section contains data points collected from a CCPP over a six-year period (2006–2011) when the power plant was set to work with a full load. The data is aggregated per hour, though the exact hour for the recorded weather conditions and energy production is not supplied in the dataset. From a practical viewpoint, this means that you will not be able to treat the data as sequence or time-series data, where you use information from previous records to predict future records.

The data is initially supplied in a CSV file,[1] so you will need to spend some time loading the data into BigQuery before you can explore it and ultimately use it to create the ML model. As you will see momentarily, there are five columns in our dataset, as shown in Table 6-1, and 9,590 rows.

Table 6-1. The five columns in the dataset: the target variable, Energy Production, is shown in bold

Column name	Minimum value	Maximum value
Temperature	1.81°C	37.11°C
Ambient pressure	992.89 millibar	1,033.30 millibar
Relative humidity	25.56%	100.16%
Exhaust vacuum	25.36 cm Hg	81.56 cm Hg
Energy production	420.26 MW	495.76 MW

These expected value ranges are well documented by the power plant engineers and have been shared with you (say, via a technical report). This will be helpful when exploring the data to ensure that there are no issues, such as null values or magic numbers, as discussed in Chapter 4.

Cleaning the Dataset Using SQL in BigQuery

As discussed before, it is important that you understand your dataset before beginning the processing of building ML models. Recall that the quality of any ML model you train will rely heavily on the quality of the dataset being used to train the model. If the dataset is filled with erroneous data or missing values, then the ML model will not be able to learn the proper insights.

In this section, you will use SQL as the tool of choice and BigQuery as the platform. All of the SQL code from this chapter is also available in the low-code-ai repository. BigQuery is Google Cloud's solution for a serverless data warehouse. Here, *serverless* will mean that you can quickly load the data into BigQuery and begin SQL data analysis without having to provision any servers. If you are not familiar with SQL, then the "Prepare Data for Exploration" course by Google on Coursera is a great free starting point. *Learning SQL* (3rd edition) by Alan Beaulieu (O'Reilly, 2020) is a good resource for those wanting to dive deeper into using SQL.

1 This dataset was created using the Combined Cycle Power Plant dataset from UC Irvine's Machine Learning Repository. Some small changes were made to better demonstrate concepts in this chapter.